

Cracking the iOS Interview

Top 100 Interview Questions and Answers



Cracking the iOS Interview

Copyright © [2023] by [Archidev Technologies Pvt. Ltd.]

All rights reserved.

No portion of this book may be reproduced in any form without written permission from the publisher or author.

Legal Disclaimer

Welcome to "Cracking the iOS Interview," a comprehensive resource crafted by a group of experienced authors and contributors. We appreciate your trust in our content, and we want to ensure that you have a clear understanding of its nature.

The content of this ebook is intended to serve as a valuable resource for general informational purposes. While our team of dedicated authors and contributors has put in significant effort to provide accurate and reliable information, it is crucial to recognise that the field of iOS development is continuously evolving. As such, the content may not always reflect the very latest practices or standards but we are committed to continually enhancing the quality and relevance of our content periodically.

While our dedicated team of authors has made every effort to create accurate and reliable content, we cannot guarantee the absence of errors or omissions. Therefore, please note that neither the authors nor the contributors can be held responsible for any errors, omissions, or discrepancies found within this ebook. We have strived to maintain the highest standards of quality, and any unintentional errors should not diminish the overall value of the knowledge and insights shared within.

We strongly recommend that you consider this ebook as a supportive tool rather than as a substitute for seeking personalised advice or tailored solutions from qualified professionals or experts. Their insights can offer specific guidance based on your unique circumstances. We encourage you to use the ebook as a source of inspiration, education, and guidance, while ultimately making your own informed decisions.

We value and respect the intellectual property rights of our authors. Therefore, reproduction, distribution, or transmission of any part of this ebook without the prior written consent of the individual authors is strictly prohibited. However, we strongly encourage you to share links to the ebook's sales page, so that others may benefit from this valuable resource.

If you have any inquiries or concerns about this ebook or its content, we welcome your feedback on our support email team@swiftanytime.com. Our goal is to provide you with the most reliable and insightful resources to support your learning journey.

Thank you for your trust and support.

Table of Contents

Introduction	7
About Swift Anytime	9
About the Authors	10
About the Contributors	12
Swift Fundamentals	16
• What is the difference between Static and Class Variable?	
• Are lazy vars computed more than once?	
• Explain the use of defer keyword in Swift.	
• How to implement a property which is public/internal but mutation is private?	
• Explain the impact of Inheritance vs Protocol Conformance on runtime performance.	
• Explain Caseliterable protocol in Swift.	
• What is the difference between self and Self?	
• Can all type be marked as final in Swift? If so, what does it mean?	
• Explain the purpose of the 'mutating' keyword in Swift.	
• What is the difference between CFBundleVersion and CFBundleShortVersionString?	
• Explain the use of AssociatedType in Protocols.	
• What is a Memberwise initialiser and why they don't Swift classes don't have one?	
• How to create optional methods in protocol?	
• Why is Swift called a protocol-oriented programming language?	
• Explain how Swift is a type-safe language?	
• What are Subscripts in Swift?	
• Can a class inherit from a struct? If not, why?	
UIKit Fundamentals	44
• What are the different states of UIViewController?	
• What are the differences between clipsToBounds and masksToBounds?	
• What is the difference between Frame and Bound?	
• What is difference between setNeedsLayout() and layoutIfNeeded()?	
• How does Auto Layout works in iOS?	
• You have a complex layout that needs to be dynamically resized based on the device's screen size. How would you use Auto Layout to ensure that the layout is always correct.	
• How does memory usage optimisation happen in UITableView?	
• What is the difference between UIView and CALayer?	
• What does UIApplicationMain mean?	

• How do you implement dynamic type and font scaling in an iOS app? • Explain the concept of the Responder Chain in iOS. • Explain the concept of trait collections and size classes in iOS. How do they work together to create adaptive user interfaces for different devices and orientations.	
SwiftUI Fundamentals	63
• What is the difference between @StateObject and @ObservedObject in SwiftUI? • What does the @Published property wrapper do in SwiftUI? • Discuss the benefits and limitations of using SwiftUI for building user interfaces.	
Modularisation	67
• How to modularise the Codebase and why is it important? • What is Dependency Injection and what are its advantages?	
Error Handling	72
• Explain the difference between throws and rethrows in Swift. • Difference between Array and NSArray.	
Networking	77
• Explain the types of sessions and tasks supported by URLSession class. • How to track image download progress in iOS?	
Frameworks & Libraries	83
• Compare static and dynamic libraries. • What is the purpose of Core Location framework?	
App and Code Optimisation	88
• Compare UITableView and UICollectionView in terms of usage, optimisation, and performance. • What are the differences between Class and Struct in Swift? • How can you improve Swift class performance? • How would you implement an Infinite Scrolling list? • How can we fix non-smooth scrolling issues? • How to identify the causes of UI errors in iOS apps? Also, how you can improve UI performance? • How can we reduce the iOS app launch time?	
Dependency Management	102
• Difference between SPM, CocoaPods, and Carthage?	
Persistent Storage	105
• What are the different ways to persist data in iOS applications and when to use them? • What is managed object context? Define delete rules in CoreData with the use cases? • Explain the benefits and limitations of using Core Data as the primary persistence framework in an iOS application?	

Memory Management 113

- How does the ARC work in Swift?
- What is Copy-on-Write? Explain how to customise its implementation?
- How do you debug memory leaks in iOS? Can you walk me through the process of finding and fixing a memory leak in your code?

Design Patterns 121

- Which UICollectionView API implements the Factory design pattern?
- Explain Factory Design Pattern usage and what problem does this pattern solve.

Concurrency 125

- Difference between Operation vs Dispatch Queue and which one should we use.
- Explain Synchronous and Asynchronous operations in Swift.
- How does DispatchWorkItem performs actions?
- What are the different types of queues provided by GCD?
- Differentiate between dispatch group and dispatch semaphore.
- What do you mean by race conditions? What are the techniques you can use to avoid it?
- What is Thread Explosion in Swift?

Application Security 139

- What is SSL Pinning and how to implement it in iOS?
- Explain Asymmetric and Symmetric encryption.
- What are some best practices for handling user credentials in Swift?

Advanced Swift 146

- Explain Access Controls in Swift
- What is the difference between == and === in Swift?
- What is Protocol Oriented Programming? How is it beneficial for us in Unit Testing?
- How to customly create a Higher-Order function in Swift?
- Explain KVO with an example in Swift.
- What is Protocol Composition in Swift?
- What is Capture lists in Swift?
- Explain Recursive enumeration and Associated values.
- Explain @autoclosure in Swift with an example.
- Explain the concepts of delegate & protocol and notification and observer in iOS.
- Explain the purpose of Hashable and why we should inherit from Equatable.
- What are generics in Swift? Can you give an example of when you would use them?
- Difference between Generics and Opaque types in Swift.
- Difference between map and compact map in Swift.
- What is Conditional Conformance?

- Why reuseIdentifier is important in UITableView.
- What are the most effective ways to write clean code?
- What is the final keyword and how does it improve run-time performance?
- What is the difference between Static Dispatch and Dynamic Dispatch?
- Explain SOLID principles.
- What is the difference between URL Schemes and Universal Links?
- Explain Equatable, Hashable, and Comparable protocol in Swift.
- Explain the Liskov Substitution principle.

Miscellaneous 191

- What is AppClip and how is it used in iOS
- What are Property Qualifiers in Swift?
- What is APNS and how does it work?
- What are the Background modes in iOS?
- Explain how iOS app state restoration works.
- What are important factors to be considered for improving iOS Apps performance?
- Explain the UIViewRepresentable protocol and its use in SwiftUI.
- What is inout Parameter in Swift?
- How can you extend or modify the UIResponder Chain in your application?

System Design Round 208

- How to approach System Design Round?
- For any OTT platform like Amazon Prime, Create a Downloader that can download and save the video to watch offline. The user can pause, resume, delete, or cancel the video that is downloading.

Introduction

We want to extend our heartfelt gratitude to you for choosing our eBook, "Cracking the iOS Interview." Your decision to embark on this journey of self-improvement and career growth is truly commendable, and we are honoured to be a part of your endeavour.

In an ever-evolving world, the global economy has been met with unprecedented challenges over the past year. At Swift Anytime, we understand the difficulties faced by fellow developers during these times, especially when it comes to the competitive job market. To lend a helping hand and equip you with the tools you need to succeed, we have poured our expertise and passion into creating this comprehensive resource.

The purpose of "Cracking the iOS Interview" is simple: to empower iOS developers like you with the knowledge and confidence needed to excel in interviews and secure your dream job. In this eBook, we have compiled 100 of the most commonly asked iOS interview questions, along with their expertly crafted answers.

It's important to note that while we have provided a substantial collection of questions, the world of iOS development is vast and ever-changing. No exhaustive list could cover every potential question, so we took a different approach to create something exceptional. We consulted over 50 iOS experts from around the globe working at top notch companies and tapping into their invaluable insights and recommendations. By incorporating their collective wisdom and diverse perspectives, we aimed to offer you a competitive edge and a global outlook on iOS interview techniques.

"Cracking the iOS Interview" is designed for iOS developers of all levels, catering to both beginners and experienced professionals alike. Whether you are preparing for your very first interview or aiming to enhance your interview skills, this eBook is tailored to meet your needs.

To make the most of this resource, we highly recommend understanding the answers rather than simply memorising them. By grasping the underlying concepts, you'll be better equipped to present the answers in your own words during interviews. Remember, it's not about rote learning; it's about demonstrating your understanding and problem-solving capabilities.

Our eBook covers major points that you should address in your explanations. However, if you ever encounter any difficulties in understanding a concept, we encourage you to conduct thorough research. Digging deeper into the topics will not only enhance your knowledge but also help you gain

confidence in your abilities.

But that's not all! We are committed to being your constant companion on your journey to success.

Here's what you can expect from us in the future:

- **50 More Interview Questions:** As the iOS landscape continues to evolve, we will regularly update our eBook with 50 additional interview questions and answers. This ensures that you stay ahead of the curve and are well-prepared for any new challenges that come your way.
- **Behavioural Round Preparation:** Cracking technical questions is just one part of the interview process. We understand the importance of behavioural rounds, and we'll equip you with the knowledge and techniques to excel in this aspect as well. From effective communication to showcasing your teamwork skills, we've got you covered.
- **Take-Home Project Round Guidance:** Many companies include take-home projects as part of their interview process to assess practical skills. We'll guide you on how to approach and excel in these projects, ensuring you leave a lasting impression on potential employers.
- **Video Explanations for Complex Topics:** Some iOS concepts can be intricate, and we recognise that written explanations may not always suffice. That's why we'll be introducing video explanations for complex topics. Visual aids can often provide a clearer understanding, making your learning experience more effective.
- **Exclusive Monthly Newsletter:** As part of our commitment to your ongoing development, we'll be delivering an exclusive monthly newsletter right to your inbox. This newsletter will be packed with additional interview tips, industry insights, the latest trends in iOS development, and much more. It's our way of keeping you informed, inspired, and motivated throughout your career journey.

Our commitment to your success as an iOS developer goes beyond this eBook. We are dedicated to empowering you with the knowledge, skills, and confidence needed to reach new heights in your career.

About Swift Anytime

Swift Anytime is a comprehensive learning platform dedicated to supporting iOS developers of all experience levels. Our mission is to help the iOS developers enhance their skills, expand knowledge, and prepare for successful technical interviews. We take pride in being one of the largest active iOS communities in India, fostering collaboration and growth among developers.

Swift Anytime Meetup: It is an initiative which brings together the iOS community, creating a space for awareness and education on iOS development, providing a space to connect with fellow developers, gain valuable insights, and stay up to date with the latest industry trends.

Swift Anytime Newsletter: Our free [weekly curated publication](#) is filled with interesting and relevant information from the iOS development ecosystem which is packed with valuable articles, insights, and updates to fuel your growth as an iOS developer.

Swift Anytime Community: It's a [community](#) for all the iOS developers, where developers share great ideas with each other and help each other grow. This community's mission is to bring those values that helps the developers to network, contribute through their valuable experience, learn new things in the Apple ecosystem and become the best version of themselves.

Swift Anytime YouTube Channel: Dive into our engaging [YouTube channel](#), where we share videos on iOS development. Additionally, we run an insightful podcast, where experienced iOS developers share their experiences and learnings. It provides valuable resources and inspiration to further enhance your iOS development journey.

At Swift Anytime, we are committed to your success and growth as an iOS developer. Together, let's thrive in the dynamic world of iOS development.

About Authors



Mayank Gupta: Mayank is a seasoned iOS expert with a remarkable track record of six years in the industry. Having played pivotal roles in multiple startups, he is renowned for nurturing and propelling them from inception to success. As the Co-founder and CEO of Swift Anytime, Mayank continues to lead the way in revolutionizing the iOS community landscape.



Ronak Garg: Ronak is a talented iOS engineer with a passion for innovation and excellence. Graduating from IIT Banaras in 2018, he embarked on an impressive journey in the tech industry, contributing his expertise to some of India's leading online payment app companies before landing his current role as a Senior iOS Engineer at a prestigious social media platform startup.



Nitin Aggarwal: Nitin is a Lead iOS Engineer at Dainik

Bhaskar. He's passionate about developing readable, clean, maintainable source code and delivering quality iOS apps. In order to build a strong iOS community, he enjoys writing technical stuff.

About Contributors



Danijela Vrzan: Danijela was formerly a Civil Engineer who turned into a passionate programmer, drawn to the creative process of problem-solving and building something from scratch. She's been writing code, publishing articles and speaking at conferences since 2020. Actively involved in the developer community, she mentor bootcamp students and share knowledge through bi-weekly articles on her personal website.



Jared Davidson: As a self-taught iOS developer and Developer Entrepreneur, Jared has made a significant impact by running a successful YouTube channel with over 65,000 subscribers, dedicated to teaching iOS development. He is the Co-founder of Sellou, a m-commerce iOS application. He is always on the lookout for what he can do better & to build the best product/business that he can.



Pedro Rojas: Pedro is an experienced software engineer with 12+ years, specializing in Apple ecosystem mobile solutions. Formerly at Meta and HP, now an Engineering Manager at Insulet, leading a team of 7 developers. Active in the developer community with "Let Swift Podcast" and "Swift and Tips" YouTube channel.



Rodolfo RocAnswer: After finishing Law school and passing the Bar, Rodolfo realized that being a Lawyer was not his true calling. He decided to start over and follow his old passion: technology. With eight years of experience as an iOS Developer and two WWDC Scholarships under his belt, he now runs a YouTube channel, sharing his journey and deep passion for tech with others.



Thang Bao: He is an experienced iOS developer skilled in UIKit and SwiftUI, with 3 years of industry experience, and a strong track record of creating user-friendly apps. He has developed and shipped 8 successful applications across diverse fields, including

Finance, AR, Gaming, Ed-tech, and POS, benefiting thousands of users. He is passionate about continually creating innovative apps to enhance user experiences.



Mike Mikina: Ever since he was a little kid, he has been tinkering with computers, and now taking this passion to the next level by running a YouTube channel, where he teaches iOS development and shares his journey. Not stopping there, Mike is also working on his own app, dreaming of one day becoming an indie developer.



Sushobhit Jain: Sushobhit is a seasoned Sr. Software Developer with plenty of experience in Data Structures, Algorithms, Computer Science, and Mobile Applications, particularly adept in iOS, Swift, and Xcode. With over 8 years of expertise in the complete application development life cycle, he is presently making valuable contributions to the team at MakemyTrip as a Sr. iOS Developer.



Catalin Patrascu: Catalin is a seasoned iOS developer known

for his expertise in crafting maintainable iOS applications. With over 13 years of experience in the field, he has become a trusted name in the industry. Catalin specializes in sharing valuable insights on system design, refactoring, and the importance of automated testing in app development. As a freelancer, he has successfully delivered more than 50 iOS applications, showcasing his commitment to excellence and innovation in the mobile development realm.

Swift Fundamentals

Q1. What is the difference between Static and Class variable?

Answer: Static and Class variables are both similar in nature, they are used to create properties that belongs to the type(Class/Struct/Enum) not it's instances. They have one major difference i.e. Inheritance. Let's learn more about it.

Static Variable:

A static variable is defined using the `static` keyword and is associated with a specific type. This means that there is only one instance of the variable across all instances of the type, and it can be accessed using the type name instead of an instance of the type. For example:

```
struct Vehicle {  
    static var wheels: Int {  
        return 0  
    }  
}  
  
print(Vehicle.wheels) // 0
```

Class Variable

A class variable is defined using the `class` keyword and is also associated with a specific type like a static variable. However, unlike a static variable, a class variable can be overridden by subclasses. For example:

```
class Vehicle {  
    class var wheels: Int {  
        return 0  
    }  
}  
  
class Car: Vehicle {  
    override class var wheels: Int {  
        return 4  
    }  
}
```

```

}

class Bike: Vehicle {
    override class var wheels: Int {
        return 2
    }
}

print(Vehicle.wheels) // 0
print(Car.wheels) // 4
print(Bike.wheels) // 2

```

Q2. Are lazy vars computed more than once?

Answer: In most cases, Lazy variable is initialised only when it is accessed for the first time. The initialisation code is executed only once, and the result is stored in the variable. Subsequent accesses to the variable return the stored value, without recomputing it.

Here is an example:

```

class Student {
    lazy var defaultMark: Int = {
        print("Computing lazy variable defaultMark")
        return 30
    }()
}

let student = Student()
print(student.defaultMark)
print(student.defaultMark)

```

Here is the output:

```

Computing lazy variable defaultMark
30
30

```

In this example, `defaultMark` is computed only once, when it is first accessed. The print statement inside the closure is executed only once, and subsequent accesses to `defaultMark` return the stored value without recomputing it.

Important Note: Lazy variables are not thread-safe by default, which means that if a lazy property is not yet initialized and multiple threads try to access it, there is a possibility that it will be initialized more than once. However, if explicit thread-safe practices are implemented on the lazy property, such cases can be avoided.

Here are some important points to remember about lazy variables in Swift:

- The lazy variable must be declared as a variable with the `lazy` keyword written before it.
- The lazy variable's type must be explicitly declared, or it must be inferable from the initialisation closure.
- After the lazy variable has been computed, its value is stored, so subsequent accesses to the variable return the stored value without recomputing it.
- Lazy variables are useful when the initialisation code is expensive or time-consuming, and you want to avoid unnecessary computation. However, they should be used with caution, as they can add complexity and make the code harder to understand.
- Lazy variables are not suitable for cases where the initialisation code has side effects or modifies state, as the side effects or state changes will occur only once, when the variable is first accessed.

Q3. Explain the use of the defer keyword in Swift.

Answer: The `defer` keyword was introduced in Swift 2.0. In a defer statement, code is executed before program control is transferred outside of the scope where the statement appears.

The defer statement is not something you will need to use often in Swift. It will be helpful in situations where you want to clean resources before they go out of scope.

Here is an example:

```
var languages = ["Swift", "Objective-C", "Kotlin", "JavaScript", "Java"]
```

```

func removeLastValue() -> String? {
    let lastValue = languages.last
    defer {
        languages.removeLast()
    }
    return lastValue
}

let lastValue = removeLastValue()
print("last value: \(lastValue ?? "")")
print("Array: \(languages)")

```

Here is the output:

```

last value: Optional("Java")
Array: ["Swift", "Objective-C", "Kotlin", "JavaScript"]

```

You can see the output and see that `languages.removeLast()` is written before the `return` statement in the above example. After completion of the execution of the function, the `defer` statement will be executed and remove the last element.

Multiple Defer Statements:

If multiple `defer` statements appear in the same scope, they are executed in the reverse order of their appearance. The last defined statement is the first to be executed. Here is an example:

```

func testingMultipleDefer() {
    defer { print("one") }
    defer { print("two") }
    defer { print("three") }
    print("end of function")
}

testingMultipleDefer()

```

Here is the output:

```
end of function
three
two
one
```

✂️ Anytime Magic Tip:

- A defer shouldn't be used to exit the current scope because `return` is ideally used for that.
- Before leaving the current scope, perform such tasks (like cleaning up the resources) in the defer block.
- It is actually guaranteed to be executed regardless of how execution leaves the current scope.

Q4. How to implement a property which is public/internal but mutation is private?

Answer: In Swift, you can use the `private(set)` keyword to make a property's setter private, and the `public` keyword to make its getter public.

Here's an example:

```
class Student {

    var name: String
    private(set) public var age: Int

    init(name: String, age: Int) {
        self.name = name
        self.age = age
    }
}

let robert = Student(name: "Robert Martin", age: 15)

// accessible because of the public getter
print(robert.age) // print: 15
```

```
// assigning or changing a value is not allowed
robert.age = 16 // error: cannot assign to property: 'age' setter is
inaccessible
```

In the above example, we set the `age` property setter as private using `private(set)`. It limits the scope of mutating this property in the `Student` class only. On the other hand, accessing `age` property is still available outside the scope of the `Student` class.

✨ Anytime Magic Tip:

- By making the setter private, you ensure that the property's value can only be changed within the scope of class, and not from outside the class. This can help you enforce encapsulation and maintain the internal state of your class.
- If a class has a critical piece of data that should never be changed from outside the class, making the setter private will prevent accidental or malicious modification.

Q5. Explain the impact of Inheritance vs Protocol Conformance on runtime performance.

Answer:

Inheritance:

Inheritance can have both positive and negative impacts on runtime performance:

Positive Impact:

- **Method Lookups:** Inherited methods are available directly in the subclass, leading to faster method lookups. This is because the runtime can quickly find and invoke the inherited method without any additional indirection.

Negative Impact:

- **Method Dispatch:** Inherited methods are resolved at compile-time or linked-time, which means the exact method to be called is determined early in the program's lifecycle. This can limit dynamic dispatch at runtime, affecting polymorphism.
- **Class Size:** Inherited properties and methods increase the size of the class, which could impact memory consumption, especially if the superclass is large.
- **Tight Coupling:** Inheritance creates tight coupling between classes, making it harder to change the implementation of a superclass without affecting its subclasses.

Protocol Conformance:

Protocol Conformance generally has a positive impact on runtime performance:

Positive Impact:

- **Dynamic Dispatch:** Protocol conformance allows dynamic dispatch at runtime, providing more flexibility and extensibility to the codebase. Since protocols allow classes to conform at runtime, method resolution can be determined at a later stage, enabling greater polymorphism.
- **Reduced Class Size:** Conforming to protocols does not increase the class size since the methods and properties are not inherited. Instead, they are implemented explicitly in the conforming class, keeping the class size smaller.
- **Improved Modularity:** Protocols promote loose coupling between classes, making the codebase more modular and maintainable. Classes can conform to multiple protocols independently, offering better code organization.

In general, when considering runtime performance, protocol conformance can be a preferred choice over inheritance. Protocols enable better flexibility, promote dynamic dispatch, and lead to more efficient memory usage compared to inheritance.

However, it's essential to remember that the choice between inheritance and protocol conformance

should not be solely based on performance implications. It depends on the design and architecture of your iOS application, as well as the relationships between classes and the overall maintainability and extensibility goals. Striking the right balance between inheritance and protocol conformance, along with other design patterns, is crucial to building a high-performance and scalable iOS application.

Q6. Explain CaseIterable protocol in Swift.

Answer: In Swift, you can use the `allCases` property to enumerate all cases of an enum that have `CaseIterable` protocol conformance. You will see an example of using the `CaseIterable` protocol that can help you to iterate all cases of an enum.

What is the CaseIterable protocol?

The `CaseIterable` is a protocol that is used to iterate over the enum cases. It synthesizes all enum instances automatically. Remember that this protocol cannot be applied to associated values. This protocol enables you to access the cases using a computed property called `allCases` which returns all the cases in a collection array.

You can iterate the cases using the `allCases` property provided by the enum:

```
import Foundation

enum Directions: CaseIterable {
    case north
    case south
    case east
    case west
}

Directions.allCases.forEach { direction in
    print(direction)
}
```

In this example, we have created a `Directions` enum and confirmed `CaseIterable` protocol to it. This allows you to run this enum through a `forEach` loop, with all the cases involved.

Output:

```
north
south
east
west
```

Q7. What is the difference between self and Self?

Answer: In Swift, the difference between `self` and `Self` lies in their usage and context. They are used for different purposes and represent different things:

`self`:

The `self` is used to refer to the current instance of a class, structure, or enumeration within its own instance methods or initializers. It is similar to `this` keyword in other programming languages. For example:

```
class MatchScore {

    let value: Int

    init(value: Int) {
        self.value = value
    }
}

let cricketScore = MatchScore(value: 325)
print(cricketScore.value) // print: 325
```

In the above example, we define an initializer `init(value:)` with an argument named `value` of type Integer. You can see that the initializer parameter and the class property have the same name (i.e. `value`). To assign value to the stored property `value`, we use `self.value`.

Self:

The `Self` is used to refer to the type of the current class, structure, or enumeration within its own methods or initializers. It is similar to `this` or `typeof` in other programming languages.

For example:

```
protocol ScoreCreation {
    static func create() -> Self
}

class MatchScore: ScoreCreation {

    required init() { }

    static func create() -> Self {
        return self.init()
    }
}

let scoreObject = MatchScore.create()
print(type(of: scoreObject)) // print: "MatchScore"
```

In this example, `MatchScore` conforms to the `ScoreCreation` protocol, which implements a `create()` method that returns `Self`, which is the instance of the current class.

Finally, the `type(of:)` function is used to print the type of the returned instance, which is `MatchScore`.

Q8. Can all type be marked as final in Swift? If so, what does it mean?

Answer: Yes, in Swift a type can be marked as `final`. But since `final` keyword helps in preventing a type from getting subclassed, therefore we can't use this keyword with a Value type(Struct/Enum). When a Class definition or it's method or variable is marked as `final`, it means it cannot be

subclassed or overridden by another Class or method.

This can be useful in cases where you have a class that should not be modified, such as a (Third-party library)Framework class. For example:

```
final class Person {  
    // Class implementation goes here  
}  
  
// error: inheritance from a final class 'Person'  
class Student: Person {  
  
}
```

Similarly, when a method or variable is marked as `final`, it cannot be overridden by any subclass. For example:

```
class Person {  
    final var name: String = ""  
  
    final func displayName() {}  
}  
  
class Student: Person {  
  
    // error: instance method overrides a 'final' instance method  
    override func displayName() {  
  
    }  
}
```

Using `final` keyword in required modules makes the code more performant since compiler tries to evaluate the project architecture to get a final picture of the build. It takes care of ensuring that if you sub-classed a BaseClass methods or variables, and didn't overridden them then it infers to define that method or variable with `final` prefix.

You can use `SwiftLint` to ensure that you are following all good practices while coding your project. It shows warnings wherever there's a scope of improvement in your codebase.

Q9. Explain the purpose of the mutating keyword in Swift.

Answer: By default, the properties of Value types like Structs cannot be mutated by the methods defined within the scope of the Struct. The `mutating` keyword is used as a prefix to methods that enables the mutation of instance properties.

For example, consider the following code without `mutating` keyword:

```
struct Stack {
    var numbers = [1, 2, 3]

    func append(_ number: Int) {
        numbers.append(number)
    }
}

var intStack = Stack()
intStack.append(4)
print(intStack.numbers)
```

Output:

```
error: cannot use mutating member on immutable value: 'self' is immutable
    numbers.append(number)
```

Consider the following code with `mutating` keyword:

```
struct Stack {
    var numbers = [1, 2, 3]

    mutating func append(_ number: Int) {
        numbers.append(number)
    }
}

var intStack = Stack()
```

```
intStack.append(4)
print(intStack.numbers)
```

Output:

```
[1, 2, 3, 4]
```

Anytime Magic Tip:

The mutating keyword is enforced by the Swift compiler, which ensures that a method cannot modify the state of an instance without explicitly being marked as mutating. This helps prevent unexpected changes to the state of an instance, which can lead to bugs and errors.

Q10. What is the difference between `CFBundleVersion` and `CFBundleShortVersionString`?

Answer: `CFBundleVersion` and `CFBundleShortVersionString` are two properties that are used to identify a specific version of an app.

CFBundleVersion:

`CFBundleVersion` is a numeric value (e.g. "1", "2", "3") that represents the app build version. This value is incremented every time an updated build of the app is created. This is regardless of any changes to the app's functionality or user interface. `CFBundleVersion` is used by the App Store and other distribution channels to identify and track different versions of an app.

Here is the code snippet to get it:

```
let buildVersion = Bundle.main.object(forKey: "CFBundleVersion")
as? String
print(buildVersion ?? "no_value") // print: 1
```

CFBundleShortVersionString:

CFBundleShortVersionString is a string value (e.g. "1.0", "1.1", "2.0") that represents the marketing version of the app. This value is typically used to communicate the app version to users, and is usually displayed on the app's About screen or on the App Store. CFBundleShortVersionString is not meant to identify different versions of an app - instead, it only communicates the current version to users.

Here is the code snippet to get it:

```
let appVersion = Bundle.main.object(forKey:
"CFBundleShortVersionString") as? String
print(appVersion ?? "no_value") // print: 1.0
```

Anytime Magic Tip:

Understanding the difference between these two properties is critical for app developers who want to ensure that their app is correctly identified and versioned across different distribution channels.

Q11. Explain the use of Associated Type in Protocols.

Answer: Associated Type is a way to define a placeholder type in a protocol that will be determined by the conforming types. This allows the protocol to define methods and properties that depend on a specific type, without knowing the type at the time when protocol is defined. Here's a simple example of a protocol that uses Associated Type:

```
protocol Stack {

    associatedtype Element

    var count: Int { get }
    mutating func push(_ element: Element)
    mutating func pop() -> Element?
```

```
func peek() -> Element?  
}
```

In protocol `Stack`, `Element` is an Associated type that represents the type of elements stored in the stack. Conforming types will need to provide a concrete type for `Element` when they implement this protocol. Here's an example of a struct that conforms to the `Stack` protocol:

```
struct IntStack: Stack {  
  
    typealias Element = Int  
  
    private var elements = [Int]()  
  
    var count: Int {  
        return elements.count  
    }  
  
    mutating func push(_ element: Int) {  
        elements.append(element)  
    }  
  
    mutating func pop() -> Int? {  
        return elements.popLast()  
    }  
  
    func peek() -> Int? {  
        return elements.last  
    }  
}
```

In this example, `IntStack` conforms to the `Stack` protocol by providing a concrete type for `Element` using the `typealias` keyword. The `IntStack` struct uses an array of integers to store the stack elements. Here's an example of how you can use the `IntStack` struct:

```
var stack = IntStack()  
stack.push(1)  
stack.push(2)  
stack.push(3)  
print(stack.peek()) // Output: Optional(3)  
print(stack.pop())  // Output: Optional(3)
```



```
print(stack.pop()) // Output: Optional(2)
print(stack.pop()) // Output: Optional(1)
print(stack.pop()) // Output: nil
```

By using the `Stack` protocol with `associatedtype`, we can create a generic stack implementation that can be used with any type, as long as that type conforms to the `Equatable` protocol (which is a requirement of the `contains(_:)` method). This allows us to write reusable code that can work with different types of data, without having to rewrite the code for each type.

Lastly, Associated type is a powerful feature of Swift that allows us to write flexible, reusable, and abstract code that can work with different types of data. By using Associated type, we can write more modular and type-safe code, which is easier to maintain and less tightly coupled to specific types.

Q12. What is a memberwise initialiser and why don't Swift classes have a memberwise initialiser?

Answer: A memberwise initialiser is an automatically generated initialiser by the compiler. Structs by default have a memberwise initialiser if there's no explicit declaration of `init` in its scope.

Here is an example of a struct with a memberwise initialiser in Swift:

```
struct Person {
    let name: String
    let age: Int
}

let alex = Person(name: "Alex Murphy", age: 28)
```

In `Person` struct, you can create an instance of the struct just by providing values for `name` and `age` using memberwise initialiser.

Note: Memberwise initialisers don't have an access level higher than `internal`. This means that we can only use Memberwise initialisers internally within the module in which their type is defined.

Why classes don't have a memberwise initialiser?

Classes do not have a memberwise initialiser because classes are reference type and support Inheritance. Classes have more relationships between properties and more intricate logic for setting initial property values. In many cases, it is simply not possible to initialize all properties of a class using a single, simple initializer.

For this reason, it is up to the programmer to provide custom initializers for a class. This gives the programmer the flexibility to define how a class should be initialized, including setting default values, validating property values, and more.

Additionally, having a memberwise initializer for classes could lead to a decrease in code quality, as the programmer might not be aware of the proper way to initialize all properties of the class and might end up using the memberwise initializer instead of writing custom initializers. This could lead to unexpected behavior and bugs.

Anytime Magic Tip:

In Xcode, you can generate memberwise initializers for Classes without having to write code. Simply right-click the Class name and choose **Refactor --> Generate Memberwise Initializer**

Some Related Questions:

- What changes with Memberwise initializers when Struct has private objects?

Q13. How to create Optional methods in Protocol?

Answer: To create optional methods in a protocol, Swift has two approaches. Each approach has its own advantages and disadvantages. Let's understand them with an example.

1. Use the optional keyword:

You can create optional methods in a protocol by using the `optional` keyword before the method declaration. The protocol can then be adopted by a class and the method does not need to be necessarily implemented.

Here's an example:

```
import UIKit

@objc protocol CountryPickerDelegate {

    // required method
    func didCountrySelected(at index: Int)

    // optional method
    @objc optional func didCountryPickerDismiss()
}

class RegisterController: UIViewController, CountryPickerDelegate {

    // MARK: - LifeCycle
    override func viewDidLoad() {
        super.viewDidLoad()
    }

    // MARK: - CountryPickerDelegate
    func didCountrySelected(at index: Int) {
        // write implementation here..
    }
}
```

Note: This approach can be used only for classes that inherit from NSObject. That means you cannot conform structs or enums to the protocol.

2. Use the Extension:

By providing an empty implementation of methods in a protocol extension, you can make them optional.

Here's an example:

```
protocol UIPickerViewDelegate {  
  
    // required method  
    func didCountrySelected(at index: Int)  
  
    // need to be optional  
    func didCountryPickerDismiss()  
}  
  
extension UIPickerViewDelegate {  
  
    // empty implementation  
    func didCountryPickerDismiss() { }  
}
```

Note: This approach can be used with all the types conforming to this protocol.

Anytime Magic Tip:

By default, all the methods of a protocol are required. It is always recommended to make optional methods in order to avoid necessary implementation of all the methods even when it's not required.

Q14. Why is Swift called a Protocol-Oriented Programming language?

Answer: Swift is called a protocol-oriented programming language because it places a strong emphasis on the use of protocols to define interfaces between different types in a program.

A protocol is a set of rules that defines a blueprint of methods, properties, and other requirements that a type can conform to provide specific functionality. Protocols are similar to interfaces in other programming languages.

By using protocols, Swift encourages developers to create flexible, reusable code that can be easily extended and adapted to different use cases. By defining protocols, developers can create a contract that specifies what a type can do, without worrying about implementation details. This makes it easier to create modular, composable code that can be easily combined and tested.

In addition to supporting traditional object-oriented programming, Swift also provides support for value types like structs and enums, which can also adopt protocols.

Note: The protocol-oriented approach in Swift promotes best practices for software design and helps developers create more robust, scalable, and maintainable code.

Let's understand how protocols are a powerful feature in Swift with an example:

Let's say we're creating an app that includes various types of shapes, such as circles, rectangles, and triangles. We could define a `Shape` protocol that requires conforming types to provide methods for calculating the area and perimeter of the shape, as well as a property to store the shape's color:

```
protocol Shape {
    var color: UIColor { get }
    func area() -> Double
    func perimeter() -> Double
}
```

Now we can create a Circle type that conforms to the Shape protocol:

```
struct Circle: Shape {

    var color: UIColor
```

```

var radius: Double

func area() -> Double {
    return Double.pi * radius * radius
}

func perimeter() -> Double {
    return 2 * Double.pi * radius
}
}

```

Similarly, we can create a Rectangle type that also conforms to the Shape protocol:

```

struct Rectangle: Shape {

    var color: UIColor
    var width: Double
    var height: Double

    func area() -> Double {
        return width * height
    }

    func perimeter() -> Double {
        return 2 * (width + height)
    }
}

```

Now we can create instances of these types and use them as Shape objects, without worrying about their specific implementations:

```

let circle = Circle(color: .red, radius: 5)
let rectangle = Rectangle(color: .blue, width: 10, height: 5)
let shapes: [Shape] = [circle, rectangle]

for shape in shapes {
    print("AreAnswer: \(shape.area()), Perimeter: \(shape.perimeter()), Color: \

```

```
(shape.color)")  
}
```

By defining a common interface for different types, we can write generic code that can work with a variety of different objects, without being tied to their specific implementation details.

Q15. Explain how Swift is a type-safe language.

Answer: There are several reasons behind how Swift is a type-safe language.

Strong Type:

In Swift, variables must be declared with a specific type, such as `Int`, `Double`, `String`, etc. Once declared, a variable cannot be assigned a value of a different type. For example,

```
var name: String = "John"  
name = 12 // error: cannot assign value of type 'Int' to type 'String'
```

In the above example, you have declared a variable `name` of string type. It will not be possible to assign a value of a different type later on.

Type Inference:

Swift also provides type inference, which means that you don't always have to explicitly specify the type of a variable. The compiler can automatically determine the type based on the value assigned to it. For example,

```
var name = "John"  
name = 12 // error: cannot assign value of type 'Int' to type 'String'
```

In the above example, you have assigned a string value to the variable `name`. It will not be possible to assign a value of a different type later on.

Type Casting:

In some cases, you may need to convert a value from one type to another. However, if the conversion fails, the program will crash at runtime. You can take precautions to prevent crashing the app. For example:

```
let number = "1990"

// We can use optional binding here to prevent crashing the app.
if let intValue = Int(number) {
    print(intValue) // print: 1990
}
```

Optional types:

In Swift, variables can be declared as optional, which means they can either contain a value or be nil. This helps prevent errors caused by the use of uninitialized variables. For example:

```
var number: Int?

// We can use optional binding here to prevent crashing the app.
if let intValue = number {
    print(intValue)
} else {
    print("Value does not found") // print: Value does not found
}
```

In the above example, you can check for a valid value before using it instead of using it directly in the code.

Generics:

Swift also has support for generics, which allow you to write flexible functions and types that work with any type, while still maintaining type safety. For example:


```
func swapping<T>(_ Answer: inout T, _ b: inout T) {
    let temp = a
    a = b
    b = temp
}

var number1 = 16
var number2 = 32
swapping(&number1, &number2)
print(number1, number2) // Output: 32 16
```

Q16. What are Subscripts in Swift?

Answer: Subscripts are shortcuts to access member elements of a collection, list, or sequence. By using square brackets, you can access elements of a collection instead of calling methods. For example, you can access an element of an array by index using a subscript, like this: `array[0]`. Subscripts can be read-write or read-only. You can make a subscript read-only by omitting the `set` block, or read-write by including both the `get` and `set` blocks.

Swift's Array and Dictionary type implements subscript to read and write values for elements on certain index and key-value respectively:

```
let friends = ["Rachel", "Ross", "Joey", "Monica"]
print(friends[1])
// prints Ross

let scoreOfFriends = ["Rachel" : 1, "Ross" : 1, "Joey" : 1, "Monica" : 1]
print(scoreOfFriends["Joey"])
// prints Optional(1)
```

Counter Question:

Q: How to create a custom Subscript?

Answer: To define a subscript in Swift, you use the `subscript` keyword, followed by a set of input parameters and a return type. This is surrounded by curly brackets.

`class`, `struct` and `enum` do not provide subscripts to access elements. You have to create a custom subscript in this case.

Read-only subscript:

```
import Foundation

struct Weekdays {

    private let days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

    // return an optional type
    subscript(index: Int) -> String? {
        if index < days.count {
            return days[index]
        }
        return nil
    }
}

let week = Weekdays()
print("First day of a week: \(week[0])")
print("Third day of a week: \(week[2])")

// print: First day of a week: Optional("Mon")
// print: Third day of a week: Optional("Wed")
```

Read-write subscript:

```
extension UserDefaults {

    subscript(key: String) -> Any? {
        get {
            return object(forKey: key)
        }
        set {
            set(newValue, forKey: key)
        }
    }
}
```

```

    }
}

// saving a value using a subscript
UserDefaults.standard["user_name"] = "John Philips"

// accessing a value using a subscript
if let username = UserDefaults.standard["user_name"] as? String {
    // access username here
}

// remove a value using a subscript
UserDefaults.standard["user_name"] = nil

```

✨ Anytime Magic Tip:

- It's worthwhile to consider whether a subscript is required in a given context or not. If not, a method might be a better choice.

Q17. Can a class inherit from a struct? If not, why?

Answer: No, Swift classes cannot inherit from a struct. This is because classes and structs are fundamentally different types and have different capabilities.

Classes are reference types, which means that when an instance of a class is passed, it is actually a reference to the object, rather than the object itself. Classes also support inheritance, which allows subclasses to inherit properties and methods from their parent class.

Structs, on the other hand, are value types. This means that when an instance of a struct is passed, it is always passed as a copy of the object itself. Structs do not support inheritance, and instead rely on protocols to define a common set of properties and methods for a group of related types.

Because of these fundamental differences, a class cannot inherit from a struct in Swift.

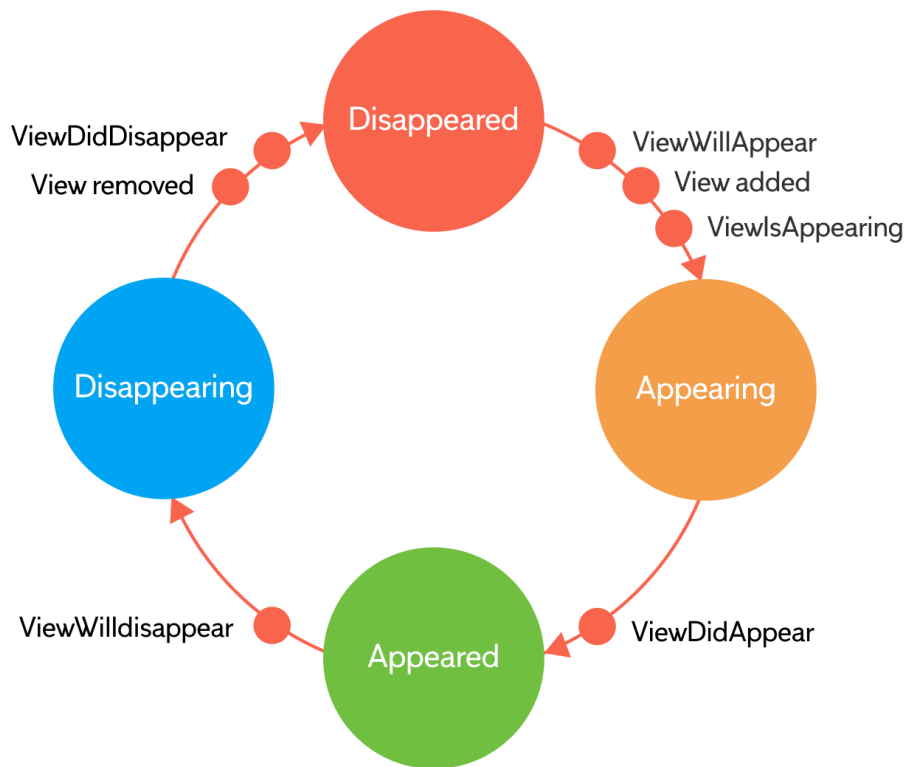
While a class cannot inherit from a struct, both struct and class can conform to a protocol. This can provide the same benefits as inheritance, such as the ability to define a common set of properties and methods for a group of related types.

UIKit Fundamentals

Q18. What are the different states of UIViewController?

Answer: While working on an application using `UIKit` framework, we often deal with the `UIViewController` classes. The components loading in a `UIViewController` go through different stages because of the different states involved in a `UIViewController`. Let's explore each state sequentially :

- **Not yet loaded:** The view controller hasn't loaded its view hierarchy into memory yet.
- **Loaded:** The view controller has loaded its view hierarchy into memory, but it may not yet be visible on the screen. In this state, the `viewDidLoad` method gets called.
- **Appearing:** The view controller is in the process of being added to the view hierarchy and is becoming visible on the screen. In this state, the `viewWillAppear` method gets called.
- **Visible:** The view controller is fully visible on the screen and has received all necessary notifications and events. In this state, the `viewDidAppear` method gets called.
- **Disappearing:** The view controller is in the process of being removed from the view hierarchy and is no longer visible on the screen. In this state, the `viewWillDisappear` method gets called.
- **Unloaded:** The view controller has been removed from the view hierarchy and its view hierarchy has been unloaded from memory. During this state, the `deinit` method might be called.



Counter Question:

Q: In which UIViewController lifecycle method you set the UITableView's dataSource and how do you reload the UITableView data?

Answer: It is recommended to set the delegate and datasource in the `viewDidLoad` method of the `UIViewController` class like below:

```
override func viewDidLoad() {  
    super.viewDidLoad()  
    tableView.delegate = self  
    tableView.dataSource = self  
}
```

To reload the table data, use `reloadData()` method of the `UITableView` instance like below:

```
tableView.reloadData()
```

It is recommended to reload the table data using `DispatchQueue.main.async` like below:

```
DispatchQueue.main.async {  
    self.tableView.reloadData()  
}
```

✨ Anytime Magic Tip:

Using `DispatchQueue.main.async` for reloading a table view in Swift ensures that the reload operation is performed on the main thread. It provides a better user experience and eliminates any potential performance issues or crashes.

Q19. What are the differences between `clipsToBounds` and `masksToBounds`?

Answer: Here's a comparison of `clipsToBounds` and `masksToBounds`:

Functionality:

The main difference between `clipsToBounds` and `masksToBounds` is their functionality. `clipsToBounds` clips the contents of a view or layer to its bounds, whereas `masksToBounds` creates a mask based on the bounds of the view or layer to hide any content outside of the desired shape.

Resulting Appearance:

The resulting appearance of a view or layer can be different depending on whether `clipsToBounds` or `masksToBounds` is used. With `clipsToBounds`, any content that extends beyond the bounds of the view or layer is simply clipped or cut off. With `masksToBounds`, any content that extends beyond the bounds of the view or layer is masked, or made invisible, according to the shape of the mask.

Performance:

There can be some differences in terms of the performance of both properties. `clipsToBounds` can be faster, as it simply clips the contents of the view or layer without any additional processing. `masksToBounds`, on the other hand, requires the creation of a mask, which can be more resource-intensive.

Use Cases:

The two properties have different use cases. `clipsToBounds` is useful when you want to ensure that the contents of a view or layer stay within its bounds, without changing the overall shape of the content. `masksToBounds` is useful when you want to create a specific shape for a view or layer, such as a circle or rounded rectangle, and hide any content that falls outside of that shape.

Property Type:

`clipsToBounds` is a property of the `UIView` class, while `masksToBounds` is a property of the `CALayer` class. This means that `clipsToBounds` can be used with any type of view, while `masksToBounds` can only be used with views that have a backing layer (such as `UIView`, `UIButton`, and `UIImageView`).

Inheritance:

If a view or layer has `clipsToBounds` set to true, any subviews or sublayers that extend beyond the bounds of the parent will also be clipped. However, if a view or layer has `masksToBounds` set to true, only the parent view or layer will be masked, and any subviews or sublayers will not be affected.

Edge Cases:

It's worth noting that there can be some edge cases where `clipsToBounds` and `masksToBounds` may not behave as expected. For example, if a view has a background color or image that extends beyond its bounds, it may still be visible even if `clipsToBounds` is turned on. In this case, `masksToBounds` may be a better option to ensure that the background content is hidden outside the

desired shape.

Overall, `clipsToBounds` and `masksToBounds` are both useful properties, and it's important to understand their differences in order to choose the right option for a given design or a functionality.

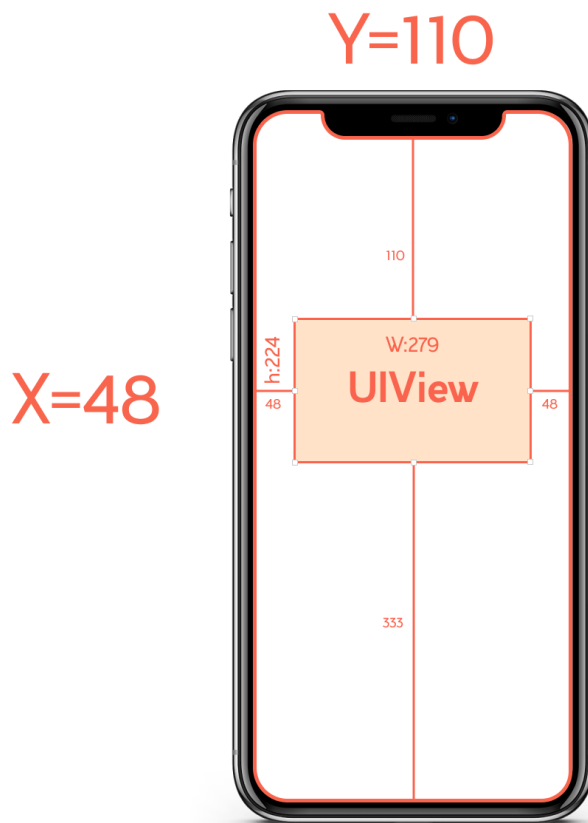
Q20. What is the difference between Frame and Bound?

Answer: A view is an object that represents a rectangular area on the screen and is responsible for drawing content and handling user interactions. The terms `frame` and `bounds` are often used when discussing the origin and dimension of views.

Frame:

The frame of the view is its origin and dimension with respect to its superview coordinate system. It's specified in points, which are a virtual unit of measurement that is independent of the device's screen resolution.

For example, if a view has a frame of (10, 10, 100, 100), it means that the view's top-left corner is positioned 10 points from the left edge of its superview and 10 points from the top edge of its superview, and its width and height are both 100 points.

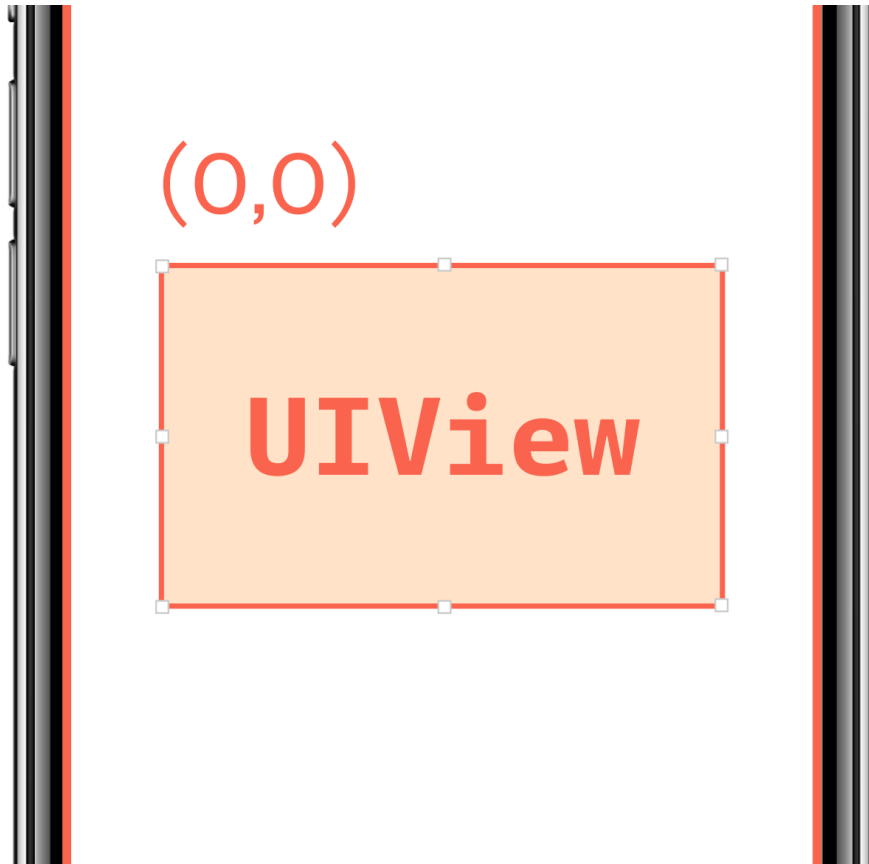


Bounds:

The bounds of a view is the rectangle that defines its position and dimension with respect to its own coordinate system. It's also specified in points. Unlike the frame, the bounds does not take view's position w.r.t its superview's coordinate system into account.

The origin of the bounds is always (0, 0), which is the top-left corner of the view. For example, if a view has a bounds of (0, 0, 100, 100), it means that the view's top-left corner is positioned at its own top-left corner, and its width and height are both 100 points.

Here's a diagram to illustrate the difference between the frame and bounds of a view:



Some Related Questions:

- When you are performing Transform for the View, will it's frame change or bounds will change?

Q21. What is the difference between `setNeedsLayout()` and `layoutIfNeeded()`?

Answer: Both `setNeedsLayout()` and `layoutIfNeeded()` methods are used to update the layout of a view and it's subviews. Though, there are some differences between the two, so let's explore that in detail.

`setNeedsLayout()`:

This method tells the system that the layout of the view is now invalid and needs to be updated. This method sets a flag on the view that tells the system that a layout update is required. However, it does

not immediately trigger a layout update. Instead, the system waits for the next layout cycle to occur (usually during the next pass of the run loop) before performing the layout update.

```
class TimerViewController: UIViewController {

    let childView = UIView(frame: CGRect(x: 0, y: 0, width: 100, height: 100))

    override func viewDidLoad() {
        super.viewDidLoad()
        view.addSubview(childView)
        childView.backgroundColor = UIColor.red
    }

    func changeColor(_ sender: UIButton) {
        childView.backgroundColor = .orange
        childView.setNeedsLayout()
    }
}
```

If you make changes to the view's layout properties (such as its frame, bounds, or center) or to the layout properties of any of its subviews while a layout update is in progress, the changes may be overridden by the layout update. To avoid this, you should make all layout changes before calling `setNeedsLayout()`.

layoutIfNeeded():

This method forces an immediate layout update, even if `setNeedsLayout()` has been called earlier but a layout update has not yet occurred. Typically, you would call this method inside an animation block to ensure that any layout changes are executed smoothly.

```
class TimerViewController: UIViewController {

    let childView = UIView(frame: CGRect(x: 0, y: 0, width: 100, height: 100))

    override func viewDidLoad() {
        super.viewDidLoad()
        view.addSubview(childView)
    }
}
```

```
func updateChildViews() {  
    UIView.animate(withDuration: 0.4) {  
        self.childView.backgroundColor = .orange  
        self.view.layoutIfNeeded()  
    }  
}
```

Counter Question:

Q: Can you explain the concept of a layout pass?

Answer: Layout pass refers to the process of recalculating the positions and sizes of views within a view hierarchy based on their constraints and the current state of the hierarchy. When a layout pass occurs, the views' frames are updated to reflect any changes in their layout. Calling `layoutIfNeeded()` triggers a layout pass to update the view hierarchy immediately.

Q: What happens if you call `layoutIfNeeded()` without calling `setNeedsLayout()`?

Answer: If you call `layoutIfNeeded()` without calling `setNeedsLayout()`, it will only perform a layout update if one is already pending. If no layout update is pending, calling `layoutIfNeeded()` will have no effect. It's important to call `setNeedsLayout()` first to ensure that the view and its subviews are marked as needing a layout update.

Q22. How does Auto Layout work on iOS?

Answer: Auto Layout is a constraint-based layout system that allows developers to create user interfaces that adapt to different screen sizes and device orientations. It works by defining a set of rules, or constraints, that describe the relationship between elements in a user interface.

Auto Layout uses a constraint solver to determine the size and position of each element based on the constraints. The solver takes into account the intrinsic size and content of each element, as well as any explicit constraints defined. This allows the layout to be flexible and adaptive, while still ensuring that all elements are positioned correctly relative to each other.

Counter Question :

Q: How can you debug layout issues in your code?

Answer: Debugging layout issues in Auto Layout can be challenging, as the system is quite complex and there are many factors that affect the layout. Here are some tips for debugging layout issues in your code:

- **Check the constraints:** Make sure all constraints are properly configured and not conflicting with each other. Check the console for any error messages related to constraints.
- **Use the visual debugger:** Xcode includes a visual debugger that allows you to see the frames and constraints of each element in your interface. Use this tool to identify any elements not positioned correctly or constraints not being applied as expected.
- **Use logging:** Add logging statements to your code to track the values of key variables and properties, such as the frame and bounds of each element. This can help you identify any unexpected values that affect the layout.
- **Simplify the interface:** If you are having trouble identifying the cause of a layout issue, try simplifying the interface by removing elements one by one until the issue disappears. This can help you narrow down the problem to a specific element or constraint.
- **Use third-party tools:** There are several third-party tools available that can help you debug layout issues in your code. For example, Reveal allows you to inspect your app's view hierarchy and constraints in real time.

Q23. You have a complex layout that needs to be dynamically resized based on the device's screen size. How would you use Auto Layout to ensure that the layout is always correct?

Answer: Auto Layout is a powerful tool in iOS development that allows you to create dynamic and responsive user interfaces that adapt to different screen sizes and orientations. To ensure your layout is correct on different devices, you can use following approaches:

Use Constraints:

You need to use constraints to specify the position and size of each element in your layout. Constraints define the relationship between different views in your layout and ensure that they are always positioned correctly relative to each other. Constraints can be set up using the Interface Builder or programmatically.

Use Size Classes:

Size classes is a way to define different layout configurations for different screen sizes and orientations. You can use size classes to specify different constraints for different screen sizes and orientations. For example, you can specify different constraints for portrait and landscape orientations, or for different screen sizes like iPhone and iPad.

Use Stack View:

Stack view is a powerful tool for creating complex layouts that adapt to different screen sizes. Stack views are container views that arrange subviews in a horizontal or vertical stack. You can use stack views to create layouts that automatically adjust their size and position based on the device's screen size.

Use Content Hugging and Compression Resistance:

Content hugging and compression resistance are properties that control how much a view can be stretched or compressed. These properties can be used to ensure that views maintain their size and shape even when the device's screen size changes.

By using these techniques, you can ensure that your layout is always correct on different devices and screen sizes. You can test your layout by using Xcode's preview window or by running your app on different simulators or devices.

Q24. How does memory usage optimization happen in UITableView?

Answer: When you create a `UITableView`, and have a large number of cells to display, it might take huge memory as tableview needs to allocate memory for each cell and if the memory usage goes beyond a certain threshold then the app will lag or crash.

For optimizing memory usage you can implement `UITableView`'s

`dequeueReusableCell(withIdentifier:for:)` method which returns a cell with the specified reuse identifier. `UITableView` uses a Queue data structure to solve the memory usage problem. When you scroll through a table view, all the cells which are moving out of the visible area are stored in queue. When they are being displayed again `cellForRowAt` method will be called and the content of that particular cell will be displayed.

There is catch when content for a particular cell is displayed, sometimes content that is not related to cell is also visible. Apple has recommended using the `prepareForReuse` method to reset or clean-up the cell content so that content not related to a particular cell is not visible.

When you use this method, you must provide a reuse identifier that corresponds to the identifier you set when you created the prototype cell in your storyboard, XIB file, or programmatically.

Here's an example usage of `dequeueReusableCell(withIdentifier:for:)`:

```
func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
UITableViewCell {
    let cell = tableView.dequeueReusableCell(withIdentifier: "MyCell", for:
```

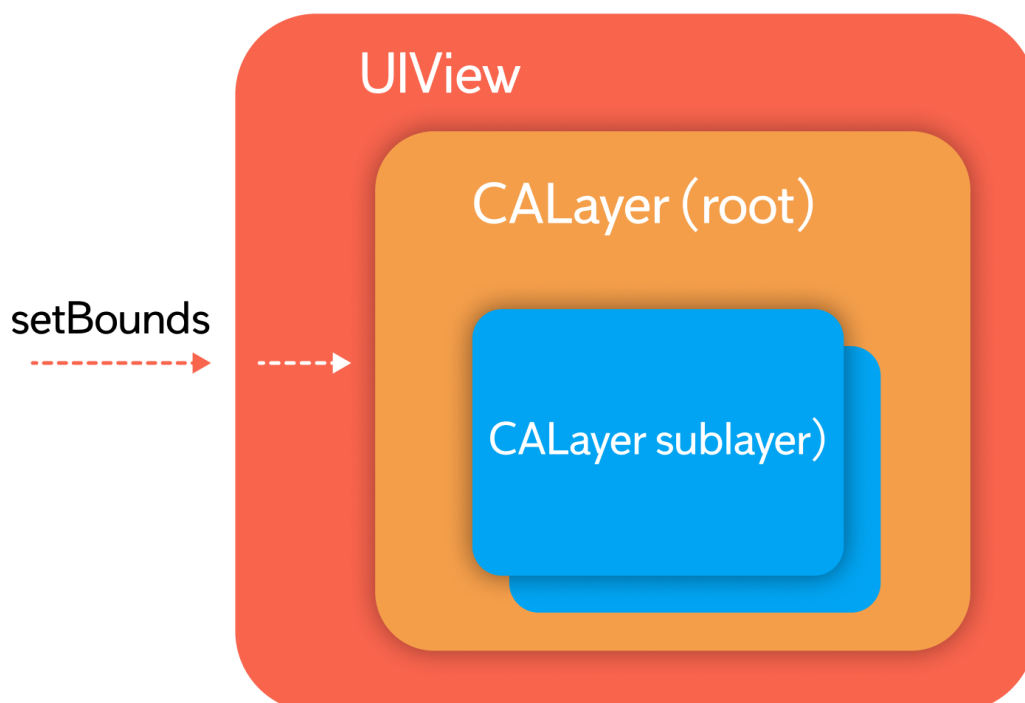
```
indexPath) as! MyTableViewCell
    // configure the cell with data
    return cell
}
```

Q25. What is the difference between UIView and CALayer?

Answer: UIView and CALayer are both fundamental building blocks of iOS apps user interface, but they have different purposes and functionalities.

UIView:

UIView is a high-level class that is responsible for managing a rectangular area on the screen where you can draw your app's interface elements, such as buttons, labels, text fields, and other views.



CALayer:

CALayer, on the other hand, is a lower-level class that provides a efficient way to draw and animate 2D graphics. It's a part of the Core Animation framework, and it's used to draw the content of UIView objects. Each UIView object has a corresponding CALayer object that manages the actual rendering of the view's content. CALayer objects can be used independently of UIView objects, but they don't provide user interaction handling or layout management.

Counter Question:

Q: How they differ in terms of memory usage and performance?

Answer: UIView and CALayer differ in terms of memory usage and performance. Here's a comparison:

Memory Usage:

UIView objects have a higher memory footprint than CALayer objects because they have more functionality and manage the layout of their subviews whereas CALayer objects are lightweight and have a smaller memory footprint than UIView objects because they don't handle user interaction or layout management.

Performance:

CALayer objects provide efficient drawing and animation capabilities, which makes them faster and more performant than UIView objects whereas UIView objects are slower to render and animate than CALayer objects because they have a higher level of abstraction and additional functionality.

Q26. What does UIApplicationMain mean?

Answer: The `@UIApplicationMain` attribute is used in Swift to designate the entry point for an iOS, iPadOS, or macOS app that uses a graphical user interface (GUI).

When you add the `@UIApplicationMain` attribute to a class that conforms to the `UIApplicationDelegate` protocol, it tells the compiler that this class contains the main entry point for the application. This class must also include a `main()` function that serves as the starting point for the app.

The `UIApplicationDelegate` protocol defines methods that manage the application's life cycle, such as launching, backgrounding, and terminating the app. By conforming to this protocol, your app's main class can implement these methods to respond appropriately to changes in the app's state.

The class to which you add `@UIApplicationMain` attribute must conform to the `UIApplicationDelegate` protocol. This defines a set of methods that manage the app's life cycle. These methods include `application(_:didFinishLaunchingWithOptions:)`, `applicationDidEnterBackground(_:)`, `applicationWillEnterForeground(_:)`, and others.

Your app's main class can implement these methods to customize the behavior of the app when it's launched, enters the background, or returns to the foreground.

If you need to access the `UIApplication` object from your app's main class, you can do so using the `UIApplication.shared` singleton instance.

Q27. How do you implement dynamic type and font scaling in an iOS app?

Answer: Dynamic type and font scaling are important features in iOS apps that allow users to customize the font size and style according to their preferences. Here are the steps to implement dynamic type and font scaling in an iOS app:

Choose a font family that supports dynamic type:

When selecting a font for your app, choose a font family that includes various font styles and weights, and supports dynamic type. Some of the popular font families that support dynamic type are San Francisco, Avenir, and Helvetica Neue.

Use the system font APIs:

To implement dynamic type, use the system font APIs provided by iOS, such as

`UIFont.preferredFont(forTextStyle:)`, `UIFontMetrics.default.scaledFont(for:)`, and `UIFontMetrics.default.scaledValue(for:)`. These APIs automatically adjust the font size and weight based on the user's preferred text size in the Settings app.

Configure font scaling in Interface Builder:

If you are using Interface Builder to design your app's UI, you can configure font scaling for labels and text views by setting the font to "preferred font" and selecting a text style from the "text style" dropdown menu.

Some Related Questions:

- What is Dynamic Type in iOS and why is it important for accessibility?
- How can you verify and test the effectiveness of your dynamic type implementation on different devices and accessibility settings?
- What we need to do to support dynamic for custom fonts?

Q28. Explain the concept of the Responder Chain in iOS.

Answer: The Responder Chain is a key mechanism in iOS that facilitates event handling and message passing between objects. It is a hierarchical chain of objects that respond to user events, such as touches, gestures, or keyboard input.

The Responder Chain follows a specific order to deliver events and messages. The chain starts with the first responder, which is typically the object that currently receives user input or has focus. If the first responder cannot handle the event, it passes it to the next responder in the chain. This process continues until the event is handled or until the end of the chain is reached.

The Responder Chain is primarily based on the `UIResponder` class, which is the superclass for

various UI classes like UIView and UIViewController. **UIResponder** provides methods to handle events, such as `touchesBegan(:with:)`, `touchesMoved(:with:)`, and so on.

Counter Question:

Q: How does Responder Chain facilitate event handling and message passing?

Answer: The Responder Chain provides a flexible and efficient way to handle events and pass messages between objects without explicit references. It allows for modular and reusable code by enabling objects to handle events and perform actions based on their position in the chain.

Event Delivery: When a user interacts with the UI, such as tapping a button or scrolling through a view, the event is initially delivered to the first responder. If the first responder can handle the event, it does so, and event processing stops. If the first responder cannot handle the event, it forwards the event to its next responder in the chain until a responder can handle it.

Message Passing: Along with event handling, the Responder Chain also enables message passing between objects. Instead of relying solely on events, objects can send messages to their next responders in the chain. This allows custom communication and actions. For example, a view controller can pass a message to its parent view controller or to its child view controllers.

Event and Message Responder Hierarchy: The Responder Chain follows a specific hierarchy. For example, within a view hierarchy, a subview's first responder is typically itself or one of its subviews. If the subview cannot handle an event, it passes it to its superview, and the chain continues up the hierarchy until a responder is found.

Q29. Explain the concept of trait collections and size classes in iOS. How do they work together to create adaptive user interfaces for different devices and orientations.

Answer: In iOS, trait collections and size classes are used to represent the user interface environment. They also represent the dimensions of the available display area in which the app will be rendered.

A trait collection is a collection of traits, such as size classes, display scales, and user interface idioms. Size classes, on the other hand, are part of trait collections and represent the display area dimensions. There are two size classes: horizontal and vertical.

Size classes allow the app to adjust its layout based on screen size, orientation, and other environmental factors. For example, an app may display a larger font on a larger device and a smaller font on a smaller device. It may adjust the layout of its UI elements depending on whether the device is in portrait or landscape mode.

The app uses trait collections to determine the appropriate layout in a given environment. The app can define different layouts for different size classes or trait collections, allowing it to adapt to various screen sizes and other environmental factors. This is particularly important for apps that run on a wide range of devices, from small iPhones to large iPads.

When the app is launched, it receives a trait collection that describes the current environment. The app can use this trait collection to determine the size class and other characteristics of the environment. It can also select the appropriate layout and UI elements to use.

If the environment changes, such as when the device is rotated, the app will receive a new trait collection that describes the new environment. The app can then update its layout and UI elements to adapt to the new environment.

Overall, the process of applying trait collections and size classes in iOS involves defining your layout constraints, defining your size classes, implementing your trait collection logic, and testing and refining your app. By following these steps, you can create a responsive and adaptive user interface that works well on a wide range of devices and orientations.

SwiftUI Fundamentals

Q30. What is the difference between @StateObject and @ObservedObject in SwiftUI?

Answer: `@StateObject` and `@ObservedObject` are two ways to manage the state of an object in SwiftUI.

`@StateObject` is used when you want to create an instance of an object that will manage the state of a view. It creates and owns the object, and keeps it alive for the view's lifetime. You can also modify the object's state using this property wrapper.

On the other hand, `@ObservedObject` is used when you already have an instance of an object that you want to use to manage the state of a view. It does not create or own the object, but only observes it. This means that the object's lifetime is not managed by this property wrapper.

However, there are some key differences between the two:

- **Ownership:** The `@StateObject` creates and owns an instance of an object, while the `@ObservedObject` only observes an instance that is passed in or injected in.
- **Initialization:** The `@StateObject` requires an initializer for the object it creates, while the `@ObservedObject` requires an instance of the object.
- **Lifetime:** The `@StateObject` keeps the object alive for the lifetime of the view, while the `@ObservedObject` doesn't create or own the object.
- **Mutability:** The `@StateObject` allows the object to be mutated within the view, while the `@ObservedObject` does not.

Q31. What does the @Published property wrapper do in SwiftUI?

Answer: The `@Published` is typically used in conjunction with the Model-View-ViewModel (MVVM) architecture pattern in SwiftUI.

When applied to a class property, `@Published` creates a publisher for that property. A publisher is an object that emits events whenever the property value changes. The publisher is a type that conforms to the `ObservableObject` protocol. This makes it easy to build reactive, data-driven user interfaces in SwiftUI.

SwiftUI uses an `ObservableObject` as a source of truth for a view. When the `ObservableObject` properties change, any views bound to those properties will automatically redraw to reflect the updated values.

Here's an example of how `@Published` can be used in a simple `ObservableObject` class:

```
class CounterViewModel: ObservableObject {
    @Published var count: Int = 0

    func increment() {
        count += 1
    }
}

struct CustomView: View {

    @StateObject var counterViewModel = CounterViewModel()

    var body: some View {
        VStack {
            Text("Count: \(counterViewModel.count)")
            Button("Increment") {
                counterViewModel.increment()
            }
            .padding()
        }
    }
}
```

In this example, the `count` property is marked with the `@Published` property wrapper. This means that whenever the value of `count` changes, the `CounterViewModel` object will automatically publish an event to any subscribers. The `increment()` method increments the `count` property by one. View

will automatically update to reflect the changes in value when `increment()` is called.

Note: Once you will try to run the above example with the below change:

```
var count: Int = 0
```

In the above line, you removed `@Published`. Now you will see no changes reflect on the view after clicking the Increment button.

Q32. Discuss the benefits and limitations of using SwiftUI for building user interfaces.

Answer: SwiftUI is a constantly evolving framework that comes with many benefits as well as some limitations. Let's explore both the sides.

Benefits of building user interfaces with SwiftUI:

- **Declarative syntax:** It uses a declarative syntax to define user interfaces, which means that you describe the final result you want to achieve rather than specifying the exact steps to get there.
- **Live Previews:** It provides a live preview feature that allows you to see changes to your code in real-time, without running your app.

Limitations of SwiftUI for iOS user interfaces:

- **Lack of backward compatibility:** SwiftUI is only supported in iOS 13 and later version, which means that apps that need to support older iOS versions cannot use SwiftUI.
- **Limited customization:** While SwiftUI provides a range of built-in components, styles and APIs, it can be more challenging to customize these components than with older frameworks like UIKit.

Modularisation

Q33. How to modularise the Codebase and why is it important?

Answer: Modularization is a way to break down a large codebase into smaller, more manageable modules. Here are some steps to modularize your codebase:

Identify logical boundaries:

Identify the different components and functionalities of your codebase that can be logically separated into smaller modules. For example, you could separate your code into modules based on features, UI components, network layers, or data models.

Create a module:

A module should be a self-contained unit of code that can be reused in other parts of your app or in other apps. Once you have identified the logical boundaries, create a new module for each one.

Define module interface:

Define a clear and concise interface for each module that specifies what the module does, what it depends on, and how it can be used. The interface should also hide the implementation details of the module from the outside world.

Use access modifiers:

Use access modifiers such as public, internal, fileprivate, and private to control the visibility of the module's components. This will help prevent other parts of the codebase from accessing internal details of a module and promote encapsulation.

Use dependency injection:

Use dependency injection to allow different modules to communicate with each other. This will help avoid circular dependencies and make it easier to test individual modules.

Use frameworks:

Finally, consider using frameworks to encapsulate your modules into reusable, distributable units. Frameworks provide a well-defined interface for other modules to use, and they can be easily integrated into other projects.

Importance of Codebase Modularization

1. **Reusability:** By breaking down your codebase into smaller modules, you can increase code reuse. Modules can be reused in other parts of your app or in other apps, reducing development time and effort.
2. **Testability:** Modularization makes it easier to test your code. Each module can be tested separately, making it easier to isolate and fix issues. This can lead to more reliable and robust code.
3. **Scalability:** As your app grows in complexity, modularization can help make it more scalable. New features can be added as separate modules, making it easier to manage and maintain the codebase.

Q34. What is Dependency Injection and what are its advantages?

Answer: Dependency Injection is a design pattern that promotes loosely coupled code by separating the creation and management of dependencies from the class that uses them. Dependency Injection can be implemented in several ways.

Constructor injection:

This involves passing dependencies into a class's init. For example, if a class depends on a network client, you can inject the client into the class's init:

```
class ViewController: UIViewController {
    let networkClient: NetworkClient

    init(networkClient: NetworkClient) {
        self.networkClient = networkClient
        super.init(nibName: nil, bundle: nil)
    }

    // ...
}
```

Property injection:

This involves passing dependencies through properties. Note that Property injection requires the dependency to be optional, which can make the code more error-prone. For example:

```
class ViewController: UIViewController {
    var networkClient: NetworkClient?

    // ...
}
```

Method injection:

This involves passing dependencies into a method when it's called. For example:

```
class ViewController: UIViewController {
    func fetchData(using networkClient: NetworkClient) {
        // ...
    }

    // ...
}
```

Advantages of Dependency Injection :

Here are some advantages of implementing dependency injection in your project:

Testability: When a class has dependencies that are tightly coupled, it can be difficult to test the class in isolation. By injecting dependencies through the constructor, you can easily swap out the dependencies with mock objects or stubs for testing purposes.

Flexibility: Dependency injection can make your code more flexible and modular. By injecting dependencies, you can easily switch out one implementation of a dependency for another, without having to modify the class that uses it.

Separation of concerns: By separating the creation and management of dependencies from the class that uses them, you can achieve a better separation of concerns in your code. This can make your code easier to understand, modifiable, and extendable over time.

Reusability: By injecting dependencies, you can make your classes more reusable in different contexts. For example, you might have a network client that is used in several different classes throughout your app.

Error Handling

Q35. Explain the difference between throws and rethrows in Swift.

Answer: In Swift, `throws` and `rethrows` are used to indicate that a function can potentially propagate an error within it to its caller.

throws:

The `throws` keyword is used to mark a function that can throw an error. When a function is marked with 'throws', it means that it can potentially generate an error during its execution. In order to handle such errors, callers of the function must either use a 'do-catch' block to handle the error or propagate the error to their own caller using the 'throws' keyword.

```
enum MyError: Error {
    case invalidInput
}

func divide(_ Answer: Int, by b: Int) throws -> Int {
    guard b != 0 else {
        throw MyError.invalidInput
    }
    return a / b
}

do {
    let result = try divide(10, by: 0)
    print(result)
} catch let error {
    print(error)
}

// print: invalidInput
```

In this example, the `divide` function takes two integers as input and returns the result of dividing the first integer by the second integer. However, the function can potentially throw an error if the second integer is 0. Therefore, the function is marked with `throws`.

rethrows:

The `rethrows` keyword is used to mark a function that takes one or more throwing functions as parameters, and itself can propagate an error. When a function is marked with `rethrows`, it means that it will only throw an error if one of its throwing function parameters throws an error. If none of the function's throwing parameters throw an error, the function will not throw an error itself. In other words, a 'rethrows' function rethrows an error that was thrown by one of its throwing function parameters.

```
enum MyError: Error {
    case invalidInput
}

func manipulateArray(_ array: [Int], using closure: (Int) throws -> Int)
rethrows -> [Int] {
    var result: [Int] = []
    for element in array {
        let transformed = try closure(element)
        result.append(transformed)
    }
    return result
}

func addOne(_ number: Int) throws -> Int {
    guard number >= 0 else {
        throw MyError.invalidInput
    }
    return number + 1
}

let input = [1, 2, 3, 4, 5]
do {
    let result = try manipulateArray(input, using: addOne)
    print(result)
} catch {
    print("An error occurred: \(error)")
}

// print: [2, 3, 4, 5, 6]
```

In this example, the `addOne` function is a throwing function that takes an integer as input and adds 1 to it. However, it throws an error if the input integer is negative. The `manipulateArray` function is called with the `addOne` closure as a parameter. The result is an array with each element incremented by 1. If any errors are thrown by the `addOne` closure, they are rethrown by the `manipulateArray` function and caught by the 'do-catch' block.

Q36. Difference between Array and NSArray.

Answer: Array and NSArray are very similar and core constructs in iOS Development, and because of these similarities we sometime forget how different these two constructs work,

Here are some major difference that you must know about Array and NSArray.

Array:

Array is a **struct** which means it is **value type** in Swift.

```
var singer = ["Taylor", "Swift", "Taylor's", "Version"]

func modifyValues(value : Array<String>) {
    value[2] = "Not Taylor's"
}

modifyValues(singer)

print(singer)
/* This prints - ["Taylor", "Swift", "Taylor's", "Version"] */
```

The singer array is not modified since array is a **struct** hence, the values are not modified outside the function.

NSArray:

NSArray is an immutable Objective C **class**, therefore it is a **reference type** in Swift and it is bridged

to **Array< AnyObject >**. **NSMutableArray** is the mutable subclass of **NSArray**.

```
var singer: NSMutableArray = ["Taylor", "Swift", "Taylor's", "Version"]

func modifyValues(value : NSMutableArray) {
    value[2] = "Not Taylor's"
}

modifyValues(singer)

print(singer)
/* This prints - ["Taylor", "Swift", "Not Taylor's", "Version"]*/
```

The singer array is modified since **NSMutableArray** is reference type. Thus, the change in values are reflected outside the function.

Conclusion:

Array is a Swift construct, and generic struct, which means that it can be an array of any specific type (Int, String, etc.) **[T]** is syntactic sugar for **Array< T >**

NSArray is an Objective-C construct that can hold any Objective-C object and is transparently mapped to and from **Array< AnyObject >**.

Networking

Q37. Explain the types of sessions and tasks supported by URLSession class.

Answer: URLSession is a powerful networking API in iOS and macOS that provides a set of classes and methods to perform various networking tasks, such as fetching data from a remote server or uploading data to a server.

There are three types of sessions supported by URLSession:

Default Session:

This is the most common and default session type used in the URLSession class. A global cache, credential storage object, and cookie storage object are used.

This is how we define the default session:

```
let config = URLSessionConfiguration.default
```

Ephemeral Session:

This session is similar to the default session but it doesn't use any storage like caches, cookies, or credentials. It's useful when you need to make requests with different or temporary credentials, like when you're authenticating with an API. It's also useful when you want to keep your requests and responses private and not stored on disk.

This is how we define the default session:

```
let config = URLSessionConfiguration.ephemeral
```

Background Session:

This session type is used when you want to download or upload data in the background, even when your app is not running. The system handles the session and provides progress updates through

delegate methods. The background session is useful when you want to download large files or perform a lengthy operation that requires more than a few seconds.

This is how we define the default session:

```
let config = URLSessionConfiguration.background(withIdentifier:
"com.example.background")
```

There are several tasks supported by URLSession, including:

Data Task: This task is used to retrieve data from a URL. It returns the server's response as Data in the completion handler. Here is the method syntax:

```
session.dataTask(with: <URLRequest>, completionHandler: <(Data?, URLResponse?,
Error?) -> Void>)
```

Download Task: This task is used to download a file from a URL. It returns the downloaded file's location on disk in the completion handler. Here is the method syntax:

```
session.downloadTask(with: <URLRequest>, completionHandler: <(URL?,
URLResponse?, Error?) -> Void>)
```

Upload Task: This task is used to upload data to a URL. It can upload data in the form of a file or a stream. Here is the method syntax:

```
session.uploadTask(with: <URLRequest>, from: <Data?>, completionHandler:
<(Data?, URLResponse?, Error?) -> Void>)
```

WebSocket Task: This task is used to establish a WebSocket connection to a URL. It allows bidirectional communication between the client and server in real time. Here is the method syntax:

```
session.webSocketTask(with: <URL>)
```

Each of these tasks has its own set of delegate methods that can be used to monitor the progress of the task, handle authentication, handle errors, and more.

Note:

- There's no way to configure or customize the caching behaviour beyond setting the cache policy and timeout interval.
- While URLSession provides the ability to perform network requests in the background, background tasks are limited to 30 seconds by default.
- URLSession does not support all possible networking protocols like FTP or SMTP.
- There's no built-in way to retry failed requests automatically.

Q38. How to track image download progress in iOS?

Answer: You can track the progress of an image download in iOS using the URLSession data task and implementing the URLSessionDataDelegate protocol. By using the delegate methods provided by the protocol, you can get the expected content length of the image. You can also receive the downloaded data, and calculate the progress of the download.

Let's understand this implementation with the following steps:

Initializing ByteCountFormatter

```
let byteFormatter: ByteCountFormatter = {  
    let formatter = ByteCountFormatter()  
    formatter.allowedUnits = [.useKB, .useMB]  
    return formatter  
}()
```

Create a URLSession download task and set the delegate to self:

```
guard let url = URL(string: "https://picsum.photos/2000") else { return }  
let config = URLSessionConfiguration.default  
let session = URLSession(configuration: config, delegate: self, delegateQueue:
```

```
nil)
session.downloadTask(with: url).resume()
```

Implement the URLSessionDownloadDelegate protocol:

```
extension ViewController: URLSessionDelegate, URLSessionDownloadDelegate {

    func urlSession(_ session: URLSession, downloadTask: URLSessionDownloadTask,
didWriteData bytesWritten: Int64, totalBytesWritten: Int64,
totalBytesExpectedToWrite: Int64) {
        // write code here
    }

    func urlSession(_ session: URLSession, downloadTask: URLSessionDownloadTask,
didFinishDownloadingTo location: URL) {
        // write code here
    }
}
```

Implement the didWriteData delegate method to track progress:

```
func urlSession(_ session: URLSession, downloadTask: URLSessionDownloadTask,
didWriteData bytesWritten: Int64, totalBytesWritten: Int64,
totalBytesExpectedToWrite: Int64) {

    // Downloaded in KB
    let written = byteFormatter.string(fromByteCount: totalBytesWritten)

    // Total size in KB
    let expected = byteFormatter.string(fromByteCount:
totalBytesExpectedToWrite)

    // Total progress
    let progress = Float(totalBytesWritten) / Float(totalBytesExpectedToWrite)

    // Print on console (or update progress on UI)
    print("Downloaded \(written) / \(expected)")
    print("Progress: \(progress * 100)%")
}
```

Implement the `didFinishDownloadingTo` delegate method to handle the downloaded image:

```
func URLSession(_ session: URLSession, downloadTask: URLSessionDownloadTask,
didFinishDownloadingTo location: URL) {
    if let data = try? Data(contentsOf: location), let image =
UIImage(data: data) {
        DispatchQueue.main.async {
            self.imageView.image = image
        }
    } else {
        fatalError("Cannot load the image")
    }
}
```

Frameworks and Libraries

Q39. Compare static and dynamic libraries.

Answer:

Static Library

When you compile the code, a unit of code is linked, which does not change. That piece of code is linked to the generated executable file by a static link during the compilation process. Static libraries can't include media files such as images or assets, which means static libraries can only contain a unit of code.

- **Compilation:** Static libraries are compiled and linked directly into the executable binary at compile time.
- **Size:** Static libraries increase the size of the final executable because the library code is duplicated in each binary that uses it.
- **Independence:** Static libraries make the binary self-contained, as all the necessary code is included within the executable.
- **Performance:** Static libraries offer better performance during runtime since the code is directly linked and doesn't require dynamic loading at runtime.
- **Update:** If a change is made to a static library, all the applications using that library need to be recompiled and redeployed to incorporate the changes.

Dynamic Library

At runtime, dynamic libraries link units of code or media files that may change. They are different from static libraries in the sense that they are linked to the app's executable at runtime, but not copied into it. In the case of a dynamic library, the executable file is smaller because the code is loaded only when it is needed at runtime.

- **Compilation:** Dynamic libraries are compiled separately and linked at runtime when the application is launched.

- **Size:** Dynamic libraries don't increase the size of the final executable as they are shared among multiple applications, reducing redundancy.
- **Dependency:** Dynamic libraries can have dependencies on other libraries and frameworks, and they can be loaded and unloaded dynamically at runtime.
- **Performance:** Dynamic libraries may have a slight performance overhead during runtime due to the dynamic loading and symbol resolution process.
- **Update:** If a change is made to a dynamic library, all the applications using that library can benefit from the changes without recompilation or redeployment.

Dynamic libraries are commonly used for system frameworks provided by Apple, while static libraries are used for third-party libraries or when developers want to bundle specific functionality directly into their application. The choice between static and dynamic libraries depends on factors such as code size, performance requirements, update flexibility, and dependency management.

Some Related Questions :

- What are the advantages of using static libraries in iOS development?
- How are dynamic libraries different from static libraries in terms of memory usage?
- How are static libraries fast for app launch time?
- When would you choose to use dynamic libraries instead of static libraries in an iOS project?

Q40. What is the purpose of Core Location framework?

Answer: Apple provides Core Location framework that allows developers to obtain location-related information from the device's hardware and software. It provides access to location data such as latitude, longitude, altitude, speed, and heading. The framework uses different technologies, such as GPS, Wi-Fi, and cellular network information, to determine the user's location.

The purpose of Core Location is to enable location-based services on iOS devices. Location-based

services are applications that provide information or functionality based on the user's location, such as mapping and navigation, weather, or location-based social networking.

Counter Question:

Q: How do you use Core Location effectively for location-based services?

Answer: To use Core Location effectively for location-based services, you need to consider the following:

1. **Accuracy:** Core Location provides location updates with varying accuracy levels. GPS provides the most accurate location data, but consumes more battery power. Wi-Fi and cellular network information provide less accurate location data, but consume less battery power. You need to decide what level of accuracy you need for your location-based service and use the appropriate location technology.
2. **Battery consumption:** Location-based services can consume a lot of battery power, especially if you use high accuracy location technologies like GPS. You need to minimize battery consumption by using low accuracy location technologies when possible. You need to stop location updates when they are not necessary, and optimize location updates frequency.
3. **User privacy:** Core Location requires user permission to access location data. You need to respect user privacy by explaining why you need access to their location data and how you will use it. You should also provide an option for the user to disable location services if they want to.
4. **User experience:** Location-based services should provide a good user experience by providing relevant and accurate information depending on the user's location. You need to design your location-based service with the user in mind and make sure it is easy to use and provides value for the user.
5. **Error handling:** Core Location can sometimes return inaccurate or invalid location data. You need to handle these errors gracefully and provide appropriate error messages to the user.

By considering these factors, you can use Core Location effectively for location-based services and provide a valuable and engaging experience for your users.

Some Related Questions:

- What are the different authorization levels for accessing a user's location in iOS?
- Explain the difference between "desired accuracy" and "distance filter" in Core Location
- How can you monitor and track changes in the user's location using Core Location?
- How would you optimize location updates to conserve battery life in an iOS app?

App and Code Optimisation

Q41. Compare UITableView and UICollectionView in terms of usage, optimization, and performance.

Answer: The decision to use either `UITableView` or `UICollectionView` in an iOS app depends on the specific requirements of the app's user interface.

In general, if you have a simple layout that requires a straightforward list of data, you should use `UITableView`. If you have a more complex layout that requires custom layouts or grouping of data, `UICollectionView` may be the better choice.

Key differences between UITableView and UICollectionView:

- **Layout:** `UITableView` is designed to display data in a vertical list, while `UICollectionView` allows for more flexible layouts, such as a grid, a custom layout, or a combination of both.
- **Cell Reuse:** Both `UITableView` and `UICollectionView` use cell reuse to conserve memory and improve performance. However, `UICollectionView` provides more control over how cells are reused and displayed, allowing for more complex layouts.
- **Section Headers and Footers:** `UITableView` supports section headers and footers out of the box, while `UICollectionView` requires more customization to display section headers and footers.
- **Selection and Highlighting:** `UITableView` has built-in support for selecting and highlighting cells, while in `UICollectionView`, you need to implement custom code to achieve the same behavior.
- **Supplementary Views:** `UICollectionView` supports supplementary views, which are additional views that can be added to a section, such as headers, footers, or decorations. `UITableView` does not have built-in support for supplementary views.

In general, `UITableView` is optimized for displaying large amounts of data in a simple list format. It uses a straightforward vertical layout and cell reuse to improve performance and memory usage. `UITableView` is particularly well-suited for displaying homogeneous data sets, where each cell has the same format and layout.

`UICollectionView`, on the other hand, is optimized for more complex layouts and displays, such as grids or custom layouts. It provides more flexibility and customization options for displaying diversified

data sets, where each cell can have a different format and layout. `UICollectionView` is particularly useful for displaying data in a non-linear format or for creating custom user interfaces.

Q42. What are the differences between Class and Struct in Swift?

Answer: Both Classes and Structs can be used to define custom data types, but they have some differences in behaviour and usage. Here are some key differences:

Inheritance:

Classes support inheritance, meaning you can create a subclass that inherits properties and methods from a parent class. Structs do not support inheritance.

```
class Vehicle {
    var color: String
    init(color: String) {
        self.color = color
    }
}

class Car: Vehicle {
    var numberOfDoors: Int
    init(color: String, numberOfDoors: Int) {
        self.numberOfDoors = numberOfDoors
        super.init(color: color)
    }
}
```

Reference vs Value Types:

Classes are reference types, which means that when you create an instance of a class and assign it to another variable, both variables point to the same instance of the class. Structs, on the other hand, are value types. This means that when you create a new instance and assign it to another variable, a

new copy of the instance is created.

```
class Person {
    var name: String
    init(name: String) {
        self.name = name
    }
}

var p1 = Person(name: "Alex")
var p2 = p1
p1.name = "Bob"
print(p1.name) // "Bob"
print(p2.name) // "Bob"

struct Point {
    var x: Double
    var y: Double
}

var point1 = Point(x: 0, y: 0)
var point2 = point1
point1.x = 1
print(point1.x) // 1.0
print(point2.x) // 0.0
```

Mutability:

By default, struct properties are immutable (cannot be changed) unless functions are marked with the `mutating` keyword. In contrast, class properties are mutable by default.

```
class BankAccount {

    var balance: Double

    init(balance: Double) {
        self.balance = balance
    }

    func deposit(amount: Double) {
```

```

        balance += amount
    }
}

struct Point {

    var x: Double
    var y: Double

    mutating func moveBy(x: Double, y: Double) {
        self.x += x
        self.y += y
    }
}

let account = BankAccount(balance: 1000)
account.deposit(amount: 500)
print(account.balance) // 1500.0

var point = Point(x: 0, y: 0)
point.moveBy(x: 1, y: 1)
print(point.x) // 1.0
print(point.y) // 1.0

```

Initialization:

Structs have a memberwise initializer by default, which means you can initialize a new instance of a struct using a set of default values for each of its properties. Classes do not have memberwise initializers by default.

```

struct Rectangle {
    let width: Int
    let height: Int
}

// Using default memberwise initializer
let myRect = Rectangle(width: 10, height: 20)

class Person {
    let name: String

```

```

var age: Int

init(name: String, age: Int) {
    self.name = name
    self.age = age
}

// Using custom initializer
let john = Person(name: "John", age: 30)

```

Memory Management:

Memory management for Classes is handled by ARC (Automatic Reference Counting), while Structs are value types and they get deallocated after their last use therefore their explicit memory management is not required.

```

class MyClass {
    var value: Int
    init(value: Int) {
        self.value = value
    }
}

var obj1: MyClass? = MyClass(value: 10)
var obj2 = obj1
obj1 = nil
obj2 = nil // ARC automatically deallocates the object

struct MyStruct {
    var value: Int
}

var struct1 = MyStruct(value: 10)
var struct2 = struct1
struct1.value = 20
print(struct1.value) // Output: 20
print(struct2.value) // Output: 10 (both struct1 and struct2 are independent
copies)
// No need for memory management as structs are copied around

```

Memory Location:

For values, such as basic types (e.g., Integers, Booleans, Floats), they are stored directly in stack (in most cases) memory where the variable is declared. When you assign a value to a variable, the actual value is copied to that memory location. This means that when you assign a value to another variable or pass it to a function, a new copy of the value is created.

References, such as instances of classes, are stored indirectly in memory. When you create an instance of a class, memory is allocated on the heap to hold the actual object. The variable or constant that holds a reference to the object stores a pointer to the memory location where the object resides. This pointer is what's passed around when you assign a reference to another variable or pass it to a function.

Counter Question:

Q: When to use Class over a Struct?

Answer: It's highly recommended to use Structs primarily but here are two reasons why you should consider using a Class over Struct:

- Use a class when you need to create a shared state accessible across multiple parts of your project. For example, you might use a class to represent a Database connection or a User Interface Controller.
- Use a class when you need to take advantage of Inheritance, where you create a new class that is based on a Base Class and inherits its properties and behavior.

Some Related Questions :

- What are the advantages of using a struct over a class?
- How does the choice between using a class or a struct affect memory management in Swift?
- Can you explain the concept of reference semantics and value semantics in the context of classes and structs?

- Can you give an example of a situation where immutability of a struct could be beneficial?

Q43. How can you improve Swift class performance?

Answer: Improving a class's performance in Swift involves identifying and addressing bottlenecks or inefficiencies in the code. Here are some strategies to improve performance:

- **Use lazy loading:** If a property is not always needed, you can use lazy loading to defer its initialization until it's accessed for the first time. This can reduce unnecessary overhead.
- **Avoid excessive closure use:** Closures can be convenient, but they can also cause performance issues if overused. Make sure to use closures only when necessary and avoid nesting them too deeply.
- **Use value types instead of reference types:** Value types are copied when passed around, while reference types are passed by reference. For smaller data types, value types are more efficient than reference types.
- **Optimize loops:** Loops are a common source of performance issues. You can optimize loops by reducing iterations, using early exit conditions, and pre-allocating arrays or collections to avoid unnecessary reallocation.
- **Use the right data structures:** Choosing the right data structure can make a big difference in performance. For example, if you need to search or sort data frequently, using a hash table or binary tree can be more efficient than an array.
- **Minimize memory usage:** Excessive memory usage can slow your code. You can reduce memory usage by using structs instead of classes, avoiding unnecessary data copying, and releasing unused resources promptly.
- **Profile your code:** Use profiling tools to identify bottlenecks and performance issues in your code. This will help you focus your optimization efforts where they will have the most impact.
- **Using final keyword:** Use static method dispatch by marking classes as final if not required to

inherit by any child, and marking methods as private if possible.

Q44. How would you implement an Infinite Scrolling List?

Answer: To implement an infinite scrolling list in iOS, follow these steps:

1. Set up a `UITableView` with a data source that provides data for the table view cells.
2. Implement the `UITableViewDelegate` method `scrollViewDidScroll` to detect when the user scrolls to the bottom of the table view.
3. When the user reaches the bottom of the table view, load more data and append it to the existing data source.
4. Call `reloadData` on the table view to update the display with the new data.

Here is the basic code snippet to follow:

```
func scrollViewDidScroll(_ scrollView: UIScrollView) {
    let offsetY = scrollView.contentOffset.y
    let contentHeight = scrollView.contentSize.height

    if offsetY > contentHeight - scrollView.frame.height {
        // The user has scrolled to the bottom of the table view.
        // Load more data here.
        loadMoreData()
    }
}

func loadMoreData() {
    // Load more data here and append it to your existing data source.
    let newData = fetchNextBatchOfData()
    dataSource.append(contentsOf: newData)

    // Reload the table view with the updated data source.
    tableView.reloadData()
}
```

Counter Question :

Q: How to improve scrolling performance of infinite List?

Answer: To improve infinite scrolling list's performance, you can follow these steps:

- **Load data incrementally:** Instead of loading all data at once, load it in smaller batches to reduce memory overhead and improve performance.
- **Reuse table view cells:** Use `dequeueReusableCellWithIdentifier` to reuse table view cells instead of creating new ones for each row. This reduces memory overhead and improves performance.
- **Cache images and other resources:** If your table view contains images or other resources, cache them to reduce loading time and improve performance.
- **Use lazy loading:** Load data only when needed, and not before. This approach can improve performance by reducing the amount of data loaded.
- **Avoid blocking the main thread:** Loading data can be time-consuming, so it's important to avoid blocking the main thread. Use background threads to perform data loading and processing, and update the UI on the main thread.

Q45. How can we fix non-smooth scrolling issues?

Answer: Here are some techniques to address non-smooth scrolling issues:

- **Avoid heavy computations on the main thread:** Heavy computations can cause non-smooth scrolling. Performing long-running tasks like network requests or file I/O on the main thread can cause non-smooth scrolling. You can use GCD or Operation queues to perform these tasks asynchronously on a background thread.
- **Minimize layout updates:** Frequent layout updates can cause non-smooth scrolling. You can minimize layout updates by setting the `UIView`'s `clipsToBounds` property to `true`, setting the `backgroundColor` property to a solid color, and avoiding the use of transparency.

- **Use instruments to identify bottlenecks:** Xcode's Instruments is a powerful tool that can help you find performance bottlenecks in your app. You can use the Time Profiler instrument to identify which parts of your code cause non-smooth scrolling.
- **Optimize image loading:** Loading large images can cause non-smooth scrolling. You can optimize image loading by resizing images to match the size of the image view they're displayed in. You can also use compression, or a library like SDWebImage or Kingfisher to handle image loading and caching.
- **Implement pagination:** If your app displays a large amount of data, it can cause scrolling issues. One way to address this is to implement pagination, which means loading and displaying data in small chunks or pages. This can significantly improve scrolling performance.
- **Implement cell reuse:** When using UITableView or UICollectionView, ensure that you're reusing cells instead of creating new ones. This can improve scrolling performance and reduce memory usage.
- **Lazy loading:** Instead of loading all the content at once, consider lazy loading or infinite scrolling to load the content gradually as the user scrolls. This will ensure that the app only loads what's necessary and reduces the overall memory footprint.

By following these steps, you can improve your app's scrolling performance and provide a better user experience.

Q46. How to identify the causes of UI errors in iOS apps? Also, how you can improve UI performance?

Answer: Identifying the causes of UI errors in iOS apps can be a complex process. However, there are some general steps you can take to diagnose the issue. Here are some tips:

1. **Reproduce the issue:** The first step is to reproduce the issue on a device or simulator. Try to recreate the steps that led to the error so you can see the issue firsthand.

2. **Check the console:** When you run the app on a device or simulator, check the console for error messages. Xcode logs all errors, so this is an ideal place to start.
3. **Analyze the view hierarchy:** Use Xcode's View Debugger to analyze the app's view hierarchy. This tool allows you to inspect the properties and constraints of each UI element in the app. This can help you identify layout issues or other UI errors.
4. **Review the code:** Review the code related to the UI element that's causing the error. Look for any mistakes or typos in the code that might cause the issue.
5. **Test on different devices:** Test the app on several iOS devices to see if the issue is device-specific. Sometimes, certain devices may have different screen resolutions or other hardware-specific issues that cause UI errors.
6. **Use third-party tools:** There are many third-party tools available to diagnose UI issues. For example, Reveal or FLEX can be useful for inspecting the view hierarchy and identifying layout issues.

Improving iOS app UI performance can have a significant impact on the user experience. Here are some tips to improve UI performance:

- **Optimize graphics and animations:** Graphics and animations can significantly impact UI performance. Use the appropriate image formats and sizes to reduce the file size and avoid using too many animations or effects that can slow down the UI.
- **Implement lazy loading:** Lazy loading loads only the necessary data and UI elements when needed. This can significantly improve performance by reducing the amount of data loaded upfront.
- **Use background threads:** Long-running tasks can block the main UI thread and make the app unresponsive. Use background threads to perform resource-intensive tasks such as data processing or file operations.
- **Test on real devices:** Testing your app on real devices can help you identify performance issues that may not be apparent on simulators. Use Xcode's profiling tools to monitor app performance and identify any bottlenecks or areas for improvement.

Q47. How can we reduce the iOS app launch time?

Answer: There are several ways to reduce app launch time in iOS. Here are some suggestions.

Improving launch code:

Make sure that your code is efficient and doesn't include unnecessary processes or calculations that could slow down the app launch. You can use profiling tools like Instruments to identify optimization areas.

For example, if you're doing something during the launch that can happen later, do it later.

```
// Preferred way
private lazy var settingsViewController: MySettingsViewController = {
    return MySettingsViewController()
}()

// Can be avoid
private func onSettingsButtonPressed() {
    present(self.settingsViewController, animated: true)
}
```

Use background threads:

If something doesn't need to happen on the main thread, offload it onto a background thread. This is so the main thread can show your UI. Be careful, because much code is not thread-safe or designed to run only on the main thread.

Compromise on behavior:

At the end of the day, the more your app tries to do when it launches, the slower it will launch. Negotiate with your team to see if there's anything user-facing you're willing to change or remove to get a faster launch time.

Reduce dependencies:

The more dependencies your app has, the longer it will take to load. Look for ways to reduce dependencies in your app.

Use Launch Screens:

Launch screens are placeholder screens that display when your app launches. By using Launch Screens, you can create the illusion that your app is launching faster, even if there is some delay in loading the app's content.

Optimize Images and Videos:

Images and videos can be large and slow to load. Optimize them by reducing their size and compressing them without compromising their quality.

Minimize App Size:

Keep the app size as small as possible by removing unnecessary resources such as unused images, libraries, or code. This reduces the time for the app to load into memory.

Dependency Management

Q48. Difference between SPM, CocoaPods, and Carthage.

Answer: These are popular dependency managers for iOS/macOS/tvOS/watchOS development, but there are some differences in the manner they are approached and implemented :

Swift Package Manager (SPM):

Advantages:

- SPM is built into Xcode, which makes it very easy to use and integrate into an Xcode project.
- SPM can handle both Swift and C-family dependencies and is designed to manage Swift packages.
- SPM is lightweight, fast, and efficient.
- Using a manifest file, SPM can automatically resolve dependencies and manage versions.

Disadvantages:

- SPM has limited support for third parties.
- SPM is designed for managing Swift packages, so it may not be the most suitable option for projects that use both Swift and Objective-C.

CocoaPods:

Advantages:

- CocoaPods has a large library of third-party libraries, which makes it very easy to find and install dependencies.
- CocoaPods can handle both Swift and Objective-C dependencies.
- CocoaPods is highly customizable and can handle complex dependencies and configurations.
- CocoaPods integrates well with Xcode and has good documentation.

Disadvantages:

- CocoaPods can be slow to install and update dependencies, especially when the dependencies

are large or complex.

- The use of a third-party service for dependency management means that there is a potential for downtime or compatibility issues.

Carthage:

Advantages:

- Carthage is very simple to use and integrate into an Xcode project.
- Carthage is designed to be fast and efficient.
- Carthage can handle both Swift and Objective-C dependencies.
- Carthage only builds the dependencies, which means the project remains lightweight and the build times are faster.

Disadvantages:

- Carthage requires developers to manually integrate dependencies into their projects, which can be time-consuming and error-prone.
- Carthage doesn't have a library of third-party libraries like CocoaPods, which means that it can be more challenging to find and install dependencies.
- Carthage doesn't manage versions, so developers must manually specify the version numbers for each dependency.

Persistent Storage

Q49. What are the different ways to persist data in iOS applications and when to use them?

Answer: You persist certain data such as bookmarking your favourite content, keeping score and status in games, or updated application settings etc.. This doesn't only keeps the application from poor User Experience but some data is highly sensitive (like login credentials in Keychain) in nature which should be persisted in order to perform certain task in the app.

In iOS, there are several ways to persist the data of your application but they come with different tradeoffs, so it can be tricky to choose the right one, especially if you're a newbie to iOS development.

Let's explore and understand the commonly used storages in detail.

UserDefaults:

For persistent data in key-value pairs, `UserDefaults` is the most commonly used storage. By providing a key, you can save whatever value you want. Values can be stored for most data types, but keys must always be strings. For example, you can store the login state of a user in the application and other application settings etc.

However, `UserDefaults` is easy to use, thread-safe, and shared between apps and app extensions. There are some cases where you should avoid using `UserDefaults`, such as storing complex data types and large files, etc.

Note: It is not recommended to store critical or sensitive information in `UserDefaults` as it is not encrypted. So such information can be compromised.

Keychain:

Apple's Keychain is an encrypted and thread-safe storage system with robust APIs. It can be used to save passwords, login credentials, certificates, secure notes, access tokens, etc. Keychain data can be

shared across devices through iCloud sync.

The keychain is the most secure storage layer in iOS, which stores all items in an encrypted state. It is not recommended to store large files or a lot of information on the Apple keychain. As the keychain does not provide a user-friendly API, you might be depending on an open source library.

File System:

Apple provides an easy and thread-safe environment for reading, writing, and editing files.

FileManager provides a Document Directory (also called a Sandbox Directory) to perform actions on files. A large file can be stored or cached, including documents, images, and videos. For example, a music app can store songs for listening offline.

In order to sync the app data across multiple devices, `FileManager` can be used in conjunction with `iCloud` storage. A subdirectory can be created, files can be moved, copied, or removed. Remember that document URLs and paths can be error-prone. Also, reading and writing large files on the disk can be very slow.

Core DataAnswer:

There are simple ways to persist data using `UserDefaults` and `FileManager`. But if you want to store complex and lots of information with some advanced features in a structured way, in that case iOS provides a powerful framework called `CoreData`. This framework lets us model and work with large, complex datasets.

`CoreData` provides a set of advanced tools like query-free database, easy to read data on main thread, easy to use background thread for saving data, support for automatic migrations of the database between app releases, easy GUI to setup database modeling, etc.

However, `CoreData` is forced to use `NSManagedObject` subclasses for every new object in the database. It is sometimes difficult for `NSManagedObjectContext` to synchronize database changes.

 **Anytime Magic Tip:**

- UserDefaults is suitable to use to store small and non-sensitive information.
- FileManager is an ideal tool for storing large files to enhance the offline experience by caching them.
- The Keychain is the best option for storing very sensitive information in a secure and encrypted manner.
- CoreData is mostly used to store complex and large datasets in a structured manner.

Some Related Questions:

- What are the merits and demerits of each approach?
- What are some limitations of User Defaults?
- How do you choose the best data persistence approach for a given iOS application? What factors do you consider?

Q50. What is managed object context? Define delete rules in CoreData with the use cases.

Answer: Managed Object Context (MOC) is an essential component of the Core Data framework, which provides an in-memory scratchpad for working with managed objects. It acts as a bridge between the persistent store coordinator and managed objects. It keeps track of changes to the objects and providing a way to save or roll back changes.

What MOC is responsible for?

A managed object context maintains a collection of managed objects, which are instances of classes generated by the Core Data model editor. It tracks changes made to managed objects, and can be used to retrieve, insert, delete, and update objects in the underlying persistent store.

Delete Rules:

Delete rules in Core Data specify what should happen to related objects when objects are deleted. Relationships between objects in the Core Data model define delete rules. The different delete rules are:

- No Action
- Nullify
- Cascade
- Deny

Nullify:

Nullify is the most commonly used delete rule in Core Data. When you set a relationship to nullify, it means that when an object is deleted, the relationship with the deleted object will be set to null. This is useful in situations where you want to preserve data in other objects related to the deleted object.

Use case: For example, if you have a User object that has a relationship with multiple Post objects, and you delete the User object, you may want to set the relationship of the Post objects to null so that they don't reference the deleted User object anymore.

Cascade:

When you set a relationship to cascade, it means that when an object is deleted, any related objects will also be deleted. This can be useful in situations where you want to ensure that related objects are deleted when the main object is deleted.

Use case: For example, if you have a Playlist object that has a relationship with multiple Song objects, and you delete the Playlist object, you may want to also delete all the related Song objects.

Deny:

When you set a relationship to deny, it means that you cannot delete an object if it has a relationship with another object. This can be useful in situations where you want to prevent accidental deletion of objects that are still being used by other objects.

Use case: For example, if you have a Product object that has a relationship with an Order object, you

may want to prevent the deletion of the Product object if it's still being used by an Order object.

No Action:

When you set a relationship to no action, it means that Core Data will not take action when an object is deleted. This is useful in situations where you want to handle related object deletion manually in code.

Use case: For example, if you have a User object that has a relationship with multiple Comment objects, and you delete the User object, you may want to handle the deletion of the related Comment objects manually in code.

It's important to choose the right delete rule for each relationship in your data model. This is to ensure that your app behaves as expected and that you don't unintentionally delete important data.

Q51. Explain the benefits and limitations of using Core Data as the primary persistence framework in an iOS application.

Answer: Core Data is a framework that provides an infrastructure for storing, retrieving, and managing data in an iOS application. It is a powerful tool that can simplify data management in an application and provide several benefits, but it also has some limitations that developers need to consider.

Benefits of using Core Data

- **Simplified data management:** Core Data provides a high-level abstraction layer for data storage and retrieval, which makes it easier to manage complex data relationships and queries.
- **Efficient data storage:** Core Data can store large amounts of data efficiently and optimize access to that data by caching frequently accessed data in memory.

- **Support for multiple data stores:** Core Data supports multiple data stores, including SQLite, binary files, and XML, which makes it flexible and adaptable to different data storage needs.
- **Automatic change tracking:** Core Data provides automatic change tracking, which simplifies the process of managing updates to data and ensures that data remains consistent across the application.

Limitations of using Core Data:

- **Steep learning curve:** Core Data can be complex and challenging to learn, especially for developers who are new to iOS development or data management.
- **Debugging issues can be challenging:** Debugging issues in Core Data can be challenging, as errors can occur at various levels, including the Core Data stack, managed objects, and queries.
- **Performance issues:** While Core Data is efficient at storing and accessing large amounts of data, it can have performance issues when working with very large datasets or complex queries.
- **Thread safety issues:** Core Data is not thread-safe by default, which means that developers need to manage thread safety manually to avoid concurrency issues.

Counter Question :

Q: Compare Core Data with alternatives like Realm or SQLite?

Answer: Here is a comparison of Core Data, Realm, and SQLite:

Framework	Features	Strengths	Weaknesses
Core Data	High-level object-relational mapping framework	Automatic change tracking, data validation, and relationship management; supports multiple storage backends and integrates with other iOS technologies	Steep learning curve and debugging issues can be challenging
Realm	Mobile database with fast performance and strong typing	Automatic synchronization and conflict resolution for data changes; well-suited for applications that require real-time updates, fast queries, and seamless offline functionality	Limited support for complex data relationships; proprietary format can make it challenging to integrate with other persistence technologies
SQLite	Popular open-source relational database with efficient storage and retrieval of structured data	Wide range of query options and supports transactions and concurrency control; suitable for applications that require flexibility and scalability	Challenging to use for complex data relationships; managing changes to data models can require significant effort

Core Data provides a high-level object-relational mapping framework with automatic change tracking and relationship management, while Realm offers fast performance and automatic synchronization for real-time updates. SQLite is a popular open-source relational database with a wide range of query options and support for transactions and concurrency control. Developers should evaluate each framework's strengths and weaknesses to determine which one is best suited for their application's specific requirements.

Memory Management

Q52. How does ARC work in Swift?

Answer: Automatic Reference Counting (ARC) was introduced in Apple's LLVM 3.0 compiler to solve the headache of Apple platform developers, who had to deal with memory management. It's a memory management system that automatically tracks and manages the memory usage of objects.

How does ARC work?

It works at compile time. The compiler automatically adds and removes retain and release operations to the objects in the application, depending on how the objects are being used in the code.

```
class Car {
    var name : String
    var passenger : String?

    init(name: String, passenger: String?){
        self.name = name
        self.passenger = passenger
    }
}

func planATrip() {
    //retain operation for car
    let car = Car(name : "Rolls Royce", passenger: nil)
    //retain operation for newCar
    let newCar = car
    //release operation for car
    newCar.passenger = "Jack"
    //release operation for newCar
    print("Trip completed")
}
```

Whenever the car object is created in function `planATrip()`, compiler triggers a retain operation to keep a reference to the object, and after the last use of object, compiler triggers a release operation to free up the memory.

How does ARC keep track of references?

The core concept of ARC is to keep track of the number of references to an object. It will automatically release object memory when it is no longer needed. When an object is created, ARC sets its retain count to 1, indicating that there is one reference to the object. Whenever a reference to the object is added, such as by assigning the object to a variable, the retain count is incremented. Whenever a reference to the object is removed, such as by setting the variable to nil, the retain count is decremented.

What are Strong Reference Cycles?

A strong reference cycle occurs when two or more class instances hold references to each other, causing a memory leak. This can happen when two objects hold a reference to each other and do not let go of the reference even when it's no longer needed.

For example, consider a parent class and a child class. The parent class has a property that points to an instance of the child class. The child class has a property that points to an instance of the parent class. By default these references are strong until you do not add any other prefix to it.

Problem with a Strong Reference cycle

When a strong reference cycle occurs, the memory occupied by these objects cannot be deallocated because both objects hold strong references to each other, preventing the compiler from freeing up memory. This can lead to a memory leak, which can cause the app to crash or perform poorly.

How to avoid a Strong Reference cycle?

To avoid strong reference cycles, one can use keywords like `weak` and `unowned`. Weak references allow one object to reference another without holding a strong reference to it, while unowned references allow one object to reference another without holding a reference that needs to be released. Both of these keywords can help break strong reference cycles and avoid memory leaks.

Some Related Questions:

- What are some common scenarios where memory leaks can occur even with ARC?
- Can you explain the difference between strong and weak references?
- How does ARC handle memory management for class instances that have a superclass?

Q53. What is Copy-on-Write in Swift? Explain how to customize its implementation.

Answer: Copy-on-Write is a memory optimization technique that allows multiple objects or variables to share the same data until one of them tries to modify it. When the modification begins, the data is copied to a new location, and the modification is performed on the new copy. This technique can significantly reduce memory usage and improve performance.

Copy-on-Write is used extensively by the system frameworks and APIs, especially in collections such as Array and Dictionary. Copy-on-Write is the magic behind value types. For starters, consider the following example:

```
var numbers = [10, 20, 30]
let numbersCopy = numbers
numbers.append(40)

print(numbers) // [10, 20, 30, 40]
print(numbersCopy) // [10, 20, 30]
```

As you probably know, collections like Arrays have Value semantics, it means that unlike Reference types (which store a reference to the object), Value types store a copy of the object. In the above example, numbersCopy gets a copy of numbers.

However, if you need to implement your own custom class that uses Copy-on-Write, you can do so like below example:

```
final class Person {
    var name: String
    init(name: String) {
```

```

        self.name = name
    }
}

struct PersonCOW {

    private var person: Person

    init(name: String) {
        person = Person(name: name)
    }

    var name: String {
        get {return person.name}
        set {
            if !isKnownUniquelyReferenced(&person) {
                person = Person(name: newValue)
            } else {
                person.name = newValue
            }
        }
    }
}

var person1 = PersonCOW(name: "John")
var person2 = person1
let names = ["Brian", "Stewie", "Peter"]
for name in names {
    person1.name = name
}

print(person1.name) // Peter
print(person2.name) // John

```

Here: `isKnownUniquelyReferenced` is a method provided by Swift's standard library that checks whether a given object is uniquely referenced in memory.

The name property's getter will simply return the name, in that case, we don't have to do anything complicated with it.

The magic happens in our setter. We first use our incantation `isKnownUniquelyReferenced` to determine whether there are other objects pointing to the same memory location.

Few scenarios where Copy-on-Write may be particularly useful:

- **Large Data Sets:** If you have a large amount of data that needs to be shared between multiple objects, copying the data every time it is accessed can be expensive in terms of both time and memory usage. Copy-on-Write can reduce the amount of memory needed to store data by sharing it between objects until one modifies it.
- **Immutable Data:** If the data you share between objects is immutable, there is no risk of unintended modification. In this case, Copy-on-Write can avoid unnecessary copying and improve performance.

Few examples of where Copy-on-Write could be used in iOS:

- **Text editing app:** If you're building a text editing app, you could use Copy-on-Write to optimize memory usage when multiple edits are made to the same piece of text. For example, if you have a large string shared between multiple views, you could use Copy-on-Write to ensure that the string is only copied when modified.
- **Graphics editing app:** If you're building a graphics editing app, you could use Copy-on-Write to optimize memory usage when multiple objects are based on the same underlying data. For example, if you have a set of shapes based on the same path, you could use Copy-on-Write to ensure that the path is only copied when it's modified.

Q54. How do you debug memory leaks in iOS? Can you walk me through the process of finding and fixing a memory leak in your code?

Answer: Debugging memory leaks in iOS can be a challenging task, but there are several techniques and tools available to help you identify and fix memory leaks in your code. Here's a general process you can follow to debug memory leaks in iOS:

Step 1 - Use Instruments:

The first step in debugging memory leaks is to use Instruments, a powerful tool included with Xcode that allows you to profile your app's memory usage. To use Instruments, choose "Profile" from the Xcode menu, select "Leaks" from the list of available templates, and then run your app. Instruments will monitor your app's memory usage and identify any leaks that occur.

Step 2 - Analyze the results:

Once Instruments has completed its analysis, you can analyze the results to identify any memory leaks. The Leaks instrument will show you the memory addresses of any leaked objects, as well as the call stack that led to the allocation of those objects. This can help you identify where the leak is occurring in your code.

Step 3 - Identify the cause of the leak:

Once you've identified a memory leak, you'll need to determine the cause of the leak. Look at the call stack in Instruments to identify the code that's responsible for allocating the leaked object. Make sure that you're properly releasing any objects that you allocate, and check for any circular references that might be preventing objects from being deallocated.

Step 4 - Fix the leak:

Once you've identified the cause of the leak, you can take steps to fix it. One common approach is to use autorelease pools to ensure that objects are released when they're no longer needed. You can also use weak references to avoid circular references that can prevent objects from being deallocated.

Step 5 -Test and iterate:

Once you've made changes to your code to fix the memory leak, be sure to test your app thoroughly to ensure that the leak has been eliminated. If necessary, repeat the process until you're confident that all memory leaks have been fixed.

Detect memory leaks using Xcode's memory graph debugger:

The Xcode Memory Graph Debugger is a powerful tool for detecting memory leaks and retain cycles in iOS apps. By capturing a GPU frame and analyzing the object graph and memory allocations, developers can identify objects that should have been deallocated but are still in memory, highlighted in red.

Anytime Magic Tip:

Overall, the process of debugging memory leaks in iOS involves using Instruments to identify leaks, analyzing the results to identify the cause of the leak, fixing the leak, and testing your app to ensure that the leak has been eliminated. It's important to be patient and methodical in your approach, as memory leaks can be difficult to track down and fix.

Keep in mind that memory leaks can be caused by third-party libraries or frameworks that you're using in your app, so it's important to check their documentation for any memory management guidelines.

Design Patterns

Q55. Which UICollectionView API implements the Factory Design Pattern?

Answer: The UICollectionViewDelegateFlowLayout protocol in the UICollectionView API implements the Factory design pattern.

The UICollectionViewDelegateFlowLayout protocol defines methods for providing layout information to a collection view. This includes the size of items, spacing between items, and section insets. When implementing this protocol, you act as a factory for creating and configuring collection view layout.

The Factory design pattern is a creation pattern that provides an interface for creating objects in a superclass. However, it allows subclasses to alter the type of objects created. In the case of the UICollectionViewDelegateFlowLayout protocol, the interface is defined by methods that provide layout information to the collection view, such as

```
func collectionView(_ collectionView: UICollectionView, layout
collectionViewLayout: UICollectionViewLayout, sizeForItemAt indexPath:
IndexPath) -> CGSize {
    // write code here...
}

func collectionView(_ collectionView: UICollectionView, layout
collectionViewLayout: UICollectionViewLayout, insetForSectionAt section: Int) ->
UIEdgeInsets {
    // write code here...
}
```

Overall, the UICollectionViewDelegateFlowLayout protocol is a powerful tool for creating and configuring the layout of a collection view. Its implementation of the Factory Design Pattern allows for flexibility and customization in the types of items created and configured.

Q56. Explain Factory design pattern usage and what problem does this pattern solve.

Answer: The Factory design pattern is a creational pattern that provides an interface for constructing

objects in a superclass. However, it allows subclasses to alter the type of objects created. This pattern solves the problem of creating objects without exposing the creation logic to the client. It decouples the client code from the specific classes being used.

Use of factory design pattern:

iOS development uses the Factory pattern for creating and managing different types of objects, such as views, controllers, or models. By using a Factory class to create these objects, the code becomes more flexible and easier to maintain.

A scenario to use it:

For example, imagine an app that displays different types of charts, such as pie charts, bar charts, and line charts. Instead of creating each type of chart directly in the view controller, a ChartFactory class can be used to create the different types of charts. This allows the creation logic to be encapsulated in one place, making it easier to change or add new types of chart in the future.

Additionally, the Factory pattern can also improve code testability by allowing easier dependency injection. Instead of directly instantiating objects, the Factory class can be injected as a dependency. This makes it easier to replace or mock creation logic during testing.

Here's an example of how the Factory pattern can be used in iOS development:

First, we create a ProductFactory protocol that defines a method for creating product view controllers:

```
protocol ProductFactory {  
    func createProductViewController(for product: Product) -> UIViewController  
}
```

Next, we create a concrete implementation of the ProductFactory protocol for each type of product. For example, here's a BookProductFactory that creates book view controllers:

```
class BookProductFactory: ProductFactory {  
    func createProductViewController(for product: Product) -> UIViewController {  
        let book = product as! Book  
    }
```

```

        let bookViewController = BookViewController()
        bookViewController.book = book
        return bookViewController
    }
}

```

Finally, in the main view controller, we can use the ProductFactory to create the appropriate product view controller based on the selected product:

```

class MainViewController: UIViewController {
    var productFactory: ProductFactory!
    var selectedProduct: Product!

    func showSelectedProduct() {
        let productViewController =
productFactory.createProductViewController(for: selectedProduct)
        navigationController?.pushViewController(productViewController,
animated: true)
    }
}

```

By using the Factory pattern, we've encapsulated the creation logic for each type of product view controller in separate classes, making it easier to maintain and extend the code in the future.

Concurrency

Q57. What is the difference between an Operation queue and a Dispatch queue and which one should we use?

Answer: Dispatch and Operation queue are both mechanisms for concurrency and parallelism in programming, but they have different trade-offs and use cases. Apple generally recommends developers to use the highest level of abstraction that is available and according to that recommendation, Operation queues must be the first choice but choosing one over the other largely depends on the specific requirements of the task at hand.

Key differences between Dispatch and Operation queues are:

- **Cancellation:** Operation queues provide a built-in mechanism for cancelling operations, whereas cancelling a task in Dispatch queues requires manual implementation.
- **Priority:** Dispatch queues have the concept of different priority levels, whereas Operation queues have a single queue and prioritise tasks based on the order in which they are added to the queue.
- **Dependency Management:** Operation queues provide a built-in mechanism for managing dependencies between operations, whereas Dispatch queues do not provide such a mechanism and it has to be handled manually.
- **Abstraction level:** Operation queues work on top of GCD. It provides the highest level of abstraction whereas Dispatch Queues belongs to GCD i.e. a low-level C API.

When to use Operation Queue?

When you want to perform several tasks in a specific order and want to manage the operations dependency on each other with the flexibility of cancelling them at any given point then Operation queues is a good solution. Though, going with this approach can sometimes lead to performance issues because of the `NSOperation` API inherent overhead.

When to use Dispatch Queue?

When you need to execute a set of tasks that don't have dependencies between them and can be executed in parallel. When you need to specify the priority level of each task, which determines its order of execution relative to other tasks. You can consider the Dispatch Queue. It is a low-level C API and quite better in performance compared to the Operation queue.

Some Related Questions:

- What is the main difference between a serial and a concurrent dispatch queue?
- Can you explain the concept of a barrier task in GCD and its usage?
- How do you execute a block of code asynchronously on a dispatch queue?
- How can you ensure that a task is executed on the main (UI) thread using GCD?

Q58. Explain Synchronous and Asynchronous operations in Swift.

Answer:

Synchronous Operations:

Synchronous operations are executed in a blocking manner, meaning the program execution will pause until the operation is complete. This means that the program will not proceed to the next line of code until the current operation has finished. Synchronous operations are useful when you need the result of the operation before continuing with the rest of the code.

Here's an example of a synchronous operation that waits for a file to be downloaded before proceeding:

```
if let url = URL(string: "https://example.com/image.png") {  
    do {  
        let data = try Data(contentsOf: url)  
        let image = UIImage(data: data)  
    }  
}
```



```

        // Use the downloaded image here
    } catch {
        // Handle the error here
    }
}

```

Another example of a synchronous operation:

```

let data = try? Data(contentsOf: URL(string: "https://example.com/data.json")!)
let json = try? JSONSerialization.jsonObject(with: data!, options: []) as?
[String: Any]
// Use the JSON data here

```

In this example, we're synchronously downloading a JSON file from a URL and deserializing it into a dictionary. The `Data(contentsOf:)` method blocks the current thread until the data has finished downloading, and the `JSONSerialization.jsonObject(with:options:)` method blocks the current thread until the deserialization is complete. This means that program execution will pause until both of these operations have completed.

Asynchronous Operations :

Asynchronous operations are executed in a non-blocking manner, meaning program execution continues immediately after the operation is initiated. This allows the program to execute other tasks while the operation is being performed. Asynchronous operations are useful when you don't need the result of the operation immediately and want to avoid blocking the main thread, which can cause the app to become unresponsive.

Here's an example of an asynchronous operation that uses a completion handler to execute code after the operation is complete:

```

if let url = URL(string: "https://example.com/image.png") {
    let task = URLSession.shared.dataTask(with: url) { data, response, error in
        if let data = data, let image = UIImage(data: data) {
            // Use the downloaded image here
        } else {
            // Handle the error here
        }
    }
}

```

```
}  
    task.resume()  
}
```

In this example, the `dataTask` function starts a background download of the image and calls the completion handler when the download is complete. Meanwhile, program execution continues immediately after the `dataTask` function is called. The completion handler is executed on a background thread, which means you can safely update the user interface from it.

Anytime Magic Tip :

Synchronous:

- Use synchronous operations when you need to block the current thread and wait for a result before proceeding.
- Avoid using custom synchronous operations on the main thread, as this can cause the app to become unresponsive.
- Handle errors that can occur during synchronous operations, as they can block the current thread and cause the app to crash.

Asynchronous:

- Use asynchronous operations when you need to perform long-running tasks in the background without blocking the main thread.
- Use completion handlers or delegates to process the results of asynchronous operations when they are complete.
- Update the user interface on the main thread, as most UIKit classes are not thread-safe and can cause the app to crash if accessed from a background thread.
- Handle errors that can occur during asynchronous operations, as they can cause unexpected behaviour in the app.

Q59. How does DispatchWorkItem perform actions?

Answer: DispatchWorkItem is a class that represents a task that can be executed on a DispatchQueue. DispatchWorkItem instance encapsulates a block of code (i.e., a closure) that can be executed synchronously or asynchronously. The closure is executed on the thread associated with the specified DispatchQueue.

Here's an example of creating a DispatchWorkItem and executing it asynchronously:

```
func incrementNumber() {  
  
    var number: Int = 5  
  
    let workItem = DispatchWorkItem {  
        number += 5  
    }  
  
    workItem.notify(queue: .main) {  
        print("After dispatching the item, the number is \(number).")  
    }  
  
    let queue = DispatchQueue.global(qos: .utility)  
    queue.async(execute: workItem)  
}  
  
incrementNumber()  
// print: After dispatching the item, the number is 10.
```

In the above example, `queue.async(execute: workItem)` ensures that WorkItem is executed asynchronously.

Here is an example of how to prioritize work items:

```
func sampleFunction() {  
  
    // create a high-priority & a low-priority queue  
    let highPriorityQueue = DispatchQueue.global(qos: .userInteractive)  
    let lowPriorityQueue = DispatchQueue.global(qos: .utility)  
  
    // create a high-priority work item
```

```

let highPriorityWorkItem = DispatchWorkItem(qos: .userInteractive) {
    sleep(5)
    print("High-priority task completed")
}

// create a low-priority work item
let lowPriorityWorkItem = DispatchWorkItem(qos: .utility) {
    sleep(5)
    print("Low-priority task completed")
}

// execute the work items asynchronously on their respective queues
lowPriorityQueue.async(execute: lowPriorityWorkItem)
highPriorityQueue.async(execute: highPriorityWorkItem)
}

sampleFunction()

// Output:
// print: High-priority task completed
// print: Low-priority task completed

```

In the preceding example, we execute the work items asynchronously on their respective queues using the `async(execute:)` method. Since the `highPriorityWorkItem` has a higher QoS level, it will be given higher priority by the system. It is likely to finish executing before the `lowPriorityWorkItem`.

Note:

- Avoid using `sleep()` or other blocking operations in your work item, as this can cause the system to create additional threads to compensate for the blocked thread.
- Be aware of retain cycles when working with `DispatchWorkItem` and closures. If you capture a reference to `self` inside a closure executed by a work item, make sure to use `[weak self]` or `[unowned self]` to avoid a retain cycle.

Q60. What are the different types of queues provided by

GCD?

Answer: Grand Central Dispatch (GCD) is an API used in iOS to implement concurrent programming. GCD provides several types of queues to manage task execution:

Serial queues:

These queues execute tasks one at a time in the order they were added to the queue. Each task must be completed before the next one can begin. Serial queues are useful when you need to perform tasks in a specific order.

```
let serialQueue = DispatchQueue(label: "com.example.serialQueue")

serialQueue.async {
    print("Task 1")
}

serialQueue.async {
    print("Task 2")
}

serialQueue.async {
    print("Task 3")
}

// Output:
Task 1
Task 2
Task 3
```

Since this is a serial queue, each task will be executed one at a time in the order they were added to the queue.

Concurrent queues:

These queues can execute multiple tasks simultaneously. Tasks are executed in the order they were added to the queue, but multiple tasks can run concurrently. Concurrent queues are useful when you

have independent tasks that can be executed in any order.

```
let concurrentQueue = DispatchQueue(label: "com.example.concurrentQueue",
    attributes: .concurrent)

concurrentQueue.async {
    print("Task 1")
}

concurrentQueue.async {
    print("Task 2")
}

concurrentQueue.async {
    print("Task 3")
}
```

Since this is a concurrent queue, each task can be executed simultaneously, and the output order may vary.

Main queue:

This is a special serial queue that executes tasks on the main thread of the application. The main queue is used for UI-related tasks, such as updating the user interface.

```
DispatchQueue.main.async {
    // Update UI here
}
```

Global queues:

These are concurrent queues managed by the system. They are divided into several quality-of-service (QoS) levels, which determine the priority of the tasks in the queue. QoS levels include user-interactive, user-initiated, utility, background, and default. Global queues are useful when you want to perform tasks in the background or when you don't care about the order in which tasks are executed.

```

let globalQueue = DispatchQueue.global(qos: .userInitiated)

globalQueue.async {
    print("Task 1")
}

globalQueue.async {
    print("Task 2")
}

globalQueue.async {
    print("Task 3")
}

```

Since this is a global queue, the tasks can be executed concurrently and the order of output may vary depending on the system load and QoS level.

Q61. Differentiate between dispatch group and dispatch semaphore.

Answer: Although both `DispatchGroup` and `DispatchSemaphore` can be used for managing concurrent tasks, they have different use cases and functionalities. Here are some key differences between them:

- **Purpose:** `DispatchGroup` is used to monitor the completion of a group of tasks, while `DispatchSemaphore` is used to manage concurrent access to a shared resource.
- **API:** `DispatchGroup` is a higher-level API than `DispatchSemaphore`. It provides a more abstract interface for managing groups of tasks, while `DispatchSemaphore` provides a lower-level interface for managing concurrent access to a shared resource.
- **Notification:** `DispatchGroup` provides a way to get notified when all tasks in the group have been completed, while `DispatchSemaphore` does not provide any notification mechanism.
- **Counting:** `DispatchSemaphore` uses a counting mechanism to manage access to a shared resource, while `DispatchGroup` does not use a counting mechanism.

- **Blocking:** DispatchSemaphore can block thread execution until a resource becomes available, while DispatchGroup does not provide any blocking mechanism.

Example of using DispatchGroup:

Suppose you want to download some images for asynchronous tasks to run in parallel. You want to perform some action when all of them have been completed. Here's how to use DispatchGroup to achieve this:

```
let downloaderGroup = DispatchGroup()

downloaderGroup.enter()
ImageDownloader.downloadImage("https://picsum.photos/id/10/2500/1667") { image,
urlString in
    downloaderGroup.leave()
}

downloaderGroup.enter()
ImageDownloader.downloadImage("https://picsum.photos/id/11/2500/1667") { image,
urlString in
    downloaderGroup.leave()
}

downloaderGroup.enter()
ImageDownloader.downloadImage("https://picsum.photos/id/12/2500/1667") { image,
urlString in
    downloaderGroup.leave()
}

downloaderGroup.notify(queue: DispatchQueue.main) {
    // perform further actions here...
    print("All tasks have completed.")
}
```

In this example, we created a dispatch group and added each task to the group using the `enter()` method. Each task has a completion block that calls the `leave()` method to signal the task completion. We then use the `notify()` method to specify a closure to be executed when all tasks in the group have been completed.

Note: In the ImageDownloader class, a function is defined to download the image. You can use your own approach whatever you prefer.

Example of using DispatchSemaphore:

Suppose you have a network service that can handle up to five concurrent requests at a time. Here's how to use DispatchSemaphore to limit concurrent requests:

```
let semaphore = DispatchSemaphore(value: 5)

func sendRequest() {
    semaphore.wait()
    // Perform network request here
    // ...
    semaphore.signal()
}

for i in 1...10 {
    sendRequest()
}
```

In this example, we created a dispatch semaphore with an initial value of 5, which means that up to five requests can be handled concurrently. We then define a function `sendRequest()` that performs a network request and acquires a semaphore before executing the request. The semaphore ensures that no more than five requests can be sent concurrently. Finally, we use a loop to send 10 requests by calling the `sendRequest()` function. The semaphore ensures no more than five requests are sent at any given time.

Q62. What do you mean by race conditions? What are the techniques that you can use to avoid it?

Answer: Race conditions can occur when two or more threads or processes access a shared resource or variable concurrently. The behavior of the app depends on the timing or order of their execution.

This can lead to unexpected results, crashes, or security vulnerabilities.

For example, imagine an iOS app that displays a list of items fetched from a remote server. If two or more threads try to update the same list simultaneously, without proper synchronization or locking mechanisms, the app may display duplicate or missing items. It may also crash due to inconsistent data.

There are several techniques that developers can use to avoid race conditions in their apps:

- **Use Synchronization Mechanisms:** Synchronization mechanisms such as locks, semaphores, or barriers can be used to ensure that only one thread can access a shared resource at a time. This can help prevent race conditions by enforcing a sequence of access to shared resources.
- **Use Atomic Operations:** It's an Objective-C API attribute. Atomic operations are thread-safe operations that ensure that a variable can only be modified by one thread at a time. You can use the `@atomic` attribute to declare a property thread-safe.
- **Use Dispatch Queues:** Dispatch queues can be used to manage concurrent tasks and ensure their execution in deterministic order. You can use serial or concurrent queues to control access to shared resources or perform tasks in a particular order.
- **Use Operation Queues:** Operation queues provide higher-level abstraction than dispatch queues, and can be used to manage more complex tasks. You can use dependencies between operations to ensure tasks are executed in a specific order.
- **Avoid Shared Mutable State:** Race conditions can often occur when multiple threads access the same mutable state. To avoid this, you should try to minimize the amount of shared mutable states in your app. Instead, use immutable data structures or thread-safe data structures such as `NSCache` or `NSMapTable`.
- **Test Your Code:** Finally, it is important to test your code thoroughly to ensure it is free from race conditions. You can use tools such as Xcode's Thread Sanitizer or the iOS simulator to simulate concurrent execution and identify potential race conditions.

Q63. What is Thread Explosion in Swift?

Answer: Thread explosion refers to a situation where a large number of threads are created, leading to performance issues, such as increased CPU usage, memory usage, and decreased responsiveness of the application.

A thread can be created using the Grand Central Dispatch (GCD), which provides a simple way to create and manage a thread. However, if GCD is not used properly, it can result in thread explosion. Some of the common causes of thread explosion include:

- If you create too many threads, it can overwhelm the system and cause performance issues.
- If you create threads in a loop, it can quickly lead to thread explosion.
- If you create threads for short-lived tasks, it can result in a lot of threads being created and destroyed, leading to performance issues.

Counter Question:

Q: What are the means of prevention?

Answer: To prevent thread explosion, you can follow these best practices:

- **Use thread pooling:** Thread pooling is technique to maintain a collection of pre-allocated threads. Instead of creating new thread for each task, thread can be assigned from the thread pool & after task is done it can be returned to the pool so that it can be reused & doesn't need to be terminated.
- **Use Quality of Service (QoS):** Using QoS to prioritise tasks can help the system to constrain availability of resources and avoid creating unnecessary threads. We have `.userInteractive`, `.userInitiated`, `.utility` and `.background` among them `.utility` and `.background` has lowest priority.

Some Related Questions:

- What is the relation between threads and GCD?
- Explain how you can manage multiple threads?

Application Security

Q64. What is SSL Pinning and how to implement it in iOS?

Answer: SSL pinning is a security mechanism used to prevent man-in-the-middle (MITM) attacks by verifying that the server certificate presented during an SSL/TLS handshake matches a predefined certificate or public key.

In iOS, SSL pinning can be implemented using several approaches. One approach is to use the `NSURLSessionDelegate` protocol. This involves creating a custom `URLSessionDelegate` class and implementing the `urlSession(_:didReceive:completionHandler:)` method to validate the server certificate.

Note: Another approach is to use third-party libraries such as TrustKit or Alamofire. These libraries provide a high-level API for SSL pinning in iOS apps.

In either approach, the basic idea is to intercept the SSL/TLS handshake and validate the server certificate against a known public key or certificate. If the certificate does not match the expected value, the connection is terminated.

It is critical to note that SSL pinning can make it harder for attackers to perform MITM attacks, but it is not a foolproof solution. Attackers can still use advanced techniques such as certificate impersonation to bypass SSL pinning. Therefore, SSL pinning should be used in conjunction with other security measures such as certificate revocation checking and certificate transparency monitoring.

Here's an example of SSL pinning using URLSession in iOS:

First, you need to create a custom `URLSessionDelegate` class that implements the `URLSessionDelegate` protocol. In this example, we'll call the class `CustomURLSessionDelegate`.

```
class CustomURLSessionDelegate: NSObject, URLSessionDelegate {  
  
    // Implement the URLSessionDelegate method to validate the server  
    certificate  
    func urlSession(_ session: URLSession, didReceive challenge:  
        URLAuthenticationChallenge, completionHandler: @escaping  
        (URLSession.AuthChallengeDisposition, URLCredential?) -> Void) {
```

```

        guard let serverTrust = challenge.protectionSpace.serverTrust else {
            completionHandler(.cancelAuthenticationChallenge, nil)
            return
        }

        // Set the SSL pinning policy
        let policies = [SecPolicyCreateSSL(true, challenge.protectionSpace.host
as CFString)]
        SecTrustSetPolicies(serverTrust, policies as CTypeRef)

        // Get the server certificate
        guard let serverCertificate = SecTrustGetCertificateAtIndex(serverTrust,
0) else {
            completionHandler(.cancelAuthenticationChallenge, nil)
            return
        }

        // Set the expected public key or certificate data
        let pinnedCertificateData = // Load the pinned certificate data

        // Compare the server certificate with the pinned certificate
        if serverCertificate.isEqualToData(pinnedCertificateData) {
            let credential = URLCredential(trust: serverTrust)
            completionHandler(.useCredential, credential)
        } else {
            completionHandler(.cancelAuthenticationChallenge, nil)
        }
    }
}

```

In the `urlSession(_:didReceive:completionHandler:)` method :

1. First, check that the `serverTrust` parameter is not `nil`.
2. Then, we set the SSL pinning policy to only accept SSL connections for the current host.
3. Next, we get the server certificate and compare it with the expected certificate or public key data.
4. If the server certificate matches the pinned certificate, we create a `URLCredential` object and call the completion handler with the `.useCredential` disposition.
5. Otherwise, we call the completion handler with the `.cancelAuthenticationChallenge` disposition to terminate the connection.

To use this custom `URLSessionDelegate`, you need to create a `URLSessionConfiguration` object and set its delegate property to an instance of the `CustomURLSessionDelegate` class. Then, you can create a `URLSession` object with the configuration and use it to make network requests as usual.

```
// Create a URLSessionConfiguration object with the custom delegate
let sessionConfig = URLSessionConfiguration.default
sessionConfig.timeoutIntervalForRequest = 10
sessionConfig.httpCookieAcceptPolicy = .always
sessionConfig.urlCredentialStorage = .shared
sessionConfig.httpShouldSetCookies = true
sessionConfig.httpCookieStorage?.setCookies(cookies, for: url, mainDocumentURL:
nil)
sessionConfig.urlCache = URLCache(memoryCapacity: 0, diskCapacity: 0, diskPath:
nil)
sessionConfig.requestCachePolicy = .reloadIgnoringLocalAndRemoteCacheData
sessionConfig.networkServiceType = .background
sessionConfig.waitsForConnectivity = true
sessionConfig.requestCachePolicy = .reloadIgnoringCacheData
sessionConfig.urlCredentialStorage = nil

let delegate = CustomURLSessionDelegate()
let session = URLSession(configuration: sessionConfig, delegate: delegate,
delegateQueue: nil)

// Make a network request using the session
let url = URL(string: "https://example.com")!

let task = session.dataTask(with: url) { data, response, error in
    // Handle the response
}

task.resume()
```

In this example, we create a default `URLSessionConfiguration` object and set its delegate property to an instance of the `CustomURLSessionDelegate` class. Then, we create a `URLSession` object with the configuration and use it.

Importance of SSL pinning?

In the context of mobile apps, SSL pinning is particularly important because mobile devices are often used on untrusted networks, such as public Wi-Fi networks, where MITM attacks are more likely to occur. Without SSL pinning, an attacker could intercept the SSL/TLS handshake and present a fake server certificate to the app, thereby gaining access to sensitive data or performing unauthorized actions on behalf of the user.

Q65. Explain Asymmetric and Symmetric encryption.

Answer: In Swift, you can implement various type of encryption algorithms, including asymmetric and symmetric encryption. Here is a brief overview of how it works :

Symmetric Encryption

In this way, concealing that message is for the sender to add a password to it and provide the receiver with that password. This happens to be of certain threat because there's a high chance that whatever method the sender uses to send the password to the receiver, it's going to be compromised. It has been proven that this method is inefficient, which is why Asymmetric Encryption has been developed.

- Swift provides built-in support for symmetric encryption using the CommonCrypto framework, which includes functions for encrypting and decrypting data with symmetric keys.
- The AES algorithm is a popular symmetric encryption algorithm, and Swift's CommonCrypto framework provides functions for encrypting and decrypting data with AES, such as `CCCrypt()` and `CCCryptorCreateWithMode()`.
- To encrypt a message with symmetric encryption in Swift, a key must first be generated, then the message can be encrypted using the key. The same key must be used to decrypt the message.

Why Use Symmetric Encryption?

The reason why symmetric encryption is popular is that it's relatively simple. This makes it easy and quicker to execute. Generally, symmetric encryption is used for encrypting larger amounts of data.

Problem with Symmetric Encryption

All parties must share the encryption key to allow for data transfer, exposing symmetric encryption to key exhaustion problems. If effective rotation isn't maintained, there's a risk that the key might be leaked.

Asymmetric Encryption

In Asymmetric Encryption, both the sender and the receiver, have a key pair. That is a public key and a private key. Both parties share their public keys to one another and keep the private key only a secret to themselves. Whenever the sender (A) wants to send a message to the receiver (B), (A) encrypts the message using (B)'s public key, and for (B) to read the message, they have to decrypt the message using their very own secret private key.

A key pair can be generated using numerous cryptography algorithms (RSA algorithm, ECC algorithm etc). As a result, both generated keys are linked together but still can't be derived from each other.

Why Asymmetric Encryption is considered more Secure?

However, a significant reason why asymmetric encryption is considered more secure and reliable is because it doesn't involve the exchange of public keys between multiple parties. Even if a hacker gains access to a public key, there's no risk of them using it for decrypting the data (since the public key is used for encryption only), as they don't know the private keys.

Problem with Asymmetric Encryption

This has to do with the longer key lengths, and more importantly, the mathematical calculations involved in asymmetric encryption are considerably more complex, which means they require more CPU resources for decryption.

Q66. What are some best practices for handling user credentials in Swift?

Answer: Handling user credentials in Swift requires careful consideration of security and privacy best

practices. Here are some best practices to remember:

- **Use secure storage:** Store user credentials securely in the app using the iOS Keychain or third-party libraries.
- **Use HTTPS:** Use HTTPS to communicate with the server to ensure that user credentials and other sensitive information are not intercepted by unauthorized parties.
- **Use encryption:** Use encryption to protect user credentials and other sensitive data while in transit and at rest.
- **Don't store plaintext passwords:** Avoid storing plaintext passwords, instead use techniques such as password hashing and salting to protect user passwords.
- **Keep user data separate:** Keep user data separate from other app data to minimize the risk of data leaks.
- **Use two-factor authentication:** Consider implementing two-factor authentication to add an additional layer of security to the authentication process.
- **Provide clear security notifications:** Provide clear notifications to the user about security events such as successful logins, failed login attempts, and password changes.
- **Use a strong password policy:** Encourage users to use strong passwords that are at least 8-12 characters long, include a mix of upper and lowercase letters, numbers, and special characters, and avoid common patterns and phrases.
- **Implement rate limiting:** Limit login attempts to prevent brute-force attacks.
- **Avoid using third-party authentication libraries:** Do not rely on third-party authentication libraries unless they have been thoroughly vetted for security vulnerabilities.

By following these best practices, you can ensure that your app handles user credentials securely and protects user privacy.

Advanced Swift

Q67. Explain Access controls in Swift.

Answer: Access controls limits the accessibility of some part of your code from code in other files or modules. It defines how types(class, structs, enumeration), properties, functions and subscripts can be accessed in your project. Let's explore all the access controls in detail.

Note: In the explanation, we have used the term 'Entities' to collectively refer to all types, properties, functions, subscripts.

Open Access (least restrictive):

Entities marked as open are accessible from within the module as well as from other modules that import the module where the entity is defined. In addition, subclasses of an open class can be defined outside of the module where the class is declared, and they can override any of its methods or properties.

Public Access (more restrictive than open):

Entities marked as public are accessible from within the module as well as from other modules that import the module where the entity is defined, but subclasses defined outside the module cannot override its methods or properties.

Internal Access (module-level access control):

This is the default access control for any entity if you don't implement any, explicitly. It enables entities to be used within any source file from their defined module, but not in any source file outside of that module.

File-private Access (file-level access control)

It restricts the use of an entity to its own defined source file. Use file-private access to hide the implementation details of a specific piece of functionality when those details are used within an entire file.

Private Access (class-level access control):

It restricts the use of an entity to the enclosing declaration, and to extensions of that declaration that are in the same file. Use private access to hide the implementation details of a specific piece of functionality when those details are used only within a single declaration.

Counter Question:

Q: What's the difference between open and public access?

Answer: The difference between open and public access control is that open entities can be subclassed and overridden outside of the module where they are defined whereas public entities cannot. In other words, open access control allows for more flexibility of code, while public access control provides a higher level of security by limiting code modification.

Anytime Magic Tip :

- It's a wise practice to use the most restrictive access level possible for a given entity. This helps keep your code secure and maintainable.
- Be careful when using open access: While open access provides more flexibility than public access, it also comes with some risks. Subclasses defined outside of the module where the base class is defined can potentially modify the behavior of the base class in unexpected ways.
- When using access modifiers, be consistent throughout your codebase. This makes your code more readable and maintainable.
- Access control enforces encapsulation. This is the idea that an entity should only be accessible within its defined scope.

Q68. What is the difference between == and === in Swift?

Answer: In Swift, == and === are comparison operators used to check for equality between values, but they have different behaviours:

Equality Operator == :

The Equality operator == is used to check if two values are equal. It returns a Boolean value of true if the values are equal, otherwise false. It allows to check the equality between basic data types like (Int, Float, String etc.) but doesn't work well with custom data types. To make them work with custom types, you need to conform the type to Equatable protocol.

Example 1:

```
let name1 = "John"
let name2 = "John"
print(name1 == name2) // print: true
```

Example 2:

```
struct Point {
    let x: Int
    let y: Int
}

let circlePoint = Point(x: 5, y: 5)
let circlePoint2 = circlePoint
print(circlePoint == circlePoint2)
// error: binary operator '==' cannot be applied to two 'Point' operands
```

You can see the error above. The Point type obviously does not know about the Equality operator. To fix this issue, you have to overload this operator like below:

```
struct Point {
    let x: Int
```

```

let y: Int

static func ==(lhs: Point, rhs: Point) -> Bool {
    return lhs.x < rhs.x && lhs.y < rhs.y
}

let circlePoint = Point(x: 5, y: 5)
let circlePoint2 = circlePoint
print(circlePoint == circlePoint2) // print: true

```

Identity Operator === :

The Identity operator `===` is used to check if two values point to the same memory location. If both values point to the same memory, then it returns true, otherwise false. It can only be used with reference types, which mean only with classes.

```

class Student {
    let name: String
    init(name: String) {
        self.name = name
    }
}

let alex = Student(name: "Alex")
let alex2 = alex
print(alex === alex2) // print: true

```

Note:

- For custom types or to define custom comparisons, you can overload the `==` operator.
- For reference types, you cannot overload the `===` operator.

Q69. What is Protocol Oriented Programming? How is it

beneficial in Unit Testing?

Answer: Protocol-oriented programming (POP) is a programming paradigm in Swift that emphasizes the use of protocols to design and implement interfaces. In POP, protocols are used to define a set of methods, properties, and other requirements that can be adopted by classes, structs, and enums. By using protocols, you can write code that is more modular, reusable, and testable.

POP is particularly useful for unit testing because it allows you to write testable code that can be easily mocked and stubbed. When you define your classes and structs in terms of protocols, you can easily create mock objects that conform to those protocols and use them in your tests. This makes it easier to test individual components of your app in isolation, without having to worry about the behavior of other parts of the system.

Here's why POP can be beneficial for Unit Testing:

Loosely coupled Interfaces:

- With POP, you define the interface of a type in terms of a protocol, which means that the implementation of the type can be easily swapped out for a mock object or stub during testing.
- This makes it possible to test each component of your app in isolation, without relying on other components or external dependencies.

Modular Codebase:

- With POP, you can break down large, complex types into smaller, more focused protocols.
- This makes it easier to test individual components of your app in isolation, and to reuse those components in other parts of your app or in other apps.

More Composable Types:

- With POP, you can design types to conform to multiple protocols.
- This makes it possible to create more flexible and reusable code that can be composed in different ways to solve different problems.

- This can also make it easier to test individual components of your app in isolation, as you can create mock objects that conform to the necessary protocols.

Maintainable Codebase:

- With POP, code is typically more modular and composable, which can make it easier to maintain over time.
- By using protocols to define the interfaces between types, you can more easily swap out implementations or add new features without affecting other parts of the codebase.

Some Related Questions:

- What is the difference between protocol-oriented programming and object-oriented programming?
- What are the advantages of using protocol-based dependency injection in unit testing?

Q70. How to customly create a Higher-Order function in Swift?

Answer: Higher-Order function is a function that takes one or more functions as arguments or returns a function as its result. We can create Higher-Order functions using closures.

Let's customly create Higher-Order functions like `filter` and `reduce`.

Example 1: Filter Function

The filter function takes an array and a closure that returns a Boolean value. The closure specifies a condition that each element in the array should satisfy. The function returns a new array that contains only the elements from the input array that satisfy the condition.

```
func customFilter<T>(_ array: [T], _ isIncluded: (T) -> Bool) -> [T] {
```

```

var result: [T] = []
for element in array {
    if isIncluded(element) {
        result.append(element)
    }
}
return result
}

let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
let filteredEvenNumbers = customFilter(numbers) { $0 % 2 == 0 }
let filteredOddNumbers = customFilter(numbers) { $0 % 2 != 0 }
print(filteredEvenNumbers) // [2, 4, 6, 8, 10]
print(filteredOddNumbers) // [1, 3, 5, 7, 9]

```

In this example, the `customFilter` function is used to filter out all the even and odd numbers from an array of integers and assigned to `filteredEvenNumbers` and `filteredOddNumbers` respectively.

Example 2: Reduce Function

The reduce function takes an array and a closure that combines two elements of the array into a single value. The function applies the closure to each pair of elements in the array, accumulating a single result. The function returns the final accumulated value.

```

func customReduce<T, U>(_ array: [T], initialResult: U, nextPartialResult: (U, T) -> U) -> U {
    var result = initialResult
    for element in array {
        result = nextPartialResult(result, element)
    }
    return result
}

let numbers = [1, 2, 3, 4, 5]
let sum = customReduce(numbers, 0) { $0 + $1 }
let product = customReduce(numbers, 1) { $0 * $1 }
print(sum) // 15
print(product) // 120

```

In this example, the `customReduce` function is used to calculate the sum and product of an array of integers. The closure that's passed to the function takes two integers as input and returns their sum and product.

🔮 Anytime Magic Tip:

Higher-Order functions can have a performance impact, especially if they're called repeatedly or on large datasets. The overhead of calling and executing the closures passed to the function can add up, and the function may not be optimized for the specific use case.

Q71. Explain KVO with an example in Swift.

Answer: KVO stands for Key-Value Observing, and it is a mechanism in Swift (and Objective-C) that allows one object to observe changes to a property of another object. When the observed property changes, the observing object is notified.

To use KVO in Swift, you can follow these steps:

Annotate a Property for Key-Value Observing:

First, you need to make the observed property dynamic by marking it with the `@objc` and `dynamic` attributes. For example, if you have a `User` class with a `name` property that you want to observe, you can define it like this:

```
class User: NSObject {  
  
    @objc dynamic var name: String  
  
    init(name: String) {  
        self.name = name  
    }  
}
```

Define an Observer:

When you create an observer, you start observation by calling the

`observe(_:options:changeHandler:)` method with a key path that refers to the property you want to observe.

```
let alex = User(name: "Alex")
alex.observe(\User.name, options: [.old, .new]) { user, change in
    print("Old name: \(change.oldValue ?? "") and New name: \(change.newValue ??
    "")")
}
```

Respond to a Property Change:

Objects that are set up to use key-value observing—such as Alex above—notify their observers about property changes. The example below changes the `name` property by assigning a new value. That method call automatically triggers the observer's change handler:

```
alex.name = "Alex Murphy"

// Output:
// Old name: Alex and New name: Alex Murphy
```

The example above responds to the property change by printing both the new and old values of the name.

Note: KVO involves dynamic dispatch, which can be slower than static dispatch. It can also lead to increased memory usage and potentially slower performance due to the need for maintaining an additional state.

Some Related Questions:

- How does KVO work internally in Swift?

- What are the key components involved in implementing KVO?
- What is the significance of using the @objc dynamic attribute for KVO?
- Can you observe multiple properties of an object using a single observer in KVO?
- How can you prevent memory leaks when using KVO?

Q72. What is Protocol composition in Swift?

Answer: Protocol composition is a technique that allows you to combine two or more protocols into a single requirement. It is useful when you need to define a new requirement that involves several different behaviours or capabilities that are defined in separate protocols.

Let's understand the Protocol composition with Codable

```
typealias Codable = Encodable & Decodable
```

Swift provides a composition Protocol named `Codable`. This is a powerful and convenient feature that allows you to easily encode and decode data to and from various formats, such as JSON and Property List. It is a `typealias` for two protocols, `Encodable` and `Decodable`, which provide methods for encoding and decoding, respectively.

```
struct Person: Codable {
    let name: String
    let age: Int
}

let person = Person(name: "John", age: 30)
let encoder = JSONEncoder()
let data = try encoder.encode(person)
let jsonString = String(data: data, encoding: .utf8)
print(jsonString!) // {"name":"John","age":30}

let decoder = JSONDecoder()
let decodedPerson = try decoder.decode(Person.self, from: data)
print(decodedPerson) // Person(name: "John", age: 30)
```

By conforming to the `Codable`, you get the ability to encode and decode your objects to and from a variety of formats, without having to write the encoding and decoding code yourself. The `Codable` is a convenient and powerful feature of the Swift language, which makes it easy to work with data in various formats in your applications.

Here is an example of Protocol composition in Swift, which combines multiple protocols to define a new requirement:

Here are the protocols we have defined:

```
protocol Playable {
    func play()
}

protocol Recordable {
    func record()
}

protocol MediAnswer: Playable, Recordable {
    var title: String { get }
}
```

Here is an example of how to conform to the Media protocol:

```
class Song: Media {

    var title: String
    init(title: String) {
        self.title = title
    }

    func play() {
        print("Playing song: \(title)")
    }

    func record() {
        print("Recording song: \(title)")
    }
}

let song = Song(title: "Bohemian Rhapsody")
```

```
song.play() // print - Playing song: Bohemian Rhapsody
song.record() // print - Recording song: Bohemian Rhapsody
```

Protocol composition allows us to define a new requirement that involves multiple behaviors or capabilities specified in separate protocols. In this example, we combined the Playable and Recordable protocols to define the Media protocol, which requires objects to implement both playing and recording functionality, as well as a title property.

Note: As you compose more protocols to define a new requirement, the resulting code can become more complex and harder to understand.

Q73. What is Capture List in Swift?

Answer: A Capture list defines which variables and constants should be captured and how they should be captured by a closure. It is a comma separated list within square brackets and is written after the opening curly brace of closure and before the parameters.

```
lazy var callbackClosure = { [unowned self, weak delegate = self.delegate] in
    // closure body goes here
}
```

Here's an example of a closure that uses a capture list to capture a variable:

```
var x = 10
let closure = { [x] in
    print("x is \(x)")
}

x = 20
closure() // prints "x is 10"
```

In this example, the closure captures the value of the variable `x` at the time it is created, which is 10. Even though the value of `x` is changed to 20 later, the closure still prints the original value of 10 because that is what was captured in the capture list.

Capture multiple variables:

```
var x = 10
var y = 20

let closure = { [x, y] in
    print("x is \(x), y is \(y)")
}

x = 30
y = 40

closure() // prints "x is 10, y is 20"
```

Capturing a variable by reference:

```
class ExampleClass {

    var counter = 1

    lazy var closure: () -> Void = { [weak self] in
        guard let self = self else {
            return
        }
        print(self.counter)
    }
}

let instance = ExampleClass()
instance.closure() // prints 1
instance.counter += 1
instance.closure() // prints 2
```

Q74. Explain Recursive enumeration and Associated values.

Answer: Recursive enumeration is a type of enumeration where one or more of the values in the enumeration refers to the enumeration itself. In other words, an enumeration can be defined in terms of itself. This is useful when you need to represent a data structure that has a recursive structure, such as a tree or a factorial.

Let's understand recursive enumeration with an example.

Defining a recursive enum with different cases:

```
enum ArithmeticExpression {  
    case value(Int)  
    indirect case addition(ArithmeticExpression, ArithmeticExpression)  
    indirect case multiplication(ArithmeticExpression, ArithmeticExpression)  
}
```

When you define an enumeration with associated values, Swift creates a copy of the associated value for each instance of the enumeration that is created. This can result in unnecessary memory usage when the associated value is a large or complex data structure, such as a tree or a linked list.

The indirect keyword allows you to avoid this duplication by telling Swift to store the associated value indirectly, using a reference rather than a copy. This means that the associated value is not copied with each instance of the enumeration, but rather, a reference to the same value is used.

Evaluate the expression using the recursive function:

```
func evaluate(_ expression: ArithmeticExpression) -> Int {  
    switch expression {  
    case let .value(value): return value  
    case let .addition(left, right): return evaluate(left) + evaluate(right)  
    case let .multiplication(left, right): return evaluate(left) *  
        evaluate(right)  
    }  
}
```

Defining variables of type ArithmeticExpression:

```
let numberTwenty = ArithmeticExpression.value(20)
let numberFifteen = ArithmeticExpression.value(15)
let numberThree = ArithmeticExpression.value(3)
let sum = ArithmeticExpression.addition(numberTwenty, numberFifteen)
let product = ArithmeticExpression.multiplication(sum, numberThree)
```

In the above code, we define some variables of the ArithmeticExpression type to evaluate the `(20 + 15) * 3` expression using recursive enumeration.

Evaluate the expression:

```
let result = evaluate(product)
print(result) // print: 105
```

As you can see, you can solve a recursive problem using an enum. Recursive enumeration works similarly to a recursive function.

Associated values

In the ArithmeticExpression enum, the value case has an associated value of type Int, which represents the numeric value of the arithmetic expression. This allows us to store additional information with the value case, which we can use in our code.

Note:

- Recursive enumeration provides strong type safety, which helps to catch errors at compile-time.
- Recursive enumeration can make your code more concise and clear. By using an enumeration to represent a recursive data structure, you can avoid the need for complex class hierarchies or recursive functions, which can be difficult to understand and maintain.
- Using recursive enumeration with the indirect keyword can lead to more memory-efficient code.

- You can use it to represent not just trees, but also graphs, linked lists, and other recursive data structures.

Q75. Explain @autoclosure in Swift with an example.

Answer: In Swift, @autoclosure allows a function or closure parameter to be automatically wrapped in a closure.

When a parameter is marked with @autoclosure, the argument passed to that parameter is automatically wrapped in a closure. This means that the argument is not evaluated until used inside the function. This can improve performance, especially when working with large or complex expressions.

Example:

```
func sampleFunction(_ lhs: Bool, _ rhs: @autoclosure () -> Bool) -> Bool {
    if lhs {
        return true
    } else {
        return rhs()
    }
}

sampleFunction(false, 1 + 1 == 2) // returns true
```

In this example, sampleFunction takes two boolean arguments, and the rhs argument is marked with @autoclosure. If the lhs argument is true, the function returns true without evaluating the rhs argument. If lhs is false, the function evaluates the rhs argument by calling the closure.

Here is another @autoclosure example:

```
enum LogLevel: Int {
    case debug
    case info
    case warning
```

```

    case error
}

class Logger {
    var logLevel: LogLevel = .warning

    func log(_ level: LogLevel, _ message: @autoclosure () -> String) {
        if level.rawValue >= logLevel.rawValue {
            print("[\(level)] \message())")
        }
    }
}

let logger = Logger()

logger.log(.info, "This is an info message")
logger.log(.warning, "This is a warning message")
logger.log(.debug, "This is a debug message")
logger.log(.error, "This is an error message")

logger.logLevel = .debug

logger.log(.info, "This is an info message")
logger.log(.warning, "This is a warning message")
logger.log(.debug, "This is a debug message")
logger.log(.error, "This is an error message")

// Output:
/*
[warning] This is a warning message
[error] This is an error message
[info] This is an info message
[warning] This is a warning message
[debug] This is a debug message
[error] This is an error message
*/

```

This example demonstrates how `@autoclosure` delays the evaluation of the log message expression until it is actually needed. This can improve performance by avoiding unnecessary computation and make the code more readable by allowing you to provide simple, inline expressions for the log message.

Note: Using @autoclosure can be useful when you want to provide a default value for an optional parameter that requires computation to evaluate. By using @autoclosure, you can delay the evaluation of the default value expression until it is actually needed.

Q76. Explain the concepts of 'delegate & protocol' and 'notification & observer' in iOS. When would you use one over the other.

Answer:

Delegate & Protocol:

In iOS, it is a design pattern used to communicate between two objects. This is where one object needs to inform another object about events or changes that have occurred. The delegate pattern is implemented through a protocol. This defines a set of methods that the delegate object can implement to respond to specific events or changes.

Notifications & Observer:

Notifications are a way for objects to broadcast information to other objects without knowing who the recipients are. Notifications are based on the observer pattern and involve three objects: the object sending the notification (the "sender"), the object receiving the notification (the "observer"), and the notification center, which acts as a mediator between the sender and the observer.

Which one to choose?

In iOS development, delegates and notifications are two common design patterns used for communication between objects. While both patterns can be used for similar purposes, there are some situations where one pattern may be more appropriate than the other.

Use delegates when:

- There is a one-to-one relationship between the objects involved in the communication.
- The relationship between the objects is known in advance.
- The delegate object needs to customize the other object's behavior.
- The delegate object needs to control an operation's flow.
- The delegate object needs to validate or approve an action before performing it.

Use notifications when:

- There is a one-to-many relationship between the objects involved in the communication.
- The recipients of the information may change dynamically at runtime.
- The sender object does not need a response or acknowledgement from the recipient.
- The sender object does not need to know how the recipients will use the information.
- There is a need for loose coupling and concern separation between objects.
- The use of delegate or other communication patterns is not appropriate or feasible in the given situation.

In some cases, it may be possible to use delegates or notifications to accomplish a specific task. In these situations, the choice between the two patterns may depend on factors such as performance, scalability, and ease of implementation.

Q77. Explain the purpose of Hashable and why we should inherit from Equatable.

Answer:

Hashable protocol:

The Hashable protocol requires types to provide a hash value for each instance. A hash value is an integer value used to uniquely identify an instance. It is used to store and retrieve values in collections such as dictionaries and sets. By conforming to the Hashable protocol, a type guarantees that its instances can be used as keys in these collections.

Equatable protocol:

The Equatable protocol requires that a type implement the operator to compare instances for equality. By conforming to the Equatable protocol, a type can be compared for equality using the `==` operator. It can be used in algorithms that require comparing values for equality.

In Swift, it's common to inherit from Equatable when conforming to Hashable because the implementation of `hashValue` often depends on the implementation of `==`. By conforming to both protocols, a type can be used as a key in collections and can be compared for equality using the `==` operator.

```
struct Person: Hashable {

    let name: String
    let age: Int

    func hash(into hasher: inout Hasher) {
        hasher.combine(name)
        hasher.combine(age)
    }
}

let john = Person(name: "John", age: 30)
let jane = Person(name: "Jane", age: 25)
let john2 = Person(name: "John", age: 30)

let set: Set<Person> = [john, jane]

print(set.contains(john)) // returns true
print(set.contains(Person(name: "John", age: 30))) // also returns true because
they have the same properties

print(john == john2) // returns true because their properties are equal
print(john == jane) // returns false because their properties are not equal
```

 **Anytime Magic Tip:**

If you conform to the Hashable protocol, you don't need to explicitly import the Equatable protocol since Hashable already inherits from Equatable. The reason for this is that the Hashable protocol requires that conforming types provide an implementation of the `==` operator to check for equality. This is the same requirement as the Equatable protocol.

Q78. What are generics in Swift? Can you give an example of when you would use them?

Answer: In Swift, generics allow you to write flexible and reusable functions, structures, and classes that work with any type, rather than a specific type. You can use generics to write code that adapts to different types without rewriting the same code for each type.

Here's an example of how to use generics in Swift:

```
func swapTwoValues<T>(_ a: inout T, _ b: inout T) {
    let temp = a
    a = b
    b = temp
}

var x = 5
var y = 10
swapTwoValues(&x, &y)
print("x is now \(x), and y is now \(y)") // x is now 10, and y is now 5
```

This function takes two arguments of any type `T`, and swaps their values. The function uses the generic placeholder type `T` to represent the type of swapped values. The function's body works with any type that conforms to the Equatable protocol. This includes many of Swift's built-in types such as `Int`, `String`, and `Double`.

Here's another example:

```
func indexOf<T: Equatable>(item: T, inArray array: [T]) -> Int? {
    for (index, value) in array.enumerated() {
        if value == item {
            return index
        }
    }
    return nil
}
```



```

    }
}
return nil
}

let strings = ["foo", "bar", "baz"]
if let index = indexOf(item: "bar", inArray: strings) {
    print("Found item at index \(index)")
} else {
    print("Item not found in array")
}
// print: Found item at index 1

```

This function takes two arguments: an item of type `T` and an array of type `[T]`. The function uses the generic placeholder type `T` to represent the type of item being searched for. The `Equatable` constraint on the generic type `T` ensures that the `==` operator is defined for the type. This is so that the function can compare values of the type.

If you call the same function with a different type of item, such as an `Int`, `Double`, etc, it will work as well.

Here are some important points to know about generics in Swift:

- Generics allow you to write flexible and reusable code that works with any type, rather than a specific type.
- Generics use a placeholder type (usually denoted by the letter `T`) to represent the actual type used at runtime.
- You can use constraints to specify what types can be used with a generic function or type.
- The `Equatable` and `Comparable` protocols are commonly used constraints for generic functions that involve comparisons.
- Generics are commonly used in Swift's standard library, such as `Array` and `Dictionary` types.
- Generics can be used for functions, structures, classes, and protocols.

Q79. Explain the difference between Generics and Opaque

types in Swift.

Answer: Generics and opaque types are two features used for creating flexible and reusable code. Both of these features allow you to write code that works with different types without having to write separate code for each type. However, there are some differences between the two.

Generics:

Generics allow you to define functions, types, and protocols that work with any type. You can use placeholders, called type parameters, to represent the types used at runtime. The caller of the function will determine the concrete type. For example, consider the following generic functions:

```
func printArray<T>(array: [T]) {
    for item in array {
        print(item)
    }
}

let intArray = [1, 2, 3, 4, 5]
let stringArray = ["apple", "banana", "orange"]

printArray(array: intArray) // [1, 2, 3, 4, 5] -> print all element in new line
printArray(array: stringArray) // ["apple", "banana", "orange"] -> print all
element in new line
```

Opaque types:

Opaque types allow you to hide the specific implementation of a type while still working with it. With opaque types, you define a protocol that specifies the type's behavior, but you don't provide implementation details. The implementation is hidden and can be changed without affecting the code that uses the opaque type. The implementation will determine the concrete type. The `some` keyword indicates that the property has an opaque type. Here's an example:

```
protocol Displayable {
    func display()
}
```

```

struct Person: Displayable {
    func display() {
        print("Name: John")
    }
}

struct Car: Displayable {
    func display() {
        print("Make: Toyota, Model: Corolla")
    }
}

func printItem(item: some Displayable) {
    item.display()
}

let john = Person()
let corolla = Car()

printItem(item: john)
// Output: Name: John

printItem(item: corolla)
// Output: Make: Toyota, Model: Corolla

```

Q80. Difference between map and compact map in Swift.

Answer: In Swift, both `map` and `compactMap` are higher-order functions that operate on a sequence or collection and apply a transformation to each element.

The main difference between `map` and `compactMap` is in how they handle the transformation of nil values.

`map` transforms every element of the sequence using the provided transformation closure. It returns an array with the transformed elements, even if the transformation closure returns nil. This means that the resulting array may contain nil values.

`compactMap`, on the other hand, applies the transformation closure to each element of the sequence,

and returns an array containing only the non-nil results of the transformation. Any nil values produced by the transformation are automatically removed from the resulting array.

Here is an example that explain the difference:

```
let numbers = ["1", "2", "3", "4", "5", "notANumber"]

let mappedNumbers = numbers.map { Int($0) }
let compactMappedNumbers = numbers.compactMap { Int($0) }

print(mappedNumbers) // [1, 2, 3, 4, 5, nil]
print(compactMappedNumbers) // [1, 2, 3, 4, 5]
```

The map always returns an array of the same length as the original sequence, even if the transformation closure returns nil for some elements. The compactMap returns an array that may be smaller than the original sequence if the transformation closure returns nil for some elements.

It's worth noting that in some cases, you may prefer to use map and filter to remove nil values, rather than with compactMap. This can be useful if you need to perform additional operations on the transformed values before filtering out the nil. However, compactMap is a more concise and efficient way to remove nil values from a sequence.

Counter Question:

Q: How can we implement our own map function in Swift?

Answer: You can implement your own map function in Swift by writing a generic function that takes an array of a specific type and a closure expression. The closure expression specifies the transformation to be applied to each element in the array. The map function returns an array with the transformed elements.

Here's an example of creating your own map function:

```
func customMap<T, U>(array: [T], transform: (T) -> U) -> [U] {
    var result: [U] = []
    for item in array {
```

```

        result.append(transform(item))
    }
    return result
}

```

The map function is defined with two type parameters, T and U, which specify the input and output types of the transform closure, respectively. The function takes an array of type [T] and a closure transform of type (T) → U.

It then loops through each element in the input array, applies the transform closure to each element, and appends the result to an array, result. Finally, the result array is returned as the map function output.

Here is an example of how to use the above map function:

```

// input array
let numbers = [10, 20, 30, 40, 50]

// convert numbers into strings using the map function
let strings = customMap(array: numbers) { String($0) }

// printing the output
print("Output:", strings)

// Output: ["10", "20", "30", "40", "50"]

```

Here is an example of how to implement your own map function using Sequence's extension:

```

extension Sequence {
    func customMap<T>(_ transform: (Element) -> T) -> [T] {
        var result = [T]()
        for item in self {
            result.append(transform(item))
        }
        return result
    }
}

```

A utility function such as a `map` should be accessible through all collections in the extension.

Here is an example of how to use the above map function:

```
// input array
let numbers = [10, 20, 30, 40, 50]

// convert numbers into strings using the map function
let strings = numbers.customMap { String($0) }

// printing the output
print("Output:", strings)

// Output: ["10", "20", "30", "40", "50"]
```

Warning:

It is critical to ensure that the closure expression returns the correct type of value and avoids mutating the original collection in the closure expression.

The map function creates a new collection, so it can have performance implications for larger collections. Keep this in mind and consider using alternative solutions when dealing with large collections of data.

Q81. What is Conditional Conformance?

Answer: Conditional conformance was introduced in Swift 4, it's a feature that allows a type to conform to a protocol under certain conditions. For example, when Generic parameters meet certain constraints.

Here are some examples of how conditional conformance can be used in Swift:

Example 1:

```

extension Dictionary: CustomStringConvertible where Key:
CustomStringConvertible, Value: CustomStringConvertible {
    var description: String {
        return "{" + self.map { "\($0.key.description): \"\($0.value.description)\" }.joined(separator: ", ") + "}"
    }
}

let names = ["Alice": 25, "Bob": 30, "Charlie": 35]
print(names) // Output: {"Alice: 25, "Bob": 30, "Charlie": 35}

```

In this example, we extend the Dictionary type to conform to the CustomStringConvertible protocol if its Key and Value types also conform to CustomStringConvertible. This allows us to print a dictionary with CustomStringConvertible keys and values using the default string representation.

Example 2:

```

struct ValueWrapper<T> {
    let value: T
}

extension ValueWrapper where T == String {
    var stringValue: String {
        return self.value
    }
}

let wrapped = ValueWrapper(value: "This is a string")
print(wrapped.stringValue) // This is a string

```

Counter Question:

Q: What are the benefits of applying Conditional Conformance?

Answer: Conditional conformance has several benefits:

- **Reusability:** With conditional conformance, we can define a single implementation of a protocol for multiple types that meet certain conditions. This makes our code more reusable and reduces duplication.
- **Flexibility:** Conditional conformance allows us to define different implementations of a protocol for different types based on their characteristics. This gives us more flexibility in designing our types and protocols.
- **Convenience:** With conditional conformance, we can add conformance to a protocol for a type without modifying its original implementation. This makes it easier to add functionality to our types without breaking changes.
- **Clarity:** Conditional conformance makes our code more clear and expressive by explicitly stating the conditions under which a type conforms to a protocol. This makes it easier to understand how types and protocols work together.
- **Type safety:** Conditional conformance ensures type safety by allowing us to restrict conformance to only those types that meet certain conditions. This prevents bugs and makes our code more reliable.

Q82. Why reuseidentifier is important in UITableView?

Answer: The reuseIdentifier is an important concept in UITableView, as it optimizes performance and memory usage when displaying a large amount of data. It allows efficient cell reuse, resulting in smooth scrolling and a reduced memory footprint.

Consistency and Maintainability:

Assigning a reuseIdentifier to cells encourages consistent cell configuration. Cells with similar layouts and functionalities are grouped together, making it easier to maintain and update their appearance and behaviour.

Performance Improvement:

Reusing cells enhances scrolling performance, especially when dealing with large data sets. By recycling cells, the table view can quickly update the content and layout of the visible cells instead of continuously creating new instances.

Memory Optimization:

Creating and initializing `UITableViewCell` objects can be an expensive operation, especially if the cell contains complex layouts or custom views. By reusing cells, you can avoid the overhead of frequently creating new cells, resulting in a more efficient use of memory.

Cell Reusability:

`UITableView` reuses its cells as the user scrolls, only creating a limited number of cells that are visible on the screen. When a cell scrolls off the screen, instead of discarding it, the table view marks it for reuse. The `reuseIdentifier` is used to identify and group cells with similar layouts and configurations together.

Q83. What are the most effective ways to write clean code?

Answer:

Single Responsibility Principle (SRP)

Each class, module, or function should have a single responsibility. It should do one thing and do it well. This helps the code stay focused, readable, and easy to understand.

Don't Repeat Yourself (DRY)

Avoid duplicate code. Instead, extract common functionality from reusable methods, functions, or classes. This improves code maintainability and reduces errors when changing.

Use meaningful and descriptive names

Choose descriptive names for variables, functions, classes, and methods. This makes the code self-explanatory and helps others understand its purpose without diving into implementation details.

Open-Closed Principle (OCP)

Design code that is open for extension but closed for modification. Use techniques such as inheritance, protocols, and composition to achieve this. This allows you to add new functionality without modifying existing code.

Proper indentation and formatting

Follow indentation and formatting rules to make the code visually appealing and easy to read. Utilize automatic formatting tools or adhere to established style guides, such as SwiftLint rules.

Write unit tests

Implement unit tests to verify code behavior and ensure its correctness. Test-driven development (TDD) can be a helpful approach, where you write tests before writing code.

Keep lines of code short

Avoid excessively long code lines that require horizontal scrolling. Limit the line length to a reasonable number of characters (usually around 80-100 characters) to improve readability.

Minimize the use of abbreviations

While some abbreviations may be common in the iOS development community (e.g., **btn** for button), it's generally better to use descriptive and readable names instead of relying on abbreviations. This helps to make the code more understandable, especially for new developers joining the project.

Q84. What is final keyword in Swift? How does it improve runtime performance?

Answer: `final` keyword is used to indicate that classes, methods or properties cannot be subclassed or overridden. Here is an example of using the final keyword to stop inheriting a class:

```
// cannot inherit this class
final class Person {
    var name: String = ""
    var age: Int = 0
}
```

Here is an example of using the final keyword to stop overriding a method:

```
class Person {

    var name: String = ""
    var age: Int = 0

    // this method cannot be overridden
    final func displayDetails() {
        print("\(name) is \(age) years old.")
    }
}
```

How does final keyword improve the performance of the app?

- When a method is marked as `final`, the compiler can skip the dynamic dispatch process. This

involves checking at runtime if the correct implementation of a method has been called. This process can be expensive in terms of performance.

- Marking a class as `final` allows the compiler to perform certain optimizations, such as inlining methods and properties. This can result in improved performance by reducing the amount of memory used and avoiding method call overhead.

Q85. What is the difference between static dispatch and dynamic dispatch?

Answer: Both techniques are used to determine which implementation of a method to call at runtime in the Swift language.

Static Dispatch:

Static dispatch is a compile-time decision on which implementation of a method to use, based on the static type of the object. In Swift, when you call a method on an object, the compiler checks the type of the object and looks for an implementation of the method that is specifically defined for that type. This implementation is then used at runtime.

```
class Vehicle {
    func drive() {
        print("Driving a vehicle")
    }
}

class Car: Vehicle {
    override func drive() {
        print("Driving a car")
    }
}

let vehicle: Vehicle = Vehicle()
vehicle.drive() // Static dispatch: "Driving a vehicle"
```

In this example, the vehicle object is of type Vehicle, and the method drive() of the Vehicle class is called. The decision of which method to call is made at compile time based on the static type of vehicle.

Dynamic Dispatch:

Dynamic dispatch, on the other hand, determines the implementation of a method to use at runtime based on the actual type of the object, rather than the static type. In Swift, this can be achieved by marking a method as dynamic or by using the objc attribute to expose the method to Objective-C.

```
class Vehicle {
    func drive() {
        print("Driving a vehicle")
    }
}

class Car: Vehicle {
    override func drive() {
        print("Driving a car")
    }
}

let vehicle: Vehicle = Car()
vehicle.drive() // Dynamic dispatch: "Driving a car"
```

In this example, the vehicle object is assigned an instance of the Car class. At runtime, dynamic dispatch is used, and the method drive() of the Car class is called because the actual type of the object is Car. The decision of which method to call is determined dynamically based on the actual type of vehicle.

Here are some of the key differences between them:

- **Timing:** Static dispatch takes place at compile time, while dynamic dispatch takes place at runtime.
- **Performance:** Static dispatch is faster and more efficient than dynamic dispatch. Because which

method to call is determined at compile time.

- **Use cases:** Static dispatch is more suitable for situations when the object's type is known at compile time and the implementation of a method does not need to change based on the type of the object. Dynamic dispatch is more appropriate in situations where the type of the object is not known at compile time and the implementation of a method needs to change based on the type of the object.

Q86. Explain SOLID principles in Swift.

Answer: SOLID is a set of five design principles that aim to make iOS apps more maintainable, scalable, and easy to understand.

Single Responsibility Principle (SRP):

This principle states that a class should have only one reason to change. In other words, a class should have only one responsibility. If a class has multiple responsibilities, changes to one responsibility may affect the others, making the code difficult to maintain. To apply this principle in iOS Swift, we can create smaller, more focused classes that each have a single responsibility.

Example:

Let's say we have a class called `UserDataManager` that is responsible for managing user data in our app. However, this class is also responsible for fetching data from the network and saving data to disk. This violates the SRP, as the class has multiple responsibilities. To apply the SRP, we can create separate classes for fetching and saving data, and have `UserDataManager` only be responsible for managing user data.

Open-Closed Principle (OCP):

This principle states that a class should be open for extension but closed for modification. In other words, we should be able to add new functionality to a class without modifying its existing code. This can be achieved in iOS Swift by using protocols and extensions. By defining a protocol for a class, we

can extend its functionality without modifying the class itself.

Example:

Let's say we have a class called Shape that represents a geometric shape. We want to add a new method to this class to calculate the perimeter of the shape. However, we don't want to modify the existing code of the Shape class. To apply the OCP, we can create a protocol called PerimeterCalculable that defines the calculatePerimeter() method. Then, we can create an extension of the Shape class that conforms to this protocol and implements the calculatePerimeter() method.

Liskov Substitution Principle (LSP):

This principle states that objects of a superclass should be replaceable with objects of a subclass without affecting the program's correctness. In iOS Swift, this means that subclasses should be able to be used interchangeably with their parent class. To achieve this, we need to ensure that our subclass conforms to the same protocols and provides the same functionality as its parent class.

Example:

Let's say we have a class called Animal with a method called makeSound(). We have a subclass called Dog that overrides the makeSound() method to bark. If we replace an instance of the Animal class with an instance of the Dog class, the program should still work correctly, as the Dog class is a subtype of the Animal class. To apply the LSP, we need to ensure that the Dog class implements the same methods as the Animal class and does not change their behavior.

Interface Segregation Principle (ISP):

This principle states that clients should not be forced to depend on interfaces they do not use. In iOS Swift, this means that we should define small, focused protocols that only contain the methods and properties required by a particular client. This allows us to decouple our code and prevent unnecessary dependencies.

Example:

Let's say we have a protocol called `Vehicle` that defines methods for moving and stopping. However, we also have a class called `Boat` that does not need to implement the `stop()` method, as it does not have brakes. This violates the ISP, as the `Boat` class is forced to implement a method it does not need. To apply the ISP, we can create a separate protocol called `MovingVehicle` that defines the `move()` method, and a separate protocol called `StoppingVehicle` that defines the `stop()` method. Then, we can have the `Boat` class only conform to the `MovingVehicle` protocol.

Dependency Inversion Principle (DIP):

This principle states that high-level modules should not depend on low-level modules. Instead, both should depend on abstractions. In iOS Swift, this means that we should use protocols to define abstractions and use dependency injection to provide these abstractions to our classes. This allows us to create loosely coupled, reusable code that is easier to test and maintain.

Example:

Let's say we have a class called `DataService` that fetches data from a network API. However, this class is tightly coupled to the network library we are using, making it difficult to test and reuse. To apply the DIP, we can create a protocol called `DataFetcher` that defines a method for fetching data. Then, we can have the `DataService` class depend on this protocol instead of the concrete network library. This allows us to swap out the network library with a different implementation without affecting the `DataService` class. We can also easily create a mock implementation of the `DataFetcher` protocol for testing purposes.

Note:

- **Increased complexity:** Following SOLID principles can result in increased complexity, as it requires the creation of smaller, more focused classes and interfaces. This can make the codebase harder to understand and maintain, especially for developers unfamiliar with SOLID principles.
- **Increased development time:** Implementing SOLID principles can take more time and effort than writing simple, straightforward code. This can result in slower development cycles and increased development costs.

Q87. What is the difference between URL Schemes and Universal Links in iOS?

Answer: URL Schemes and Universal Links are both mechanisms that allow iOS apps to integrate with other apps or the web. However, there are some differences between them.

Definition:

URL Schemes are a way to launch an app from another app or from a web page using a URL.

Universal Links, on the other hand, are a way to link to a specific piece of content within an app from a website or from another app.

Security:

URL schemes can be a security risk because they can be used to launch an app and perform actions without the user's consent. Universal links, on the other hand, are more secure because they require user permission before opening the app.

User Experience:

URL schemes can provide a seamless user experience because they launch an app with just one tap. However, they may not always work if the app is not installed or if the user is not familiar with the URL format. Universal links require an extra tap to confirm the user wants to open the app. However, they provide a more reliable user experience because they can fall back to a web page if the app is not installed.

Setup and configuration:

Setting up URL Schemes requires adding a custom URL scheme to the app's Info.plist file and

handling incoming URLs in the app's code. Universal Links, on the other hand, require additional setup on the web server and in the app's entitlements file.

How do URL schemes work?

Let's say you have a music app that allows custom playlists. You could use a custom URL scheme like `"my-music-app://createPlaylist"` to allow users to create their own playlist from within another app or from a web page. When the user clicks on the link, iOS will launch your app and execute the `"createPlaylist"` action, which could take the user directly to the playlist creation screen.

URL Schemes typically use a custom URL scheme identifier followed by a colon and a set of parameters. For example, `"twitter://"` is a custom URL scheme used by the Twitter app, and `"my-music-app://createPlaylist"` is a custom URL scheme that includes the `"createPlaylist"` action as a parameter.

How do Universal Links work?

Let's say you have an e-commerce app that allows users to view product details and make purchases. You could use a Universal Link like `"https://my-ecommerce-app.com/products/1234"` to allow users to view the details of a specific product within your app from a web page or from another app. When the user clicks on the link, iOS will display a prompt asking if they want to open the link in your app. If the user confirms, your app will launch and display the product details. If the user declines or your app is not installed, the link will open in Safari or another web browser.

Universal Links, on the other hand, use a standard HTTPS URL format that includes the domain name of the website or app, followed by a path to the specific content. For example, `"https://en.wikipedia.org/wiki/Example"` is a Universal Link to a Wikipedia page, and `"https://my-ecommerce-app.com/products/1234"` is a Universal Link to a product in an e-commerce app.

Q88. Explain Equatable, Hashable, and Comparable protocol in Swift.

Answer:

Equatable Protocol

This protocol allows two objects to be compared for equality using the == operator. Equatable is also the base protocol for the Hashable and Comparable protocols, which allow more uses of your custom type, such as constructing sets or sorting the elements of a collection.

Here is an example of the Equatable protocol:

```
struct User: Equatable {  
  
    let firstName: String  
    let lastName: String  
  
    static func == (lhs: User, rhs: User) -> Bool {  
        return lhs.firstName == rhs.firstName && lhs.lastName == rhs.lastName  
    }  
}  
  
let alex1 = User(firstName: "Alex", lastName: "Bush")  
let alex2 = User(firstName: "Alex", lastName: "Daway")  
let alex3 = User(firstName: "Alex", lastName: "Bush")  
  
print(alex1 == alex2) // print: false  
print(alex1 == alex3) // print: true
```

Hashable Protocol:

This protocol enables instances of a class to be used as keys in a dictionary. To conform to Hashable, an instance must implement the hashValue property, which returns an Int. When two instances are equal, they must have the same hash value.

```
struct User: Hashable {  
    let firstName: String  
    let lastName: String  
}
```

```

let alex1 = User(firstName: "Alex", lastName: "Bush")
let alex2 = User(firstName: "Alex", lastName: "Daway")
let alex3 = User(firstName: "Alex", lastName: "Bush")

print("Alex1 hash value: \(alex1.hashValue)")
print("Alex2 hash value: \(alex2.hashValue)")
print("Alex3 hash value: \(alex3.hashValue)")

// Output:
// Alex1 hash value: 8130103028233918979
// Alex2 hash value: 1859436915308576384
// Alex3 hash value: 8130103028233918979

```

In the above example, you can see that the objects alex1 and alex3 have the same hash values because they both have identical properties.

Note: To compare the selective properties of the type, you may use the hash(into hasher: inout Hasher) function inside the type.

Comparable Protocol:

The Comparable protocol in Swift enables instances of a class to be compared for ordering. To conform to Comparable, an instance must implement the < operator, which returns a Bool indicating whether the left-hand side instance is less than the right-hand side instance.

```

struct Point: Comparable {

    let x: Int
    let y: Int

    static func < (lhs: Point, rhs: Point) -> Bool {
        return lhs.x < rhs.x && lhs.y < rhs.y
    }
}

let pointA = Point(x: 0, y: 0)
let pointB = Point(x: 1, y: 4)

```

```
let pointC = Point(x: 2, y: 2)

print(pointA < pointB) // true
print(pointB < pointC) // false
print(pointA < pointC) // true
```

Anytime Magic Tip:

- In Swift, many types conform to the Equatable protocol by default like Int, Double, String, Array etc.
- In the Hashable protocol, hash values cannot be guaranteed to be equal across different executions of your program. Do not save hash values for use during future executions.
- When conforming to multiple protocols, make sure that they are consistent with each other. For example, if two instances are equal, they should have the same hash value and have the same ordering when compared.

Q89. Explain the Liskov Substitution principle?

Answer: The Liskov substitution principle is a fundamental concept in object-oriented programming. It states that objects of a superclass should be able to be replaced with objects of a subclass without affecting the program's correctness. This principle can be applied to ensure that subclasses of a given class can be substituted with the superclass in all contexts.

To apply the Liskov substitution principle in iOS, you should ensure that the subclass conforms to the same interface as the superclass. For example, if the superclass has a method that returns a particular type of object, the subclass should also return the same type of object. Additionally, any preconditions specified by the superclass should also hold for the subclass.

Here's an example of the Liskov substitution principle:

Let's say you have a superclass called `Shape` that defines a method named `area()` that calculates

the area of the shape. You also have a subclass called `Rectangle` that inherits from `Shape` and overrides the `area()` method to calculate the area of a rectangle.

```
class Shape {
    func area() -> Double {
        return 0.0
    }
}

class Rectangle: Shape {
    var width: Double
    var height: Double

    init(width: Double, height: Double) {
        self.width = width
        self.height = height
    }

    override func area() -> Double {
        return width * height
    }
}
```

Now, let's say you have another class called `AreaCalculator` that takes an array of `Shape` objects and calculates the total area of all the shapes.

```
class AreaCalculator {
    func totalArea(shapes: [Shape]) -> Double {
        var area = 0.0

        for shape in shapes {
            area += shape.area()
        }

        return area
    }
}
```

Now, let's create some `Shape` and `Rectangle` objects and pass them to the `totalArea()` method:

```
let rectangle1 = Rectangle(width: 10, height: 5)
```

```
let rectangle2 = Rectangle(width: 6, height: 8)
let shapes = [rectangle1, rectangle2]

let calculator = AreaCalculator()
let totalArea = calculator.totalArea(shapes: shapes)

print(totalArea) // Output: 98.0
```

As you can see, the `totalArea()` method works correctly with both `Shape` and `Rectangle` objects. This is because the `Rectangle` class conforms to the same interface as the `Shape` class and does not violate the Liskov substitution principle.

Miscellaneous

Q90. What is AppClip and how is it used in iOS?

Answer: AppClip feature was introduced by Apple in the iOS 14+ versions. It allows users to access small parts of an application without having to download the full app on their device.

AppClip is designed to be secure and private, and it can only access the user's data with explicit permissions. Additionally, AppClip cannot collect data from other apps or from the user's device, and it will automatically be removed from the device after a certain period of time.

AppClips can be triggered by scanning a QR code, tapping an NFC tag, or a link sent through iMessage or the Safari browser. Once triggered, the AppClip will load and present the user with the specific functionality they need.

The use cases for AppClips:

- Paying the parking rent
- Ordering food from a restaurant
- Renting a bicycle
- Signing up for a subscription and many more...

Advantages of AppClips:

- **Convenience:** AppClip allows users to access specific features of an app quickly and easily.
- **App Visibility:** AppClip helps to increase app visibility with an innovative way to promote their app-based services among customers.
- **Security:** AppClip is designed to be secure and private. It cannot access the user's data without their permission.
- **Cost Effective:** AppClip reduces the development cost of an application and promotes the application idea with customers, making it cost-effective.

Limitations of AppClip:

- Similar to the full application, AppClip needs to be reviewed by Apple by following the strict guidelines before available to users on AppStore.

Q91. What are Property Qualifiers in Swift?

Answer: In Swift, a property qualifier is a keyword that can be used to modify the behavior of a property. Swift offers several property qualifiers:

- **lazy:**
This qualifier delays property initialization until it is first accessed. This is useful for properties that are expensive to initialize or that may not be needed.
- **weak:**
This qualifier is used to create a weak reference to an object. This is useful when you want to avoid creating a strong reference cycle between objects.
- **unowned:**
This qualifier creates an unowned reference to an object. This is similar to a weak reference, but it is assumed that the object being referenced will never be nil.
- **private:**
This qualifier makes a property accessible only within the same source file.
- **fileprivate:**
This qualifier makes a property accessible only within the same file.
- **internal:**
This qualifier makes a property accessible within the same module.
- **public:**
This qualifier makes a property accessible from any source file or module.

- **open:**

This qualifier makes a property accessible and overridable from any source file or module.

- **mutating:**

This is used to indicate that a method or function can modify the properties of a struct or an enum. By default, methods and functions of a struct or an enum cannot modify stored properties because the instance of the struct or the enum is immutable.

- **static:**

This qualifier is used for type-level properties that are shared among all instances of a class or structure. Static properties are associated with the type itself rather than individual instances.

- **class:**

This qualifier is similar to static that properties are also associated with the type itself, but class properties can be overridden by subclasses.

- **final:**

This qualifier is used to prevent a property, method, or entire class from being overridden or subclassed. When applied to a property, it ensures that the property's implementation cannot be changed in any subclass.

Q92. What is APNS and how does it work?

Answer: APNS (Apple Push Notification Service) is a cloud service that allows the application's server to send notifications to the device when a new event triggers. It works by establishing a persistent connection between an iOS device and Apple's server.

When an app wants to send a notification, it sends the message to the APNS server, which then delivers the message to the target device. The device displays the notification, which can include text, sound, or an icon badge, to the user. APNS requires a secure connection and uses a combination of certificate-based authentication and encryption to ensure that only authorized applications can send notifications to iOS devices.

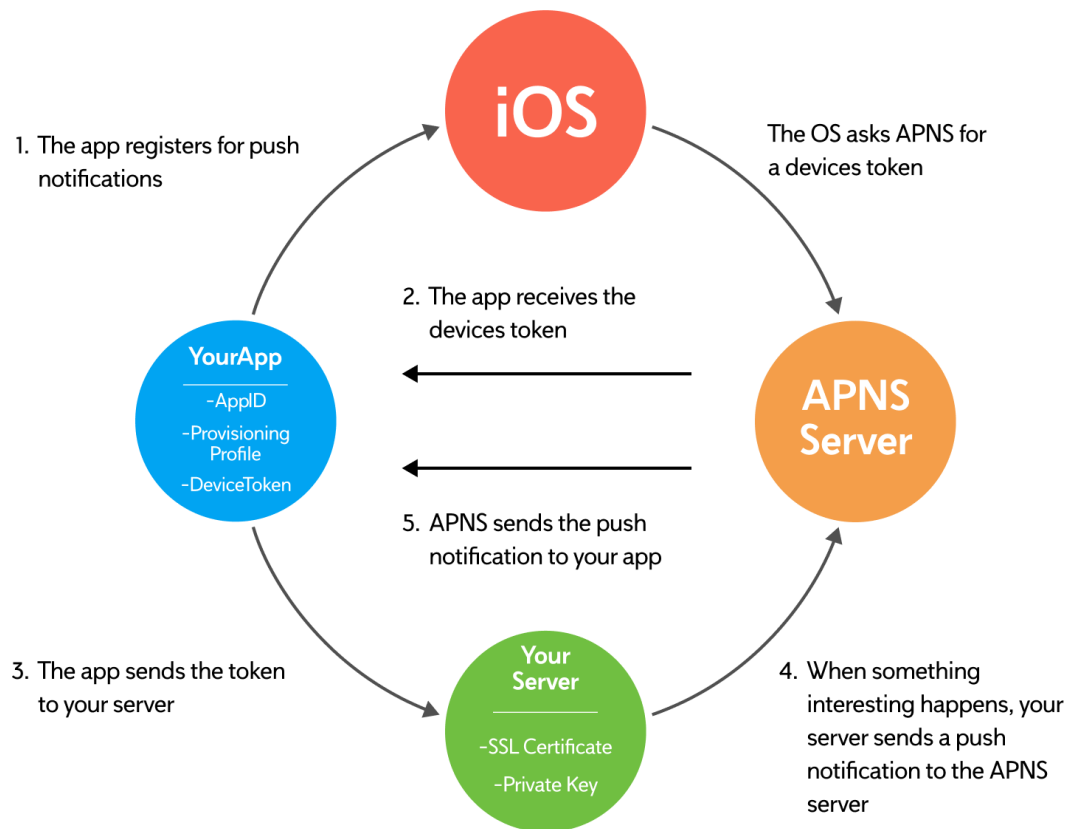
How APNS works:

- Enable push notification by asking user's permission to send notifications.
- Once a device registered for push notifications, application's server sends a push notification request to the APNS server, including the target device token and the notification payload.
- The APNS server receives the request and sends the notification to the target device over a persistent and secure connection.
- The device receives the notification and displays it to the user, either as an alert, a banner, or a sound, depending on the user's settings and the type of notification.
- If the user interacts with the notification, such as tapping it, the app associated with the notification launches and receives the user's response.

Note: The device token for one app can't be used for another app, even when both apps are installed on the same device. Both apps must request their own unique device token and forward it to your provider's server.

The delivery of remote notifications involves several key components:

- Your application's server, known as the provider server
- Apple Push Notification service (APNs)
- The user's device
- User has allowed notifications for your app on the device



Important: In your developer account, enable the push notification service for the App ID assigned to your project.

Counter Question:

Q: What are silent/background push notifications and when are they implemented?

Answer: Silent/background push notifications are the notifications that wake up the app in background and doesn't show an alert, badge number or play a sound. The OS considers these notification as low priority task and it may restrain the delivery of these notifications if the count is excessive. As per Apple's recommendation, the frequency of these notifications should not be more than 2-3 notifications per hour.

When to use Silent/Background Notifications:

1. Background App Refresh:

Silent push notifications can be used to call background refresh, allowing apps to update their content even when they are not actively running.

2. Geolocation:

Some apps that use location based services can use these notifications to monitor a user's location in the background. For example, food delivery apps can send a personalised offer notification when user enters a particular location.

3. Content Synchronisation:

Messaging apps can use silent notifications to fetch new messages or updates in the background. This keeps the app's content always updated even without app being opened by the user.

Some Related Questions:

- What is a device token, and how is it used in APNS?
- How does an iOS app handle push notifications when it is in the foreground?
- What are the different types of notifications that can be displayed on an iOS device?
- Can you describe the structure of a typical notification payload in APNS?

Q93. What are the Background modes in iOS?

Answer:

In iOS, there are several background modes that allow apps to continue functioning even when they are not actively running in the foreground. Here are some common background modes in iOS and how they work:

Audio:

This background mode allows apps to continue playing audio even when they are not in the foreground. For example, a music streaming app can continue playing music while you use other apps or lock your device.

Location Updates:

This background mode allows apps to track your location even when they are not in the foreground. For example, a navigation app can continue to provide turn-by-turn directions while you use other apps.

Background Fetch

This background mode allows apps to periodically fetch new content or data in the background. For example, a news app can fetch new articles in the background so that they are ready for you to read when you open the app.

Remote Notifications:

This background mode allows apps to receive and process remote notifications in the background. For example, a messaging app can receive and display new messages even when it is not in the foreground.

VoIP:

This background mode allows apps to receive and handle VoIP (Voice over Internet Protocol) calls in the background. For example, a video conferencing app can continue to receive and handle calls even when it is not in the foreground.

Background Processing:

The Background Processing allows apps to perform long-running tasks in the background, even if the app is not actively running in the foreground. This is useful for apps that need to perform tasks like uploading or downloading large amounts of data, or processing images or videos.

External Accessory:

The External Accessory allows apps to communicate with external hardware accessories in the background, even if the app is not actively running in the foreground. This is useful for apps that need to interact with accessories like fitness trackers, heart rate monitors, or car audio systems.

Counter Question:

Q: How do Background modes work?

Answer: When an app uses one of these background modes, it is still running in the background, but it may be using less resources than when it is in the foreground. The exact behavior of an app in the background depends on the specific background mode it is using and how it is designed.

Q94. Explain how iOS app state restoration works.

Answer: App state restoration in iOS allows an app to save its current state, such as the user's location, app preferences, or the current view hierarchy, and restore it when the app is relaunched. This feature provides a seamless user experience by allowing the user to pick up where they left off in the app, even if the app was terminated in the background.

Here are the general steps to implement app state restoration in iOS :

1. **Enable state preservation and restoration in your app:** You can do this by adding the `UIApplication` method `application(_:shouldSaveApplicationState:)` and `application(_:shouldRestoreApplicationState:)` in your app delegate. In these methods, you can specify which parts of the app's state you want to save and restore.
2. **Identify the objects you want to save and restore:** You can use the `encodeRestorableState(with:)` and `decodeRestorableState(with:)` methods in your view controllers or other objects to save and restore their state.
3. **Save the app's state:** When the app is about to be terminated, iOS calls the

`application(_:willEncodeRestorableStateWith:)` method on your app delegate. In this method, you can encode and save any relevant data that you want to restore later.

4. **Restore the app's state:** When the app is relaunched, iOS calls the `application(_:didDecodeRestorableStateWith:)` method on your app delegate. In this method, you can decode the saved data and use it to restore the app's state.
5. **Update the app's user interface:** After the app's state has been restored, you may need to update the user interface to reflect the restored state. You can do this by calling methods on your view controllers or other objects to update their state.

Overall, app state restoration requires a bit of setup, but it can greatly enhance the user experience by allowing users to easily resume where they left off in the app.

Q95. What are important factors to be considered for improving iOS App's performance?

Answer: iOS users expect apps to perform well, there are many factors which can hamper their experience and may lead them to uninstall the app. As a developer we should consider every measure to improve user experience. We are going to discuss three important factors which hamper user experience.

- App Size
- Battery Consumption
- Memory usage
- Crashes

App Size -

When a user is going to download an app the App Size plays an important role in making the decision whether any alternative option is available with lesser App Size. A developer can control the App Size by

- **Removing unwanted resources**

Resources keep on growing as the app grows. We need to revisit them once to check if we are not adding duplicate resources. This practice can bring down our App Size by a significant amount.

- **Make use of server hosted image assets**

We should always consider hosting the image assets on the server so that we don't need to package them with the binary which is the. An app binary is a file that contains machine code for a computer to execute.

- **App Thinning**

Apple launched App Thinning from iOS 9 onwards which helped developers in significant reduction to the App Size. App thinning is a process of delivering an IPA file which consists of resources or code that is necessary to run the app on a particular device. It majorly consists of three processes: Slicing, On-Demand Resources, BitCode.

1. **Slicing:** It is a technique of App Thinning performed by AppStore which ensures to install relevant assets on a particular device. If a user is downloading an iPad version of the App then only those assets which are specifically used in iPad will be downloaded.
2. **On-Demand Resources:** It is a technique of App Thinning which ensures that resources which belong to features which are not available to a user without completing any specific task or scenario are not downloaded during the installation process. One can think of a gaming app where resources for level 2 and 3 are not downloaded if the user has not completed level 1.
3. **BitCode:** It is an intermediate representation of a compiled program, it is a prerequisite for Slicing for that you need to enable bitcode in Xcode, then Xcode will generate bitcode binary which when uploaded will be recompiled by AppStore to generate small binaries for slicing to happen.

Battery Consumption:

Battery consumptions are most affected by high CPU usage and second to CPU usage more

networking activities consume more battery. To optimise CPU usage we can follow multiple steps which involve:

- Using GCD to dispatch tasks which are heavy or long running to a different thread other than Main thread as doing costlier tasks on Main thread hampers UI performance as Main thread responsible for making UI updates.
- Whatever the process you start make sure to finish it, if a process is running continuously use more CPU and drain more battery.
- Sometimes heavy animations can also use higher CPU which will again hamper performance and drain battery make sure to simplify the animation.

To optimise Network calls we can follow these steps:

- Make sure to reduce the number of API calls if you are calling them frequently.
- There will be few API calls which might depend on the User's role make sure that you don't call them.
- While doing pagination always check for the last page so that you can avoid the last API call.

Memory Usage:

The memory (RAM) is also shared by other iOS applications and system processes as well. Operating system ensures an app which is using high memory will be sent a warning or may even terminate when in background as memory is a shared resource which has to cater to other applications and processes.

To reduce memory usage we can follow these steps:

- Make sure that objects that are getting created are deallocated when their use is over, ARC automatically handles deallocation of those objects but there are cases when a strong reference cycle is there then those objects will always remain in memory and increase memory usage. So try to break strong reference cycles.
- Images which are getting downloaded from the server can have higher resolution. Make sure to download the image and resize it according to your requirement as higher resolution images consume more memory.
- Caching images on NSCache can also lead to more memory usage. Try to remove images from cache if the usage is over.

Crashes:

Crashes occur when there are runtime errors, app becomes unresponsive due to high CPU usage, code exception. If an application crashes then it is a really bad experience for a user and the user might remove your app if it crashes frequently. You should make sure to reduce App crashes.

There are few steps which you can follow:

- Make sure to safely unwrap optional objects instead of force unwrapping as safe wrapping will ensure that objects won't be nil and we won't get code exceptions.
- Make sure to reduce CPU usage as when it is more than 100% and for a longer time than it will lead to crash. Make sure to optimise processes which are continuously using CPU.
- Make sure to check crash reports given by TestFlight or third party tools like Firebase Crashlytics they give insights to the crash to resolve it.

Q96. Explain the UIViewRepresentable protocol and its use in SwiftUI.

Answer: The `UIViewRepresentable` protocol is a fundamental part of SwiftUI, introduced to facilitate the integration of UIKit components into SwiftUI views. There are instances where you might want to leverage existing UIKit components or take advantage of certain features not yet available in SwiftUI. This is where `UIViewRepresentable` comes into play.

`UIViewRepresentable` is a bridge between SwiftUI and UIKit. It allows you to wrap a UIView (or its subclass) so that it can be used as a SwiftUI view. By adopting the `UIViewRepresentable` protocol, you create a two-way binding between the UIKit component and SwiftUI, enabling seamless integration and communication between the two frameworks.

To create a `UIViewRepresentable`, you need to implement two essential methods:

- **`makeUIView(context:)`**: This method is responsible for creating and configuring the underlying UIKit view. You create the desired UIView subclass, set up its properties, and return it.
- **`updateUIView(_:context:)`**: This method gets called whenever the SwiftUI view representing the `UIViewRepresentable` needs to be updated. Here, you update the UIView's properties based on

the changes in SwiftUI's state.

Here's a basic example of a `UIViewRepresentable` that wraps a `UILabel`:

```
import SwiftUI

struct UILabelWrapper: UIViewRepresentable {
    var text: String

    func makeUIView(context: Context) -> UILabel {
        let label = UILabel()
        label.textAlignment = .center
        return label
    }

    func updateUIView(_ uiView: UILabel, context: Context) {
        uiView.text = text
    }
}
```

With this `UIViewRepresentable`, you can now use it as a SwiftUI view:

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        UILabelWrapper(text: "Hello, World!")
    }
}
```

In this example, the `UILabelWrapper` represents a `UILabel` in SwiftUI. The `text` property is a binding, and whenever it changes, SwiftUI will automatically call `updateUIView(_:context:)` to update the underlying `UILabel`.

`UIViewRepresentable` is invaluable when you want to use any custom `UIView` or even a complex third-party UI component in your SwiftUI app. It allows developers to mix and match UIKit's flexibility with SwiftUI's declarative nature, offering a smooth migration path from UIKit to SwiftUI.

Q97. What is inout Parameter in Swift?

Answer: The **inout** parameter in Swift allows you to pass a parameter to a function by reference, enabling the function to modify the value of the parameter directly.

By default, function parameters are constants, which means you cannot modify their values inside the function body. However, using the **inout** keyword, you can achieve two-way communication between the calling code and the function, allowing changes to persist beyond the function call.

Let's take an example to understand this better:

```
var number = 6

func doubleNumber(num: inout Int) -> Int {
    num *= 2
    return num
}

print(doubleNumber(num: &number))
// Prints 12

print(number)
// Prints 12
```

In this example, we have a variable `number` with an initial value of 6. We define a function `doubleNumber`, which takes an `inout` parameter `num`. Inside the function, we multiply the `num` by 2, effectively doubling its value. By using the `inout` keyword, the changes made to the parameter `num` are reflected back to the original variable `number`.

When calling a function with an `inout` parameter, you need to pass the argument with an ampersand (&) directly before the variable's name, indicating that it can be modified by the function.

The `inout` parameter is powerful and allows you to modify values directly within the function without the need to return them explicitly. It is particularly useful when you want to change the state of a variable and have that change propagate back to the calling scope. However, there are some rules to

consider:

- You can only use a variable (var) to pass as an argument for an `inout` parameter, not a constant (let). Constants are immutable, and using them as `inout` arguments would result in a compile-time error.
- In-out parameters cannot have default values. When you define a parameter as `inout`, you are indicating that the variable must be passed by reference explicitly.

Overall, **inout** parameters provide a way to have mutable function arguments, creating more flexible and interactive functions that can modify their input values directly.

Q.98 How can you extend or modify the UIResponder chain in your application?

Answer: You can modify the UIResponder chain in an iOS application by changing the order of objects in the chain or by adding or removing objects from the chain. Here are some ways to modify the chain:

Step 1 - Change the order of objects in the chain: The order of objects in the chain determines the order in which they receive events. You can change the order by changing the order in which the objects are added to their parent view. For example, if you want a specific view to receive events first, you can add it to the parent view before other views.

Step 2 - Add or remove objects from the chain: You can add or remove objects from the chain by overriding the UIResponder methods `canBecomeFirstResponder`, `becomeFirstResponder`, `canResignFirstResponder`, and `resignFirstResponder` in your custom classes. These methods determine whether an object can become or resign from the first responder, and they allow you to modify the UIResponder chain accordingly.

Step 3 - Override touch event handling methods: You can also modify the UIResponder chain by overriding touch event handling methods such as `touchesBegan(:with:)`, `touchesMoved(:with:)`, and `touchesEnded(_:with:)` in your custom classes.

Overall, modifying the UIResponder chain can be a powerful way to customise your iOS application's

event-handling behaviour and to create more engaging and responsive user interfaces.

System Design Round

Q99. How to Approach System Design round?

Answer: The system design round is more about discussion and collaboration skills than judgement based on what is right or wrong.

Read the statement carefully:

When the interviewer gives the problem statement, try to read it carefully. Sometimes a similar problem comes with little twists or different requirements. So read and discuss what exactly the problem statement says is the best policy here.

Requirement Gathering:

Never assume anything if you know real-life examples of the problem or system you are asked to design. Always ask or tell the interviewer that you are assuming this and why you are assuming it.

Finalise the Scope:

Then try to note down the requirements and finalise the scope of the problem. This is very important because if scope is not defined, maybe you spend most of the time designing something that is not required or that the interviewer does not want to discuss.

Design HLD:

High level Design, also known as macro-level design, describes the overall description/architecture of the application. It includes a description of system architecture, database design, a brief description of systems, services, platforms, and the relationships among modules.

- After requirement gathering, it's time to take 5–10 minutes and start brainstorming to design the system.
- The best way is to identify the components that are required to design the system and are part

of the system.

- Design how these components interact with each other. How the data was transferred between them.
- With the help of a use case diagram or other visual representation, you can explain your design, which is, in my experience, the best way.
- The most important part of designing the HLD is communicating to the interviewer why I created these components or why I did not go with another approach, if any other approach is also on your mind.
- This part also helps you identify the interest of the interviewer and which part the interviewer wants to deep-dive into further.

Points to consider while designing:

- The solution should be loosely coupled, which helps make code more scalable to add more features and testability.
- Modularize and segregate properly so that each component takes charge of its responsibility and solves a part of the problem.

Design LLD:

Low Level Design, also known as micro-level or detailed design, describes a detailed description of each and every module, which means it includes actual logic for every system component and goes deep into each module's specification.

After creating the HLD, most interviewers stop here, or if time allows, ask more specific questions about a component you created in the HLD.

Then this is a place where you can code the actual component or create flow charts or class diagrams to create LLD. which is again very helpful and the best way to discuss.

Here, sometimes the interviewer asks about the API design for the particular case. He may be more interested in knowing how to contribute to the design of an API for a particular client.

Point while designing LLD:

- Implement solid principles.
- Choose the right design patterns to solve a particular problem and explain why you chose them.
- Memory management, if applicable in your case.
- Multithreading approach to efficiently solve the problem and also handle issues that arise due to threading concepts.
- Space and time complexity are sometimes asked about.
- Access modifier
- Error handling and exception handling

The last part is to analyse the pros and cons of the system you proposed and discussed in the round.

As you know, nothing is perfect, and so is your solution. You should know the pitfalls of your design.

As software development is an iterative process, you always have the option to enhance the existing design. It would be good if you suggested some solutions that might overcome the limitations of your design.

Sometimes we use singletons to achieve some database read-write handling; it has its own benefits, but we agree that with singletons, we may face some major issues if not analysed properly. Like singletons, they will reside in memory, are never dead, and are not thread-safe.

Q100. For any OTT platform like Amazon Prime, Create a Downloader that can download and save the video to watch offline. The user can pause, resume, delete, or cancel the video that is downloading.

Answer: If you see the above statement, the first thing you need to do is analyse it and gather the information that supports it.

Step 1: Ask questions

Questions like:

- What is the maximum and average size of the downloaded video files in MB?
- Which formats are supported by the App to download?
- How many videos the downloader can download at a time.
- Maximum video length of video download support
- Does Downloader have a mechanism to automatically delete expired or outdated downloads to free up storage space?

So if you see, this question is basically helping you find the scope on which you will discuss in the rest of the interview, and also understand the actual requirement.

Step 2: Design HLD

As discussed above, you first need to find the components and then find a relationship between them.

- In this downloader, we see that we need a **network manager** that actually connects to the server to download the video.
- Then we need a **Storage manager** that saves the actual download file on the device or finds the saved file. (this may depend on file management or document directory management.)
- Also have **user management**, which will check the user account type, for example, if the user account is allowed to download or what the policy is for storing, let's say, 2-3 day expiry periods only.

If we save an encrypted file, then we need an **encryptor** to save the encrypted file and a **decryptor** to play the downloaded video.

In terms of UI,

- **Download Button:** Add a download button or icon next to each video or content item in the app's interface to allow users to initiate the download process.
- **Download Progress:** Show a progress bar or indicator to display the download progress to users.
- **Downloaded Content Library:** Include a section in the app where users can access their downloaded content for offline viewing.

Step 3: Design LLD

After creating a high level system, you need to identify, or sometimes the interviewer starts discussing, the particular components that go from your design to deep down.

There, you may be asked to write some class diagrams or flow charts that show the interaction between modules.

Now, while creating these classes, you may explain the use of threads if required, the access modifier, generics and protocol-oriented programming, testable code, scalability, and most importantly, the solid principles of which you take care while defining classes and their relations.

You need to create a downloader which is responsible for getting the url and returning the state of video downloading like progress, pause, complete, failed, etc.

Now the downloader needs a store manager to store a file that is downloaded from the server, which requires encryption to securely store the file on the device.

You need to maintain a queue, which may contain more than one downloading task, to support 2-3 video downloads in parallel.

In this process, try to engage and communicate with the interviewer. Explain why you should try this approach and why you should not use it.

Anytime Magic Tip:

Most of the time, there are chances that the system you design may not complete an interview. But the best you can do is sometimes let the interviewer know the pros and cons of the solution or proposed system.

For example, there may be a chance that the same video is downloaded multiple times. We should manage a map that checks that the URL is not repeated while downloading.

Does video save in encrypted format?

Some Related Questions:

- Discuss the Twitter Home page feed and posting feature - The user can see their followers posts and also publish their own.

Rock your next iOS Interview 🚀
- Swift Anytime